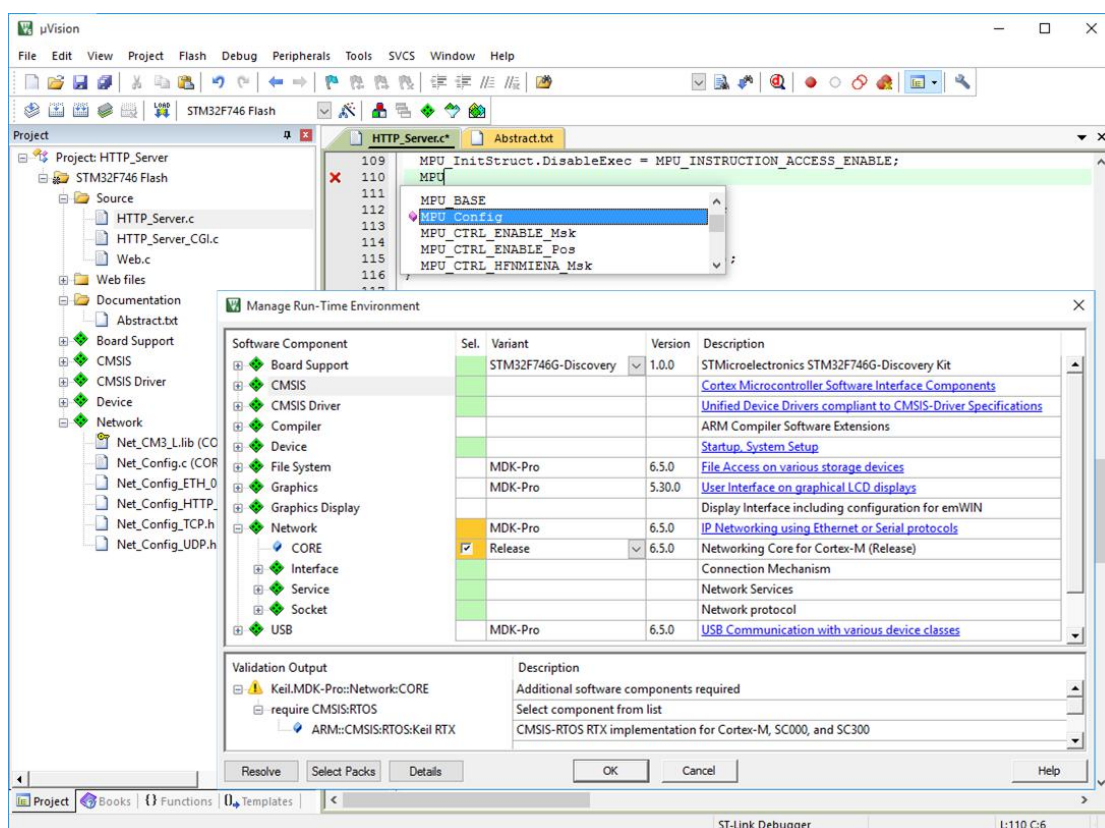


Getting Started with MDK

Create Applications with μ Vision®

for ARM® Cortex® -M Microcontrollers



版权所有©1997-2016 ARM Germany GmbH。

Keil®, μ Vision®, Cortex®, CoreSight™和 ULINK™是德国 ARM 有限公司和 ARM 公司的商
标或注册商标。

Microsoft®和 Windows™是 Microsoft 公司的商标或注册商标。

PC®是 IBM 公司的注册商标。

提示:

我们假设您熟悉 Microsoft Windows，硬件和 ARM®Cortex®-M 处理器的指令集。

尽一切努力确保本手册的准确性，并给予适当的对本文引用的人员，公司和商标的信誉。

imyumeng@qq.com

Translated By YuMeng

2017.6.25

前言

感谢您使用可从 ARM®Keil®获得的 MDK Version 5 微控制器开发套件。为了给您提供最好的软件工具来开发基于 ARM®Cortex®-M 处理器的嵌入式应用程序，我们设计了我们的工具，使软件工程变得简单而高效。ARM 还提供互补产品，如 ULINK™调试和跟踪适配器以及一系列评估板。MDK 可以使用各种第三方工具、入门套件和调试适配器进行扩展。

章节概述

本书从安装 MDK 开始，并介绍了软件组件以及从项目启动到硬件调试的完整工作流程。它包含以下章节：

MDK 简介(MDK Introduction)提供了有关 MDK 工具，软件包的概述，并介绍了产品安装以及示例项目的使用。

CMSIS 是基于 Cortex-M 的微控制器上运行的嵌入式应用程序的软件框架。它提供一整套软件接口和的硬件抽象层，用来简化软件的重用。

软件组件(Software Components)可以对各种标准 I/O 通道进行 I/O 功能的重新定位，并为各种评估板增加板卡支持。

创建应用程序(Create Applications)可以指导您使用 CMSIS 和与设备相关的软件组件来创建和修改项目。实践教程会显示用于设置工具选项的主要配置对话框。

调试应用程序(Debug Applications)描述了在实际硬件上调试应用程序的过程，并介绍了如何使用各种调试适配器连接到开发板。

中间件(Middleware)提供了可用于 MDK-Professional 和 MDK-Plus 版本的用户的中间件的更多详细信息。

使用中间件(Using Middleware)解释如何创建使用 MDK-Professional 和 MDK-Plus 可用的中间件的应用程序，并包含重要提示和技巧，以便您快速启动。

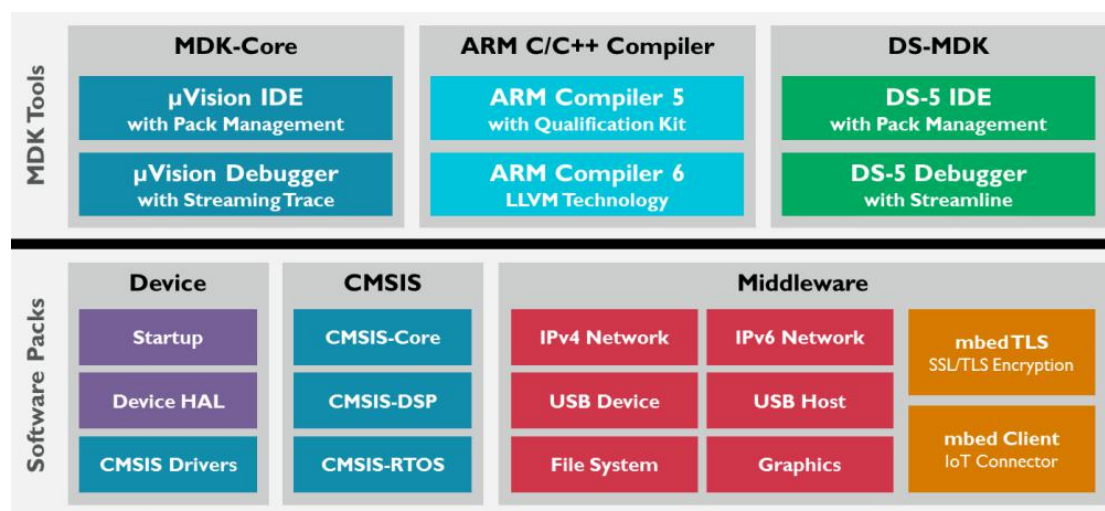
目录

MDK 简介(MDK Introduction)	1
MDK 工具	1
软件包	1
MDK 版本	2
安装	2
软件和硬件要求.....	2
安装 MDK Core.....	2
安装软件包.....	3
MDK 专业试用许可证.....	3
使用示例项目验证安装.....	4
使用软件包.....	7
访问文档.....	10
请求协助.....	10
学习平台.....	10
快速入门指南.....	11
CMSIS	11
CMSIS-CORE	12
使用 CMSIS-CORE.....	12
CMSIS-RTOS RTX	14
软件概念.....	15
使用 CMSIS-RTOS RTX.....	15
CMSIS-RTOS 系统和线程查看器.....	21
CMSIS-DSP	22
CMSIS Driver(CMSIS 驱动)	23
配置.....	24
验证.....	25
软件组件	25
编译器.....	25
板支持.....	26
创建应用程序	27
CMSIS-RTOS RTX 闪灯项目	27
设置项目.....	27
配置设备时钟频率.....	29
自定义 CMSIS-RTOS RTX 内核.....	29
创建源代码文件.....	30
构建应用程序映像.....	33
设备启动变化.....	33
例子: 英飞凌 XMC1000 使用 DAVE.....	33
例子: STM32Cube.....	36
调试应用程序	39
调试器连接.....	39

使用调试器.....	40
调试工具栏.....	41
命令窗口.....	41
汇编窗口.....	41
断点.....	42
观看窗口.....	42
调用堆栈和本地窗口.....	43
寄存器窗口.....	43
内存窗口.....	43
外设寄存器.....	44
跟踪.....	44
跟踪串行线输出.....	45
跟踪例外.....	46
事件查看器.....	47
逻辑分析仪.....	48
调试（printf）查看器.....	48
事件计数器.....	49
4 引脚跟踪输出.....	49
跟踪片上跟踪缓冲区.....	50
中间件.....	50
网络组件.....	51
文件系统组件.....	52
USB 组件.....	52
图形组件.....	53
物联网 IoT.....	54
迁移到中间件版本 7.....	54
核心变化.....	54
配置文件更新.....	54
API 更改.....	55
FTP 服务器示例.....	55
使用中间件.....	56
USB 设备 HID 示例.....	57
添加软件组件.....	57
配置中间件.....	58
配置驱动程序.....	60
调整系统资源.....	60
实施应用功能.....	61
构建和下载.....	64
验证和调试.....	64

MDK 简介(MDK Introduction)

Keil 微控制器开发套件（MDK）可帮助您为基于 ARM Cortex-M 处理器的设备创建嵌入式应用程序。MDK 是一个强大但易于学习和使用的开发系统。MDK 由 MDK Core plus 和相应的设备的软件包组成，可根据您的应用要求进行下载和安装。



MDK 工具

MDK 工具包括为基于 ARM 的单片机设备的创建、构建和调试嵌入式应用程序所需的所有组件。MDK-Core 基于正版 Keil µVisionIDE / Debugger，主要支持基于 Cortex-M 处理器的微控制器设备，包括新的 ARMv8-M 架构。DS-MDK 包含基于 Eclipse 的 DS-5 IDE / Debugger，并为基于 32 位 Cortex-A 处理器或具有 32 位 Cortex-A 和 Cortex-M 处理器的混合系统的设备提供多处理器支持。

MDK 包括两个具有汇编器、链接器和高度优化运行时库的 ARM C / C ++编译器，可根据最佳代码大小和性能进行定制：

- ARM 编译器版本 5 是具有 TÜV 认证资格套件和长期支持和维护的参考 C / C ++编译器。
- ARM 编译器版本 6 基于创新的 LLVM 技术，并支持最新的 C 语言标准，包括 C ++ 11 和 C ++ 14。

软件包

软件包包含驱动支持、CMSIS 库、中间件、板支持、代码模板和示例项目。它们可以随时添加到 MDK Core 或 DS-MDK，使得新的设备支持和中间件更新独立于工具链。IDE 可管理提供应用程序的软件组件作为构建块。

MDK 版本

产品选择器，可在 <http://www.keil.com/mdk5/selector> 查看，这里概述一下各版功能特点：

- **MDK-Professional** 专业版包含 **MDK-Plus** 的所有功能。此外，它支持 IPv4 / IPv6 双栈网络、物联网连接和一个 USB 主机堆栈。一旦可用，MDK 专业版包括 ARMv8-M 架构支持和许可 DS-MDK。
- **MDK-Plus** 版包含中间件库 IPv4 网络、USB 设备、文件系统、图形界面。它支持 ARM Cortex-M 系列，选择 ARM Cortex-R、ARM7、和基于微控制器的 ARM9 处理器。
- **MDK-Cortex-M** 版支持 Cortex-M 处理器的微控制器。
- **MDK-Lite** 是代码大小限制为 32 字节，用于产品评估、小项目、与教育市场。

许可证类型

MDK-Lite 版除外各个版本都需要使用许可证代码进行激活。以下许可证类型可用：

单用户许可证书(Single-User License)（节点锁定）授权一个开发人员同时在两台计算机上使用该产品。

自由用户许可证书(Floating-User License)或 **FlexLM 许可证书**授予多个开发人员同时多台计算机上使用该产品。

7 天 MDK 专业测试许可证书(7-Day MDK-Professional Trial License)，用于测试没有代码大小限制的综合中间件。

有关更多详细信息，请参阅 www.keil.com/support/man/docs/license 上的许可用户指南。

安装

软件和硬件要求

MDK 具有以下最低硬件和软件要求：

运行 Microsoft Windows（32 位或 64 位）操作系统的 PC

4 GB RAM 和 8 GB 硬盘空间

1280 x 800 或更高的屏幕分辨率；鼠标或其他指点设备

安装 MDK Core

从 www.keil.com/download 下载 MDK5 并运行安装程序。

按照说明在本地计算机上安装 MDK Core。该安装还添加了用于 ARM CMSIS 和 MDK 中间件的软件包。

MDK5 在安装 www.keil.com/mdk5/legacy 上的 Legacy Support 兼容包后，能够使用 MDK 版本 4 项目。并且增加了对 ARM7，ARM9 和 Cortex-R 处理器的设备的支持。

MDK Core 安装完成后，软件包安装程序将自动启动，从而允许您添加补充软件包。您至少需要安装一个支持您的目标微控制器设备的软件包。

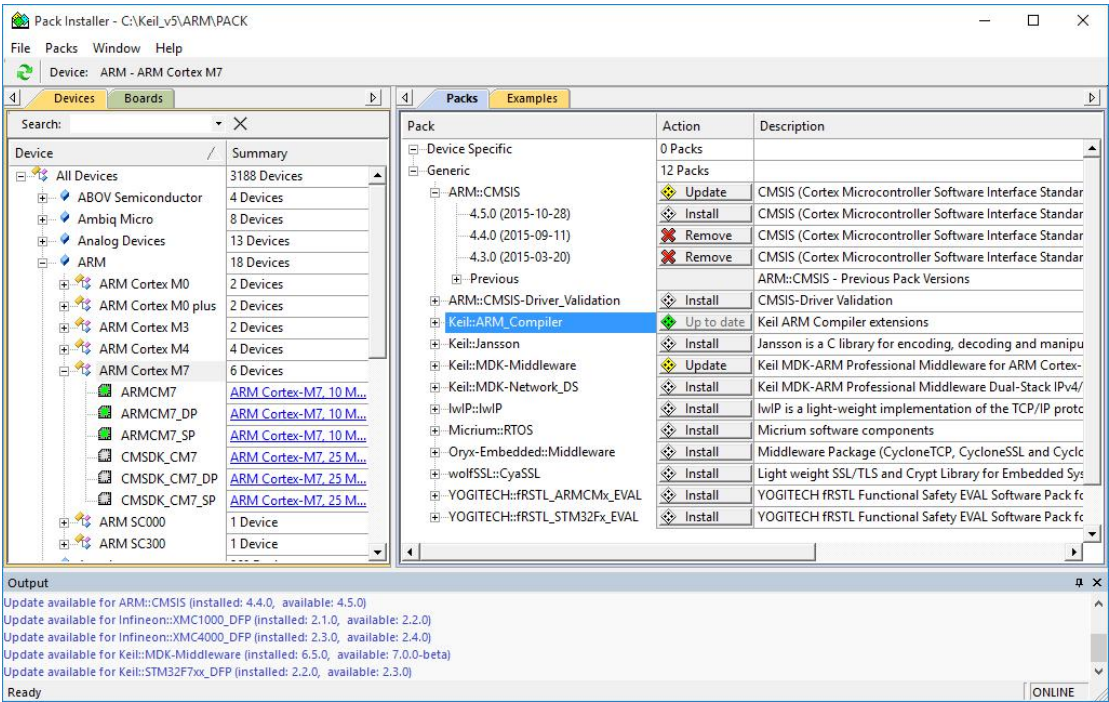
安装软件包

包安装程序(Pack Installer)是用于在本地计算机上管理软件包的实用程序。

 Pack Installer 在安装过程中自动运行，也可以从 μ Vision 中使用菜单项 **Project - Manage - Pack Installer** 运行。要访问设备和示例项目，您应该安装与目标设备或评估板相关的软件包。

注意

软件包安装程序会连接到 www.keil.com/pack 获取已发布软件包的信息。



位于 Pack Installer 底部的状态栏显示有关 Internet 连接和安装进度的信息。

提示: www.keil.com/dd2 上的设备数据库列出了所有可用设备，并提供了相关软件包的下载访问权限。如果 Pack 安装程序无法访问 www.keil.com/pack，则可以使用菜单命令“文件-导入(File-Import)”或双击*.PACK 文件手动安装软件包。

MDK 专业试用许可证

MDK 内置三天免费的 MDK 专业试用许可证。这将删除代码大小限制，您可以浏览和测试全面的中间件。

启动具有管理权限的 μ Vision。

1. 在 μ Vision 中，转到 **File - License Management...**，然后单击评估 **Evaluate MDK Professional**。

License Management

Single-User License | Floating License | Floating License Administrator | FlexLM License

Customer Information

Name:

Company:

Email:

Computer ID

CID:

Get LIC via Internet...

Product	License ID Code...	Support Period
MDK-Lite	Evaluation Version	

New License ID Code (LIC):

Add LIC Uninstall...

Evaluate MDK Professional Close Help

2. 在下一个屏幕上，单击启动 **Start MDK Professional Evaluation for 7 Days**。安装完毕后，屏幕会显示有关有效日期和时间的信息。

注意

7 天激活的 MDK Professional 试用版可以在 FlexLM 许可证的选项卡中使用 **Flex Server**，因此此许可证基于 **FlexLM**。

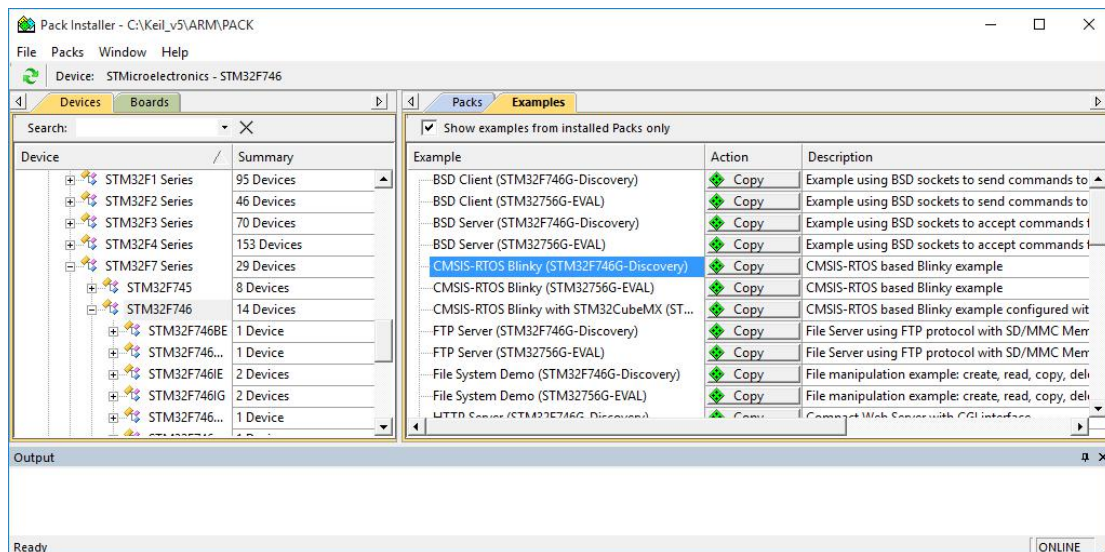
使用示例项目验证安装

一旦您为您的设备选择、下载、安装软件包好后，您可以使用软件包中提供的示例之一来验证安装。要验证软件包安装，我们建议使用 **Blinky** 示例，通常会在目标板上闪烁 LED。

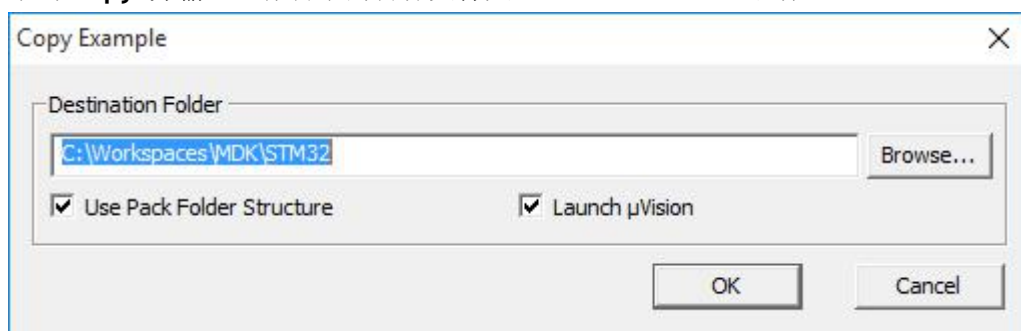
提示： 查看 <http://www.keil.com/mdk5> 上的入门视频，介绍如何连接和使用评估套件。

复制示例项目

 在包安装程序中，选择 **Examples** 选项卡。使用工具栏中的过滤器缩小示例列表。



单击 **Copy** 并输入工作目录的目标文件夹 **Destination Folder** 名称。



注意

您必须将示例项目复制到您选择的工作目录。

启用 **Launch µVision** 直接在 IDE 中打开示例项目。

启用 **Use Pack Folder Structure** 将示例项目复制到公用文件夹中。这样可以避免覆盖其他示例项目中的文件。禁用 **Use Pack Folder Structure** 以减少示例路径的复杂性。

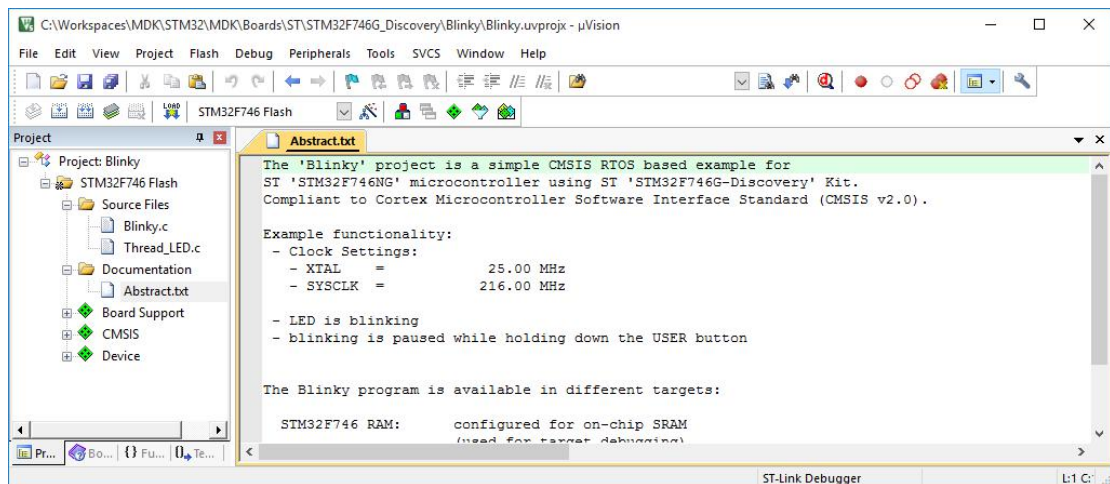
单击 **OK** 开始复制过程。

使用 µ Vision 的示例应用程序

现在，µ Vision 启动并加载示例项目，您可以在其中：

-  构建应用程序，编译和链接相关的源文件。
-  下载到应用程序到设备上，通常是片上 Flash ROM。
-  使用调试器在目标硬件上运行应用程序。

分步说明将告诉您如何执行这些任务。复制示例后，µ Vision 启动并看起来与下图相似。

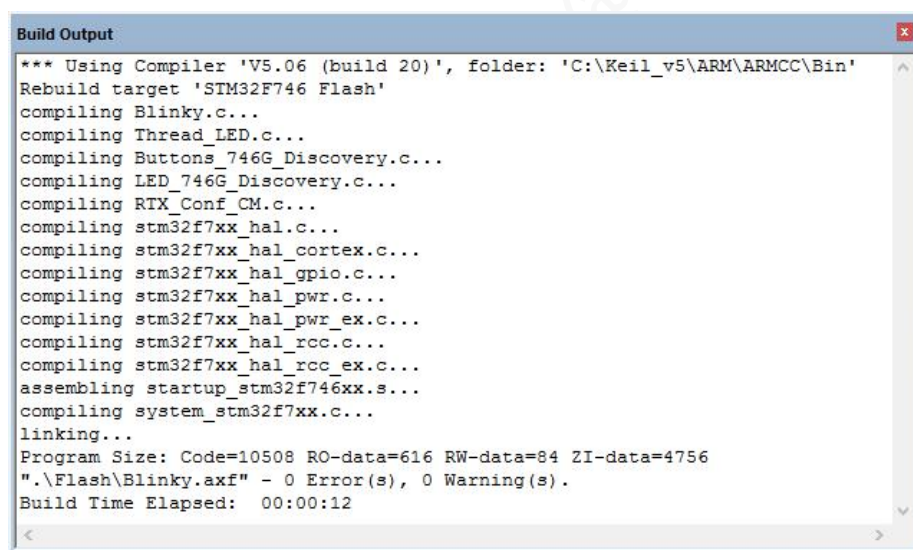


提示：大多数示例项目包含一个 **Abstract.txt** 文件，其中包含有关操作和硬件配置的基本信息。

构建应用程序

使用工具栏按钮重建“构建 **Rebuild**”应用程序。

“构建输出 **Build Output**”窗口显示有关构建过程的信息。无错误的构建显示有关程序大小的信息。



下载应用程序

使用通常通过 **USB** 连接的调试适配器将目标硬件连接到计算机。几个评估板提供了一个板载调试适配器。



现在，查看调试适配器的设置。通常，示例项目是预先配置的评估套件；因此，您不需要修改这些设置。

单击工具栏上的“**Options for Target**”选项，然后选择“**Debug**”选项卡。验证您选择并启用了正在使用的评估板的正确调试适配器。例如，**CMSIS-DAP 调试器**是一个调试适配器，它是几个入门工具包的一部分。

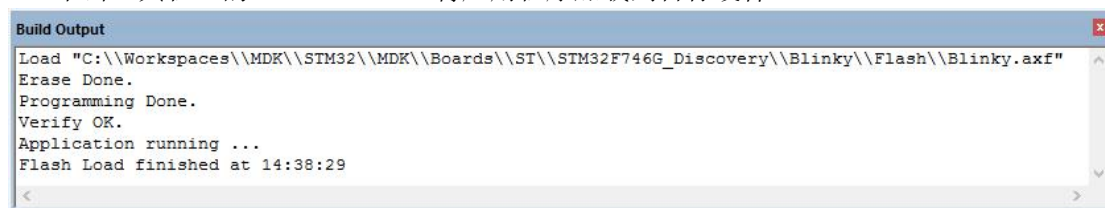


✎ 启用 **Load Application at Startup** 时，只要启动调试会话，将应用程序加载到 μ Vision 调试器中。

✎ 启用 **Run to main()** 时，每次复位时执行到 **main()** 函数的第一个可执行语句。

提示： 单击 **Settings** 按钮以检查通信设置并判断目标硬件有没有问题。有关详细信息，请单击对话框中的“帮助 **Help**”按钮。如果您有任何问题，请参阅入门工具包的用户指南。

 单击工具栏上的“**Download**”将应用程序加载到目标硬件。



“**Build Output**” 窗口显示有关下载进度的信息。

运行应用程序

 单击工具栏上的 **Start/Stop 调试会话(Start/Stop Debug Session)** 开始在硬件上调试应用程序。

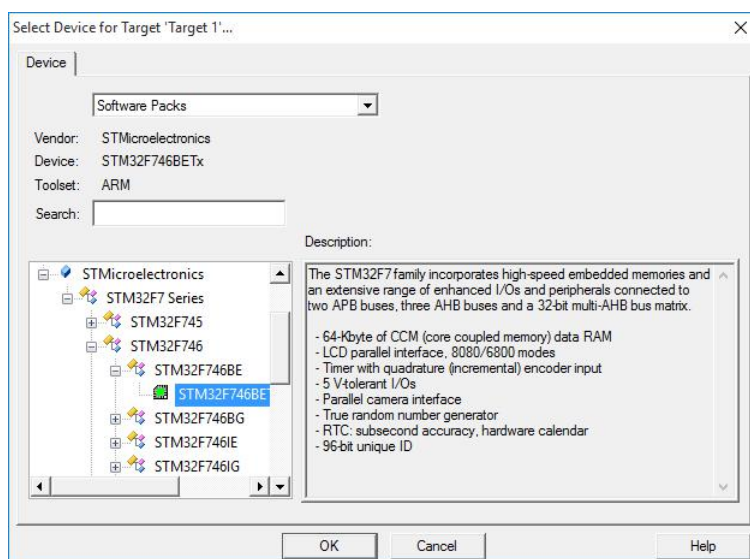
 单击调试工具栏上的 **Run** 来开始执行应用程序。LED 应在目标硬件上闪烁。

使用软件包

软件包包含有关可应用程序作为构建块的单片机设备和软件组件的信息。

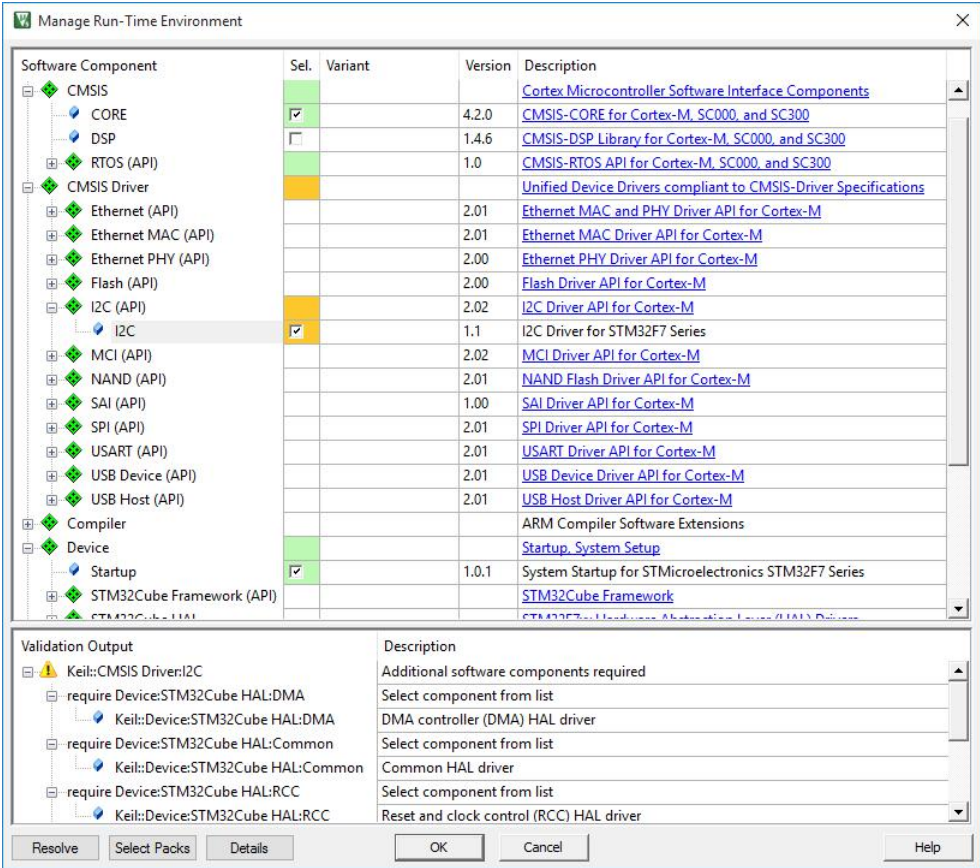
设备信息为您预配置开发工具，并仅显示与所选设备相关的选项。

 启动 μ Vision 并使用菜单 **Project - New μ Vision Project**。选择项目目录并指定项目名称后，选择目标设备。



提示： 仅显示已安装软件包一部分的设备。如果您缺少设备，请使用 **Pack Installer** 添加相关软件包。搜索框可帮助您缩小设备列表。

 选择设备后，“**Manage Run-Time Environment**”窗口将显示此设备的相关软件组件。



提示： 列中的链接说明 *Description* 提供对每个软件组件的文档的访问。

注意

符号:: <Component Class>: <Group>: <Name>用于引用组件。例如，:: **CMSIS: CORE** 是指在上面对话框中选择的组件 **CMSIS-CORE**。

软件组件的概述

下表显示了典型安装的软件组件。根据您选择的设备，某些软件组件可能在管理运行时环境窗口中不可见。如果您安装了额外的软件包，将提供更多的软件组件。

软件构件	描述	页
Board Support(板支持)	评估板外围设备的接口。	点击到达
CMSIS	CMSIS 接口组件，如 CORE 、 DSP 、 CMSIS-RTOS 。	点击到达
CMSIS Driver(CMSIS 驱动)	中间件和用户应用程序的统一设备驱动程序。	点击到达
Compiler(编译器)	ARM 编译器特定的软件组件重定向 I/O 操作例如 printf 调试。	点击到达
Device(驱动)	系统启动和低级设备驱动程序。	点击到达

File System (文件系统)	用于各种存储设备类型的文件访问的中间件组件。	点击到达
Graphics(图形)	用于创建图形用户界面的中间件组件。	点击到达
Network(网络)	用于以太网或串行协议的TCP/IP 网络的中间件组件。	点击到达
USB	支持 usb 设备类的 usb 主机和 usb 设备的中间件组件。	点击到达

使用软件包进行产品生命周期管理

MDK 允许您安装软件包的多个版本。这使得产品生命周期管理（PLM）成为许多项目的常见问题。

PLM 有四个不同的阶段：

概念(**Concept**): 主要项目要求的定义和功能原型的探索。

设计(**Design**): 根据最终技术特点和要求对产品进行原型测试和实施。

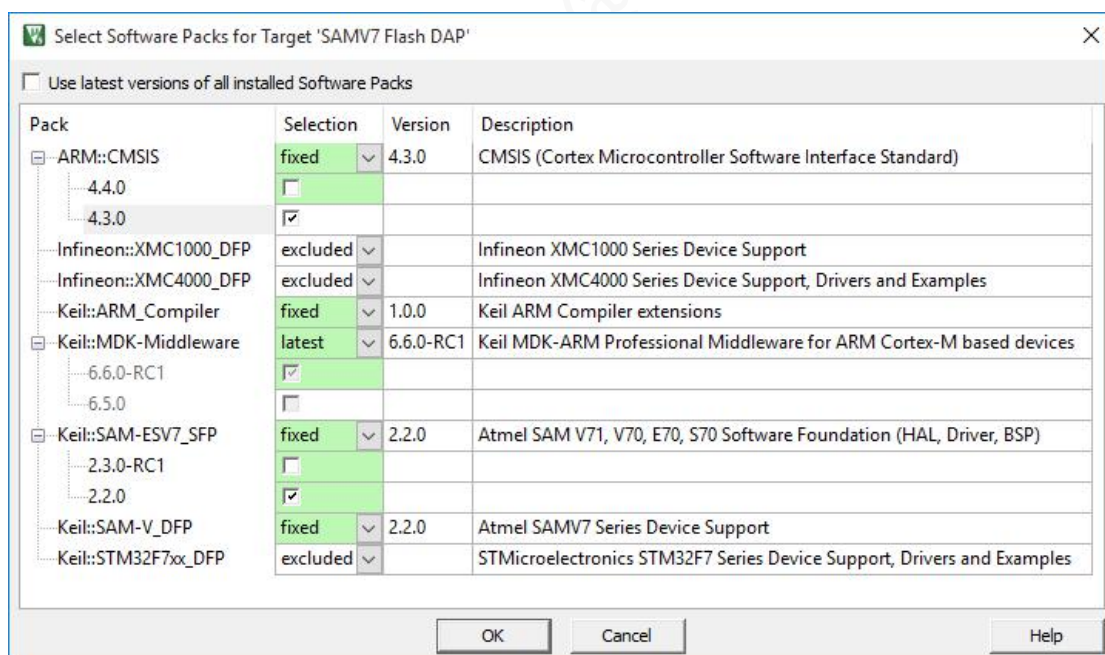
发布(**Release**): 该产品制造并上市。

服务(**Service**): 维护产品，包括客户支持;最后逐步淘汰或报废。

在概念和设计阶段，您通常希望使用最新的软件包来快速整合新功能和错误修复。在产品发布之前，您将冻结软件组件到已知的测试状态。在产品服务阶段，使用固定版本的软件组件来应用到客户的现场。



选择软件包对话框可帮助您管理项目中每个软件包的版本：



项目完成后，禁用选项“使用最新版本的所有已安装的软件包(Use latest version of all installed Software Packs)”，并使用“**Selection**”下的设置指定软件包：

最新版本(latest): 使用最新版本的软件包。当安装较新的软件包版本时，软件组件将更新。

固定(fixed): 指定软件包的已安装版本。项目目标中的软件组件将使用这些版本。

排除(excluded): 不使用本软件包中的软件组件。

颜色表示当前项目目标中软件组件的使用情况：

使用此软件包中的某些软件组件。

☒ 使用此软件包中的某些软件组件，但是不包括软件包。

☐ 不使用此软件包中的软件组件。

软件版本控制系统 (SVCS)

μ Vision 携带 GIT, SVN, CVS 等的模板文件，以支持软件版本控制系统 (SVCS)。

应用笔记 279 “使用 Git 进行项目管理与 μ Vision”

www.keil.com/appnotes/docs/apnt_279.asp 介绍了如何使用软件包对项目的版本控制建立强大的工作流程。

访问文档

MDK 提供在线手册和上下文相关帮助。 μ Vision 帮助菜单打开包括 μ Vision 用户指南 (*μVision User's Guide*)、入门手册、编译器、链接器和汇编器参考指南的主要帮助系统。

许多对话框都有上下文相关的帮助按钮，用于访问文档并说明对话框选项和设置。

您可以在编辑器中按 **F1** 来访问诸如 RTOS 功能、编译器指令或库例程等语言元素的帮助。

在“输出(**Output**)”窗口的命令行中使用 **F1**，以获得有关调试命令的帮助，以及一些错误和警告消息。

书籍(**Books**)窗口可能包括设备参考指南，数据表或电路板手册。您甚至可以添加自己的文档，通过使用菜单“项目(**Project**) - 管理(**Manage**) - 组件(**Components**)，环境(**Environment**)，图书(**Books**) - 书籍(**Books**)”来使书籍(**Books**)窗口启用。

“管理运行时环境(**Manage Run-Time Environment**)”对话框通过“说明(**Description**)”列中的链接提供对文档的访问。

在“项目(**Project**)”窗口中，您可以右键单击软件组件组并打开相应元素的文档。

您可以在线访问 μ Vision 用户指南，网址为：www.keil.com/support/man/docs/uv4。

请求协助

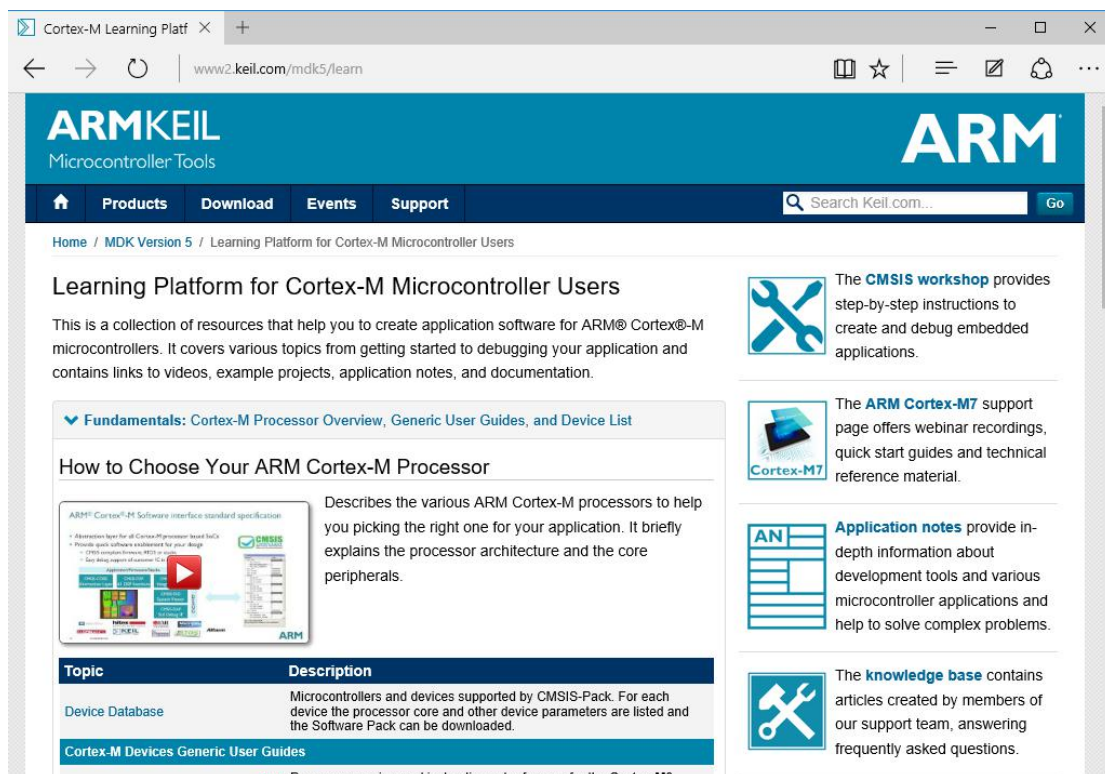
如果您有建议或您发现该软件的问题，请向我们报告。支持和信息渠道可通过

www.keil.com/support 访问。

报告问题时，请包括您的许可证代码（如果您有）和产品版本，可从 Vision 菜单帮助 - 关于。

学习平台

我们提供一个网站，可帮助您更多地了解基于 ARM Cortex 的微控制器的编程。它包含教程、视频、深入的文档，以及其他网站的有用链接，可从 www.keil.com/learn 获取。



快速入门指南

快速入门指南可帮助您快速调出目标硬件。他们描述了使用 MDK 获取开发板并运行所需的步骤，并列出了所需的软件包以及集成调试适配器的驱动程序要求。

注意

<http://www.keil.com/mdk5/qsg> 介绍如何下载快速入门指南

CMSIS

微控制器软件接口标准 **Cortex Microcontroller Software Interface Standard Cortex(CMSIS)**为基于 Cortex-M 的微控制器运行的嵌入式应用提供了一个基本的软件框架。CMSIS 为处理器和外设提供一致而简单的软件接口，简化了软件重用，减少了微控制器开发人员的学习曲线。

注意

本章是参考部分。[创建应用程序](#)一章（第45-51页）显示了如何使用 CMSIS 创建应用程序代码。CMSIS 与各种半导体软件供应商密切合作，为界面外围设备，实时操作系统和中间件组件提供了一种常见的方法。

CMSIS 应用软件组件如下：

§ **CMSIS-CORE**：定义 API 的 Cortex-M 处理器内核和外设，包括一个一致的系统启动代码。软件组件::CMSIS:CORE 和 ::Device:Startup 启动都需要创建和运行在真实的处理器，使用异常，中断和外设。

§ **CMSIS-RTOS RTX**：提供标准化的实时操作系统 API，并支持可以在支持的 RTOS 系统上工作的软件模板、中间件、库和其他组件。本手册介绍了 CMSIS-RTOS RTX 实现的用法。

§ **CMSIS-DSP**: 用于数字信号处理（DSP）的库集合，具有 60 种以上各种数据类型的功能：固定点（分数 q7，q15，q31）和单精度浮点数（32 位）。

§ **CMSIS-Driver**: 是介绍中间件堆栈和用户应用程序的外围驱动程序接口的软件 API。
CMSIS-Driver API 设计为通用的，独立于特定的 RTOS，使其可以在各种受支持的微控制器设备中重复使用。

CMSIS-CORE

本节说明在本地运行一个 CMSIS-CORE 应用在 Cortex-M 处理器使用。这种类型的操作称为裸机，因为它不使用实时操作系统。

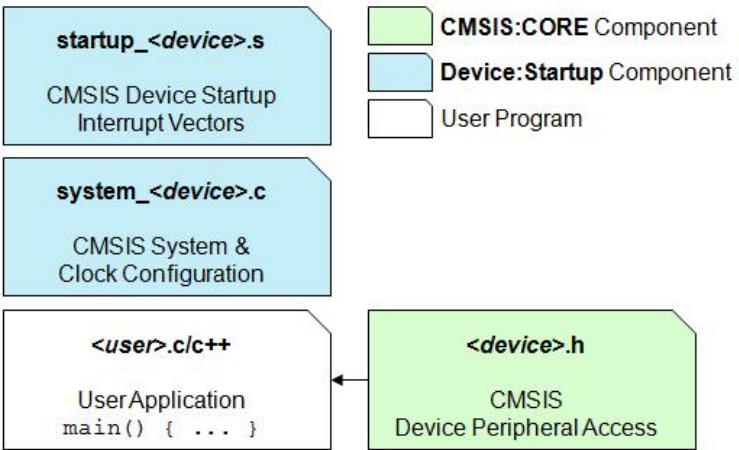
使用 CMSIS-CORE

一个土生土长的 Cortex-M 应用 CMSIS 采用软件组件::CMSIS:CORE,这应该与软件组件一起使用::Device:Startup.启动装置。这些组件提供以下中央文件：

注意

在实际文件名中，<device>是微控制器设备的名称。

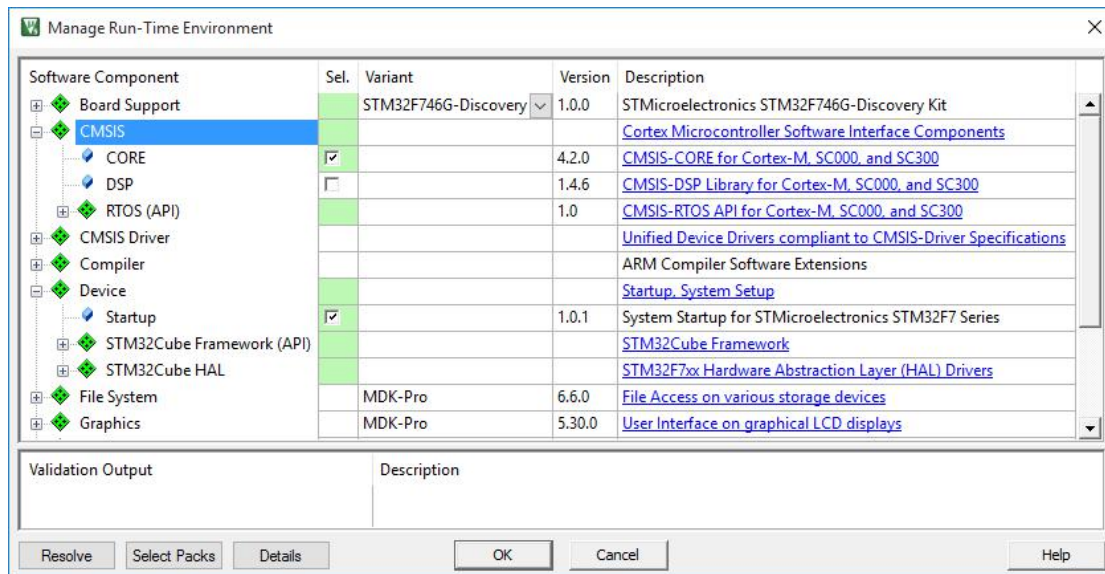
startup_<device>.s 文件句柄和复位异常向量。
system_<device>.c 配置文件的基本设置（时钟、存储器总线）。
<device>.h 包含用于用户代码访问微控制器设备的文件。
<device>.h 文件包含在 C 源文件中并定义：具有标准寄存器布局的外围访问。



访问中断和异常的，嵌套的中断向量控制器（NVIC）。
生成特殊指令的内部函数，例如激活睡眠模式。
SysTick 定时器（SysTick）的功能配置和启动周期定时器中断。
调试 printf 风格的 I/O 和 ITM 的通信通过片上 coresight™访问。

向项目中添加软件组件

组件的文件:: CMSIS: CORE 和:: Device: Startup 将使用 μ Vision 对话框“管理运行时环境 (Manage Run-Time Environment)” 添加到项目中。只需选择软件组件，如下所示：



µ Vision 环境会添加相关文件。

源代码示例

以下源代码行显示了 CMSIS-CORE 层的用法。

使用 CMSIS-CORE 层的示例

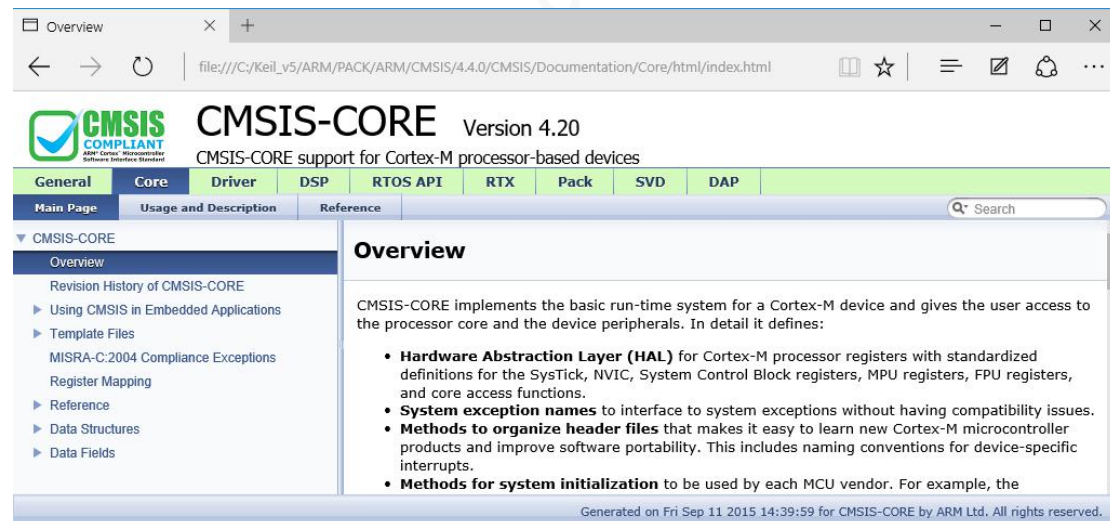
```
#include "stm32f4xx.h" // File name depends on device used
uint32_t volatile msTicks; // Counter for millisecond Interval
uint32_t volatile frequency; // Frequency for timer
void SysTick_Handler (void) // SysTick Interrupt Handler
{
    msTicks++; // Increment Counter
}
void WaitForTick (void)
{
    uint32_t curTicks;
    curTicks = msTicks; // Save Current SysTick Value
    while (msTicks == curTicks) // Wait for next SysTick Interrupt
    {
        __WFE (); // Power-Down until next Event
    }
}
void TIM1_UP_IRQHandler (void) // Timer Interrupt Handler
{
    ; // Add user code here
}
void timer1_init(int frequency) // Set up Timer (device specific)
{
    NVIC_SetPriority (TIM1_UP_IRQn, 1); // Set Timer priority
    NVIC_EnableIRQ (TIM1_UP_IRQn); // Enable Timer Interrupt
}
// Configure & Initialize the MCU
```

```

void Device_Initialization (void)
{
    if (SysTick_Config (SystemCoreClock / 1000))    // SysTick 1ms
    {
        : // Handle Error
    }
    timer1_init (frequency); // Setup device-specific timer
}
// The processor clock is initialized by CMSIS startup + system file
int main (void)    // User application starts here
{
    Device_Initialization (); // Configure & Initialize MCU
    while (1)    // Endless Loop (the Super-Loop)
    {
        __disable_irq (); // Disable all interrupts
        // Get_InputValues ();
        __enable_irq (); // Enable all interrupts
        // Process_Values ();
        WaitForTick (); // Synchronize to SysTick Timer
    }
}

```

有关详细信息，请在“项目”窗口中右键单击“CMSIS”组，然后选择“打开文档”，或参阅 CMSIS-CORE 文档 <http://www.keil.com/cmsis/core>。



CMSIS-RTOS RTX

本节介绍 CMSIS-RTOS RTX 实时操作系统，介绍其功能和特点，并介绍了该 RTOS 的配置设置。

注意

MDK 与许多第三方 RTOS 解决方案兼容。然而，CMSIS-RTOS RTX 可以很好地集成到 MDK 中，功能丰富，针对深度嵌入式系统的要求而定制。

软件概念

嵌入式应用程序有两个基本的设计理念：

无限循环设计(Infinite Loop Design)： 涉及将程序运行作为无限循环。程序函数（线程）从循环中调用，而中断服务程序（ISR）执行包括某些数据处理在内的时间关键任务。

RTOS 设计(RTOS Design)： 涉及使用实时操作系统（RTOS）运行多个线程。 RTOS 提供线程间通信和时间管理功能。抢占式 RTOS 降低了中断功能的复杂性，因为高优先级线程可以执行时间关键的数据处理。

无限循环设计 Infinite Loop Design

在无限循环中运行嵌入式程序是简单嵌入式应用程序的恰当解决方案。通常由硬件中断触发时间关键功能函数(译者注：时效要求高的函数)，并在 ISR 中执行任何所需的数据处理。主循环仅包含不是时间关键并且在后台运行的基本操作。

RTOS 内核的优点

RTOS 内核，像 CMSIS-RTOS RTX 一样，是基于并行执行线程（任务）的思想。与现实世界一样，您的应用程序将必须完成多个不同的任务。基于 RTOS 的应用程序在您的软件中重新创建了该模型，具有各种优势：

- 1. 线程优先级和运行时调度由 RTOS 内核使用经过验证的代码库来处理。
- 2. RTOS 为线程之间的通信提供了一个定义明确的界面。
- 3. 抢占式的多任务概念简化了即使跨越更大的开发团队逐步增强应用程序。可以添加新的功能，而不会有更多关键线程的响应时间。
- 4. 无限循环软件概念通常会轮询发生的中断。相比之下，RTOS 内核本身是中断驱动的，可以很大程度上消除轮询。这允许 CPU 更频繁地睡眠或处理线程。
- 5. 现代 RTOS 内核对中断系统是透明的，这对于具有硬实时要求的系统是强制性的。通信设施可用于 IRQ 到任务通信，并允许上半部分/下半部分处理中断。

使用 CMSIS-RTOS RTX

CMSIS-RTOS RTX 作为库实现，并通过头文件 cmsis_os.h 公开功能。

CMSIS-RTOS RTX 的执行以 main()函数作为第一个线程开始。这样做的好处是开发人员可以初始化内部创建线程的其他中间件库，但是用户应用程序的剩余部分只使用主线程。因此，

RTOS 的使用对应用程序员来说是不可见的，但库可以使用 CMSIS-RTOS RTX 功能。软件组件::CMSIS:RTOS:Keil RTX 必须与组件::CMSIS:CORE 和 ::Device:Startup 一起使用。 选择这些组件提供以下中央 CMSIS-RTOS RTX 文件：

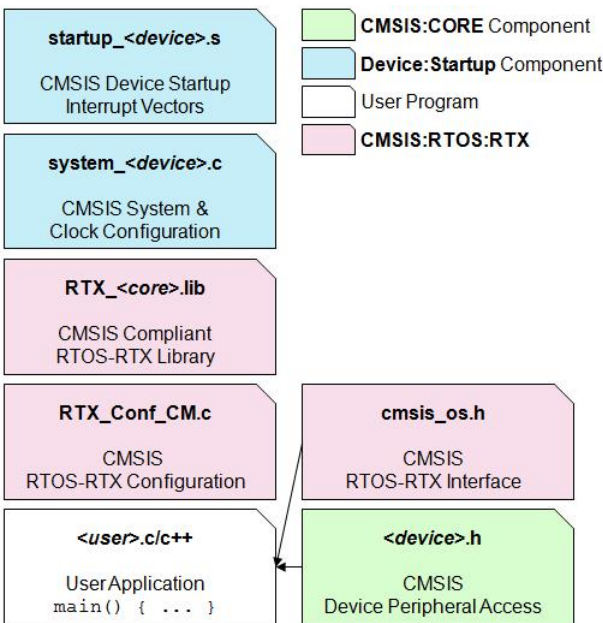
注意

在实际文件名中，<device>是微控制器设备的名称；<device core>表示设备处理器系列。

文件 RTX_<core>.lib 是具有 RTOS 功能的库。

RTX_Conf_CM.c 用于定义线程选项，定时器配置和 RTX 内核设置的配置文件。

头文件 cmsis_os.h 将 RTX 功能公开给用户



应用程序。

函数 `main()` 被执行为一个线程。

一旦这些文件是项目的一部分，开发人员就可以开始使用 CMSIS-RTOS RTX 功能。代码示例显示了使用 CMSIS-RTOS RTX 功能：

使用 CMSIS-RTOS RTX 功能的示例

```
#include "cmsis_os.h" // CMSIS RTOS header file
void job1 (void const *argument) // Function 'job1'
{
    // execute some code
    osDelay (10); // Delay execution for 10ms
}
osThreadDef(job1, osPriorityLow, 1, 0); // Define job1 as thread
int main (void)
{
    osKernelInitialize (); // Initialize RTOS kernel
    // setup and initialize peripherals
    osThreadCreate (osThread(job1), NULL); // Create the thread
    osKernelStart (); // Start kernel & job1 thread
}
```

头文件 `cmsis_os.h`

文件 `cmsis_os.h` 是 CMSIS-RTOS RTX 的模板头文件，包含：

CMSIS-RTOS API 函数定义。

参数和返回类型的定义。

CMSIS-RTOS API 函数使用的状态和优先级值。

用于定义线程和其他内核对象（如互斥量，信号量或内存池）的宏。

所有定义都以 `os` 为前缀，为 CMSIS-RTOS 功能提供了唯一的名称空间。前缀为 `os_` 的定义不会在应用程序代码中使用，而是在本头文件中使用。属于模块的所有定义和函数都是分组的，并具有公共的前缀，例如，用于线程的 `osThread`。

定义和参考对象定义

使用 `#define osObjectsExternal`，对象被定义为外部符号。这允许为整个项目创建一致的头文件，如下所示：

头文件示例： `osObjects.h`

```
#include "cmsis_os.h" // CMSIS RTOS header
extern void thread_1 (void const *argument); // Function prototype
osThreadDef (thread_1, osPriorityLow, 1, 100); // Thread definition
osPoolDef (MyPool, 10, long); // Pool definition
```

头文件 `osObjects.h` 定义了包含在 C / C++ 源文件中的所有对象。当头文件包含之前 `#define osObjectsExternal` 存在时，对象被定义为外部符号。因此，可以在整个项目中使用单个一致的头文件。

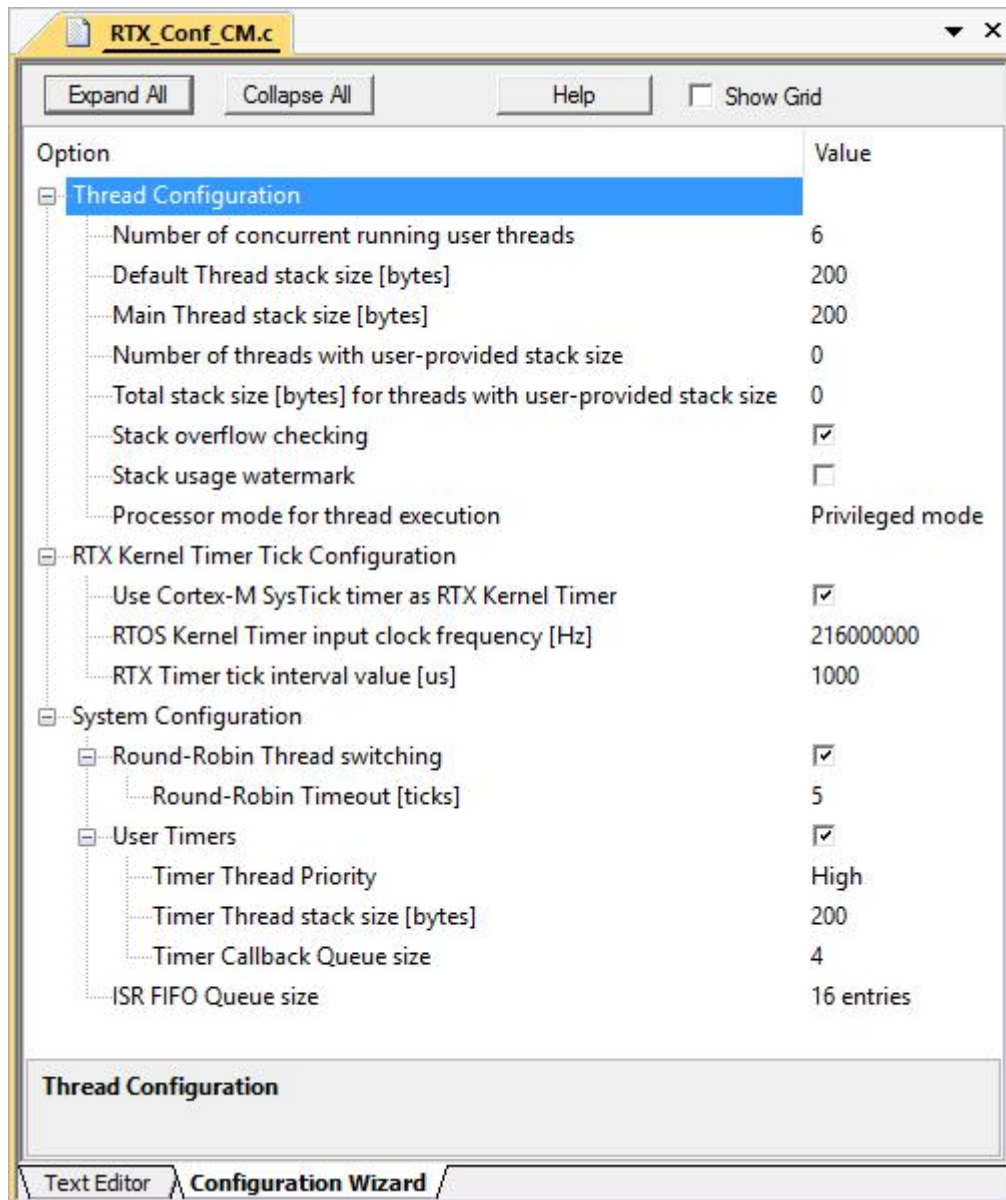
头文件在 C 文件中使用一致

```
#define osObjectExternal // Objects defined as external symbols
#include "osObjects.h" // Reference to the CMSIS-RTOS objects
```

有关详细信息，请参阅在线文档 www.keil.com/cmsis/rtos，“标题文件模板(Header File Template)” 部分： `cmsis_os.h`。

CMSIS-RTOS RTX 配置

文件 RTX_Conf_CM.c 包含 CMSIS-RTOS RTX 的配置参数。该文件的副本是使用 RTX 组件的每个项目的一部分。



您可以设置线程堆栈(thread stack)的参数，配置系统时钟节拍(Tick Timer)，设置轮转(Round-Robin)时间片，并定义线程的用户定时器行为。

有关配置选项的更多信息，请从“管理运行时环境(Manage Run-Time Environment)”窗口打开 RTX 文档。 CMSIS-RTOS RTX 的配置部分(Configuration of CMSIS-RTOS RTX)描述了所有可用的设置。以下内容突出显示了在应用程序中需要配置的最重要的设置。

线程栈配置

线程在代码中用 `osThreadDef()` 函数定义。参数 `stacksz` 指定线程的堆栈需求，并对分配堆栈的方法有影响。CMSIS-RTOS RTX 提供了两种在文件 RTX_Conf_CM.c 中分配堆栈需求的方法：

A. 使用固定内存池：如果参数 `stacksz` 为 0，则为 **Default Thread stack size [bytes]** 指定的值为线程函数设置堆栈大小。

B. 定义用户空间：如果 `stacksz` 不为 0，则从用户空间分配线程堆栈。该用户空间的总大小由具 **Total stack size [bytes] for threads with user-provided stack size** 指定。

Thread Configuration	
Number of concurrent running user threads	6
Default Thread stack size [bytes]	200
Main Thread stack size [bytes]	200
Number of threads with user-provided stack size	0
Total stack size [bytes] for threads with user-provided stack size	0
Stack overflow checking	☒
Stack usage watermark	☒

并发运行线程数(Number of concurrent running threads)指定从固定大小的内存池分配堆栈的最大线程数。

默认线程堆栈大小[字节](Default Thread stack size [bytes])指定在没有用户提供的堆栈的情况下定义的线程的堆栈大小（以字为单位）。

主线程堆栈大小[字节](Main Thread stack size [bytes])是 main()函数的堆栈要求。具有用户提供的堆栈大小的线程数指定了以特定堆栈大小定义的线程数。

具有用户提供的堆栈大小的线程的总堆栈大小[字节](Total stack size [bytes] for threads with user-provided stack size)是以特定堆栈大小定义的所有线程的组合要求（以字为单位）。

堆栈溢出检查(Stack overflow checking)可以在线程切换时进行堆栈溢出检查。启用此选项会稍微增加线程切换的执行时间。

堆栈使用水印(Stack usage watermark)在线程创建时用水印模式初始化线程堆栈。这样可以监视每个线程的堆栈使用情况（不仅在线程切换时），还有助于在线程中查找堆栈溢出问题。启用此选项会显著增加 `osThreadCreate()` 的执行时间。

注意

仔细考虑这些设置。如果您没有分配足够的内存，或者您没有指定足够的线程，则您的应用程序将无法正常工作。

RTX 内核时钟节拍(Timer Tick)配置

CMSIS-RTOS RTX 功能提供从滴答定时器值(**Timer tick value**)导出的以毫秒为单位的延迟。我们建议配置 **Timer tick 值**以产生 1 毫秒的间隔。配置更长的间隔可能会降低能耗，但会对超时的粒度产生影响。

RTX Kernel Timer Tick Configuration	
Use Cortex-M SysTick timer as RTX Kernel Timer	☒
RTOS Kernel Timer input clock frequency [Hz]	180000000
RTX Timer tick interval value [us]	1000

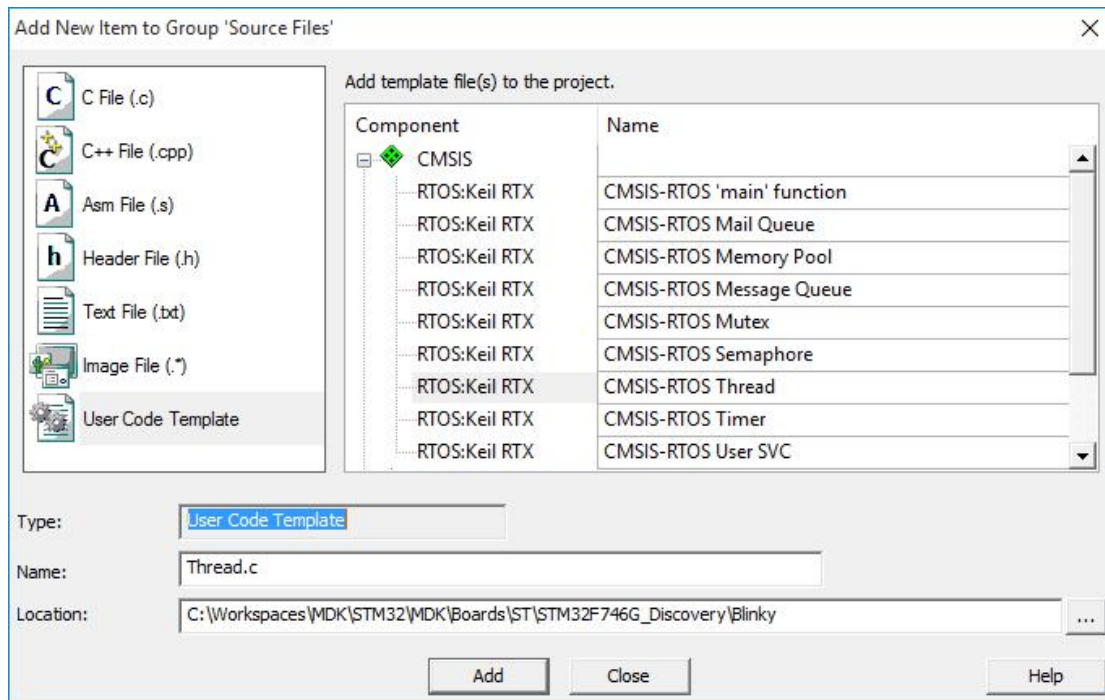
使用 Cortex-M 的滴答定时器作为 RTX 内核定时器(**Use Cortex-M SysTick timer as RTX Kernel Timer**)是个好习惯。这将选择内置的 SysTick 定时器，处理器时钟作为时钟源(**RTOS Kernel Timer input clock frequency**)。在这种情况下，RTOS 内核定时器输入时钟频率应与启动文件 `system_<device>.c` 的 CMSIS 变量 `SystemCoreClock` 相同。

有关详细信息，请参阅在线文档部分 **CMSIS-RTOS RTX -Tick Timer Configuration** 的配置。

CMSIS-RTOS 用户代码模板

MDK 提供用户代码模板，可用于为应用程序创建 C 源代码。

 在“项目(Project)”窗口中，右键单击一个组，选择“添加新项目到组(Add New Item to Group)”，选择“用户代码模板(User Code Template)”，选择一个模板，然后单击“添加(Add)”。



CMSIS-RTOS RTX API 函数

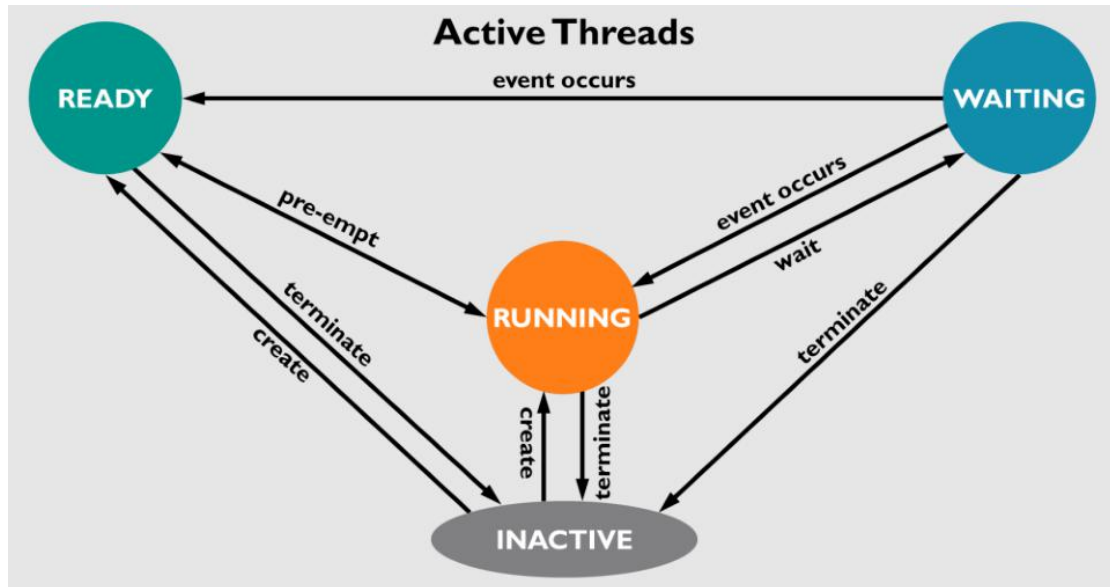
下表列出了 CMSIS-RTOS RTX 可用的各种 API 功能类别。

API 分类	描述
线程管理	定义，创建和控制线程函数。
计时器管理	创建和控制定时器和回调函数。
信号管理	控制或等待信号标志。
互斥管理	通过互斥量同步线程执行。
信号量管理	控制对共享资源的访问。
内存池管理	定义和管理固定大小的内存池
消息队列管理	控制，发送，接收或等待消息。
邮箱队列管理	控制，发送，接收或等待邮件。

提示：可在 www.keil.com/pack/doc/CMSIS/RTX/html/index.html 上提供的 CMSIS-RTOS RTX 教程解释了 API 函数的用法。

线程管理

线程管理功能允许您在系统中定义、创建和控制自己的线程函数。函数 `main()` 是一个特殊的线程函数，它在系统初始化时启动，初始优先级为 `osPriorityNormal`。



CMSIS-RTOS RTX 假定线程如上图所示进行安排。线程状态如下所述改变：

使用函数 `osThreadCreat()` 创建一个线程。这将使线程进入 **READY** 或 **RUNNING** 状态（取决于线程优先级）。

CMSIS-RTOS 是抢占式的。具有最高优先级的主动线程将成为 **RUNNING** 线程，只要它不等待任何事件。线程的初始优先级由 `osThreadDef()` 定义，但在执行过程中可能会使用 `osThreadSetPriority()` 函数进行更改。

RUNNING 线程在等待事件时转移到 **WAITING** 状态。

活动线程可以随时使用 `osThreadTerminate()` 函数终止。线程也可以通过从通常的永久循环 (forever loop) 退出并从线程函数返回 (return) 来终止。终止的线程处于 **INACTIVE** 状态，通常不消耗任何动态内存资源。

单线程程序

标准 C 程序使用 `main()` 函数开始执行。对于嵌入式应用程序，此函数通常是无限循环，可以被视为连续执行的单个线程。

抢占式线程切换

具有相同优先级的线程需要轮询超时或显式调用 `osDelay()` 函数来执行其他线程。在以下示例中，如果 `job2` 的优先级高于 `job1`，则 `job2` 的执行将立即启动。`job2` 抢占执行 `job1`（这是一个非常快的任务切换，只需要几 ms）。

简单 RTX 程序使用循环任务切换

```

#include "cmsis_os.h"
int counter1;
int counter2;
void job1 (void const *arg)
{
    while (1)    // Loop forever
    {
        counter1++; // Increment counter1
    }
}
  
```



```

void job2 (void const *arg)
{
    while (1)    // Loop forever
    {
        counter2++; // Increment counter2
    }
}

osThreadDef (job1, osPriorityNormal, 1, 0); // Define thread for job1
osThreadDef (job2, osPriorityNormal, 1, 0); // Define thread for job2
int main (void)    // main() runs as thread
{
    osKernelInitialize (); // Initialize RTX
    osThreadCreate (osThread (job1), NULL); // Create and start job1
    osThreadCreate (osThread (job2), NULL); // Create and start job2
    osKernelStart (); // Start RTX kernel
    while (1)
    {
        osThreadYield (); // Next thread
    }
}

```

以更高的线程优先级开始 **job2**

```
osThreadDef (osThread (job2), osPriorityAboveNormal, 1, 0);
```

CMSIS-RTOS 系统和线程查看器

CMSIS-RTOS RTX 内核内置支持 RTOS 感知调试。在调试过程中，打开 **Debug->OS Support**，并选择 **System and Thread Viewer**。此窗口显示系统状态信息和正在运行的线程。

System and Thread Viewer							
Property		Value					
System	Item	Value					
	Tick Timer:	1.000 mSec					
	Round Robin Timeout:	5.000 mSec					
	Default Thread Stack Size:	200					
	Thread Stack Overflow Check:	Yes					
	Thread Usage:	Available: 7, Used: 6 + os_idle_demon					
Threads	ID	Name	Priority	State	Delay	Event Value	Event Mask
	1	osTimerThread	High	Wait_MBX			
	2	main	Normal	Wait_DLY	84		
	3	USBD_HID0_Thread	AboveNor...	Wait_OR		0x0000	0xFFFF
	4	USBD0_Core_Thread	AboveNor...	Wait_OR		0x0000	0xFFFF
	5	USBD_HID1_Thread	AboveNor...	Wait_OR		0x0000	0xFFFF
	6	USBD1_Core_Thread	AboveNor...	Wait_OR		0x0000	0xFFFF
	255	os_idle_demon	None	Running			

系统(System)属性显示有关应用程序中 RTOS 配置的一般信息。**线程使用(Thread Usage)**显示当前活动的可用线程数和线程数以及使用过的线程数。

线程(Threads)属性显示有关应用程序的线程执行的详细信息。它显示每个线程关于优先级，

执行状态和堆栈使用情况的信息。

如果在文件 RTX_Conf_CM.c 中为**线程配置(Thread Configuration)**启用了**堆栈使用水印(Stack usage watermark)**选项, 则**堆栈使用(Stack Usage)**字段将显示 **cur:** 和 **max:** 堆栈负载情况。值 **cur:** 是实际程序位置的当前堆栈使用情况。值 **max:** 是线程执行期间发生的最大堆栈负载, 这是基于堆叠使用水印模式的覆盖。这允许您:

在线程执行期间识别堆栈溢出或者

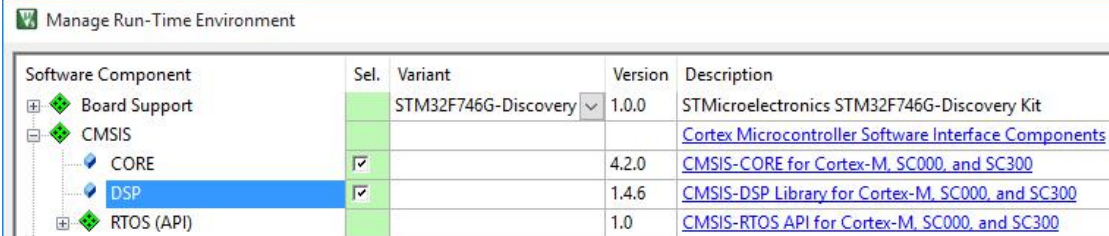
优化和减少用于线程的堆栈空间。

注意 使用 Trace, 调试器还提供了具有详细时序信息的视图。有关详细信息, 请参阅第 75 页的**事件查看器(Event Viewer)**。

CMSIS-DSP

CMSIS-DSP 库是一套通用的数字信号处理 (DSP) 功能。该库可用于针对不同 ARM Cortex-M 处理器进行多种的变量优化。

当在“**管理运行时环境(Manage Run-Time Environment)**”对话框中启用软件组件::CMSIS:DSP 时, 所选设备的相应库将自动包含在项目中。



Software Component	Sel.	Variant	Version	Description
Board Support		STM32F746G-Discovery	1.0.0	STMicroelectronics STM32F746G-Discovery Kit
CMSIS				Cortex Microcontroller Software Interface Components
CORE	☒		4.2.0	CMSIS-CORE for Cortex-M, SC000, and SC300
DSP	☒		1.4.6	CMSIS-DSP Library for Cortex-M, SC000, and SC300
RTOS (API)			1.0	CMSIS-RTOS API for Cortex-M, SC000, and SC300

下面的代码示例显示了使用 CMSIS-DSP 库函数。

使用 DSP 函数对两个矩阵进行乘法运算

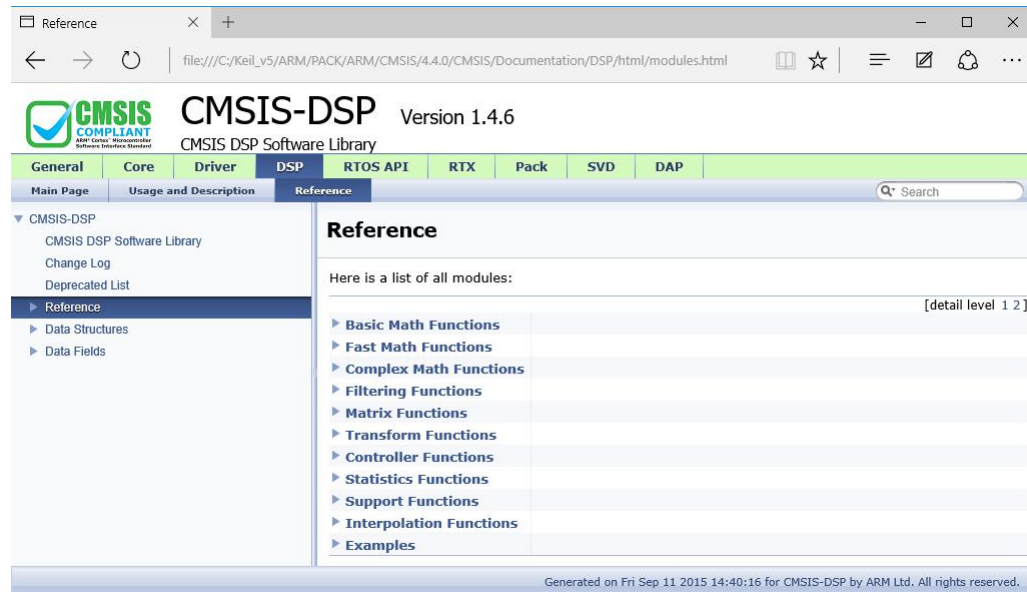
```
#include "arm_math.h" // ARM::CMSIS:DSP
const float32_t buf_A[9] = // Matrix A buffer and values
{
    1.0, 32.0, 4.0,
    1.0, 32.0, 64.0,
    1.0, 16.0, 4.0,
};
float32_t buf_AT[9]; // Buffer for A Transpose (AT)
float32_t buf_ATmA[9]; // Buffer for (AT * A)
arm_matrix_instance_f32 A; // Matrix A
arm_matrix_instance_f32 AT; // Matrix AT(A transpose)
arm_matrix_instance_f32 ATmA; // Matrix ATmA( AT multiplied by A)
uint32_t rows = 3; // Matrix rows
uint32_t cols = 3; // Matrix columns
int main(void)
{
    // Initialize all matrixes with rows, columns, and data array
    arm_mat_init_f32 (&A, rows, cols, (float32_t *)buf_A); // Matrix A
    arm_mat_init_f32 (&AT, rows, cols, buf_AT); // Matrix AT
    arm_mat_init_f32 (&ATmA, rows, cols, buf_ATmA); // Matrix ATmA
    arm_mat_trans_f32 (&A, &AT); // Calculate A Transpose (AT)
```

```

    arm_mat_mult_f32 (&AT, &A, &ATmA); // Multiply AT with A
    while (1);
}

```

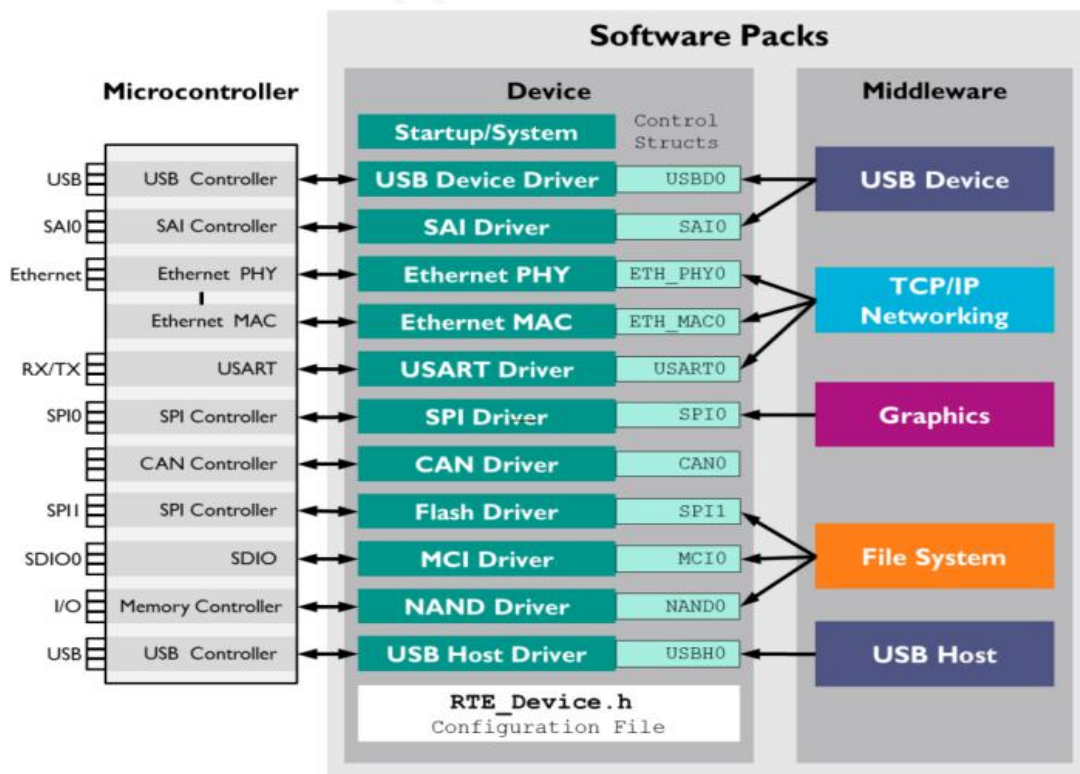
有关更多信息，请参阅 www.keil.com/cmsis/dsp 上的 CMSIS-DSP 文档。



CMSIS Driver(CMSIS 驱动)

特定于设备的 **CMSIS 驱动程序** 提供中间件和微控制器外设之间的接口。这些驱动程序不限于 MDK 中间件，并且可用于各种其他中间件堆栈以利用这些外设。

设备特定的驱动程序通常是软件包的一部分，它支持微控制器设备并符合 CMSIS 驱动程序标准。 www.keil.com/dd2 上的设备数据库列出了设备软件包中包含的驱动程序。



中间件组件通常具有连接到这些驱动程序的各种配置文件。对于大多数设备，RTE_Device.h 文件将驱动程序配置为微控制器设备的实际引脚连接。

中间件/应用程序代码通过控制结构(control struct)连接到驱动程序实例。此控件结构的名称反映了设备的外设接口。大多数通信外设的驱动程序是提供设备支持的软件包的一部分。

使用传统的 C 源代码根据 CMSIS-Driver 标准实现缺少的驱动程序。

有关这些 CMSIS 驱动程序的 API 接口的详细信息，请参考 www.keil.com/cmsis/driver。

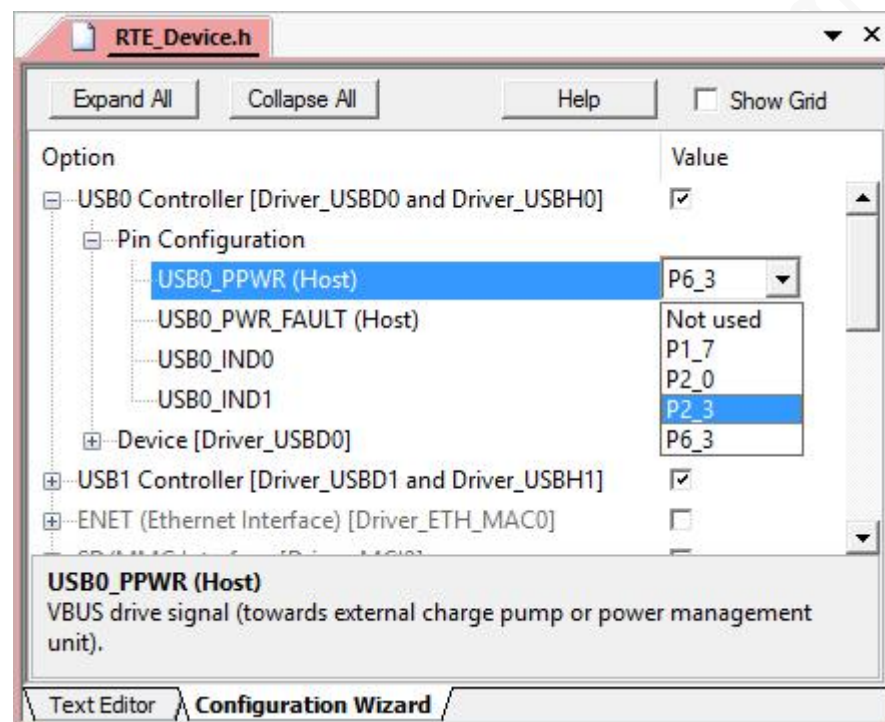
配置

有多种方法来配置 CMSIS 驱动程序(CMSIS-Driver)。经典的方法是使用设备支持附带的 RTE_Device.h 文件。

可以使用允许用户配置设备引脚位置和相应驱动器的第三方图形配置工具来配置其他设备。通常，这些配置工具会自动创建所需的 C 代码，以便导入到 μ Vision 项目中。

使用 RTE_Device.h

对于大多数设备，RTE_Device.h 文件将驱动程序配置为微控制器设备的实际引脚连接：



使用配置向导视图，您可以以图形模式配置驱动程序接口，而且无需手动编辑此头文件中的 #define。

使用 STM32CubeMX

MDK 支持使用 STM32CubeMX 的 CMSIS 驱动程序配置。该图形软件配置工具允许您使用 St-microelectronics 设备的图形向导生成 C 初始化代码。

只需在“管理运行时环境”窗口中选择所需的 CMSIS 驱动程序，然后选择设备

Device:STM32Cube Framework (API):STM32CubeMX。

这将打开 STM32CubeMX 的设备和驱动程序配置。完成后，生成配置代码并将其导入 μ Vision。

有关更多信息，请访问 www.keil.com/pack/doc/STM32Cube/General/html/index.html 上的在线文档。

验证

用于 CMSIS 驱动程序验证测试的软件包可从 www.keil.com/pack 获得。它包含 CMSIS 驱动程序验证套件的源代码和文档以及必需的配置文件, 以及显示各种目标平台上的使用情况的示例。

CMSIS 驱动程序验证套件执行以下测试:

- API 函数调用的通用验证
- 验证配置参数
- 通过环回测试验证
- 通信参数如波特率的验证
- 事件功能的验证

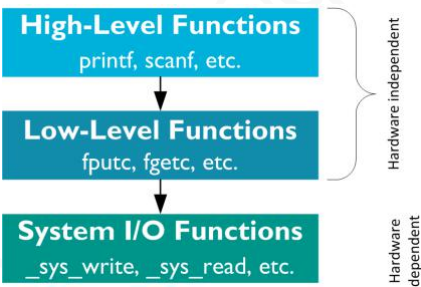
测试结果可以打印到控制台, 通过 ITM printf 输出, 或输出到内存缓冲区。请参阅位于 www.keil.com/cmsis/driver 的 CMSIS 驱动程序文档中的驱动程序验证一节。

软件组件

编译器

软件组件**编译器**允许您重新定位标准 C 运行时库的 I/O 功能。应用程序代码经常使用标准 I/O 库函数, 如 printf(), scanf()或 fgetc()来执行输入/输出操作。

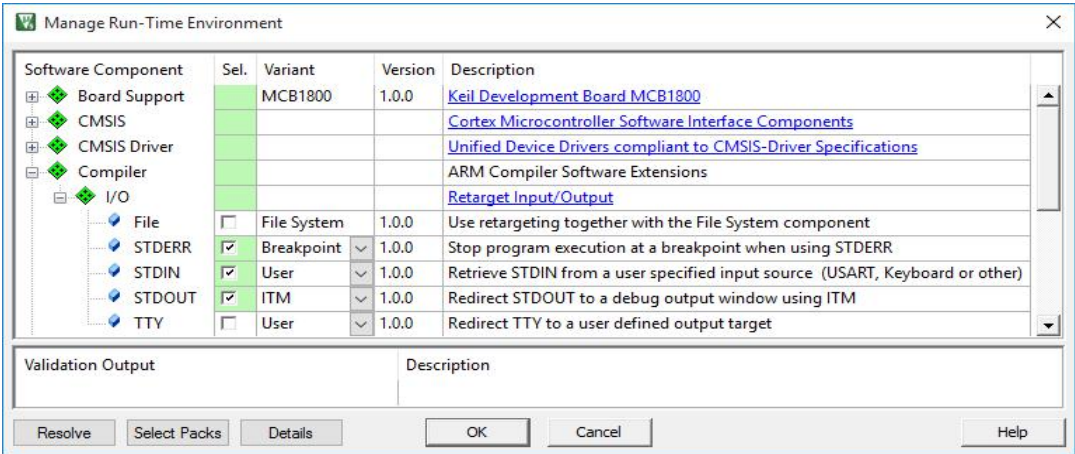
标准 ARM 编译器 C 运行库中的这些功能的结构是:



高级(high-level)和低级功能(low-level functions)与目标无关, 使用系统 I/O 功能(system I/O functions)与硬件接口。

ARM 编译器 C 运行时库的微库(MicroLib)通过低级功能与硬件接口。 MicroLib 实现了一组精简的高级功能, 因此不实现系统 I/O 功能。

软件组件编译器重新定位各种标准 I/O 通道的 I/O 功能: 文件, STDERR, STDIN, STDOUT 和 TTY:



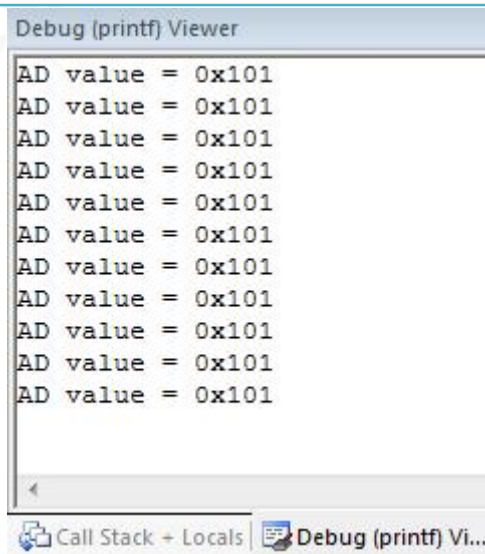
I/O 通道	描述
File	所有文件相关操作的通道(fscanf, fprintf, fopen, fclose 等)
STDERR	应用程序的标准错误(Standard error)流输出诊断消息。
STDIN	进入应用程序的标准输入(Standard input)流 (scanf 等)。
STDOUT	应用程序的标准输出(Standard output)流 (printf 等)。
TTY	电传打字机(Teletypewriter)是错误输出的最后手段。

变量(variant)选项允许您更改 I/O 通道的硬件接口。

变种	描述
File System	使用文件系统组件作为文件相关操作的界面
Breakpoint	当使用 I/O 通道时, 应用程序停止使用 BKPT 指令。
ITM	使用仪器跟踪宏单元 (ITM) 通过调试器进行 I/O 通信。
User	重定向 I/O 功能用于用户定义的例程 (如 USART, 键盘)。

软件组件编译器添加了将根据变量设置进行配置的文件 retarget_io.c。对于用户(User)变量, 用户代码模板可用于帮助您实现自己的功能。有关详细信息, 请参阅文档。

Cortex-M3 / M4 / M7 中的 ITM 支持 printf 样式调试。如果选择变量 ITM, I/O 库函数通过 **Debug (printf) Viewer** 窗口执行 I/O 操作。



板支持

在开发板上经常使用几个接口, 如 LED, 按钮, 操纵杆, A/D 和 D/A 转换器, LCD 和触摸屏以及外部传感器, 如温度计, 加速度计, 磁力计和陀螺仪。

Board Support Interface API 提供对这些接口的标准化访问。这使软件开发人员能够专注于其应用程序代码, 而不是检查设备手册以进行注册设置来切换特定的 GPIO。

许多设备系列包 (DFP) 都包含电路板支持。您可以从“管理运行时环境”窗口中选择板卡支持:

Software Component	Sel.	Variant	Version	Description
Board Support		STM32F746G-Discovery	1.0.0	STMicroelectronics STM32F746G-Discovery Kit
Buttons (API)			1.0.0	Buttons Interface
Buttons	☒		1.0.0	Buttons Interface for STMicroelectronics STM32F746G-Discovery Kit
Drivers				Kinetis BSP Drivers
Graphic LCD (API)			1.0.0	Graphic LCD Interface
LED (API)			1.0.0	LED Interface
LED	☒		1.0.0	LED Interface for STMicroelectronics STM32F746G-Discovery Kit
Touchscreen (API)			1.0.0	Touchscreen Interface
emWin LCD (API)			1.1	emWin LCD Interface

确保选择正确的 **Variant** 变量以为您的特定开发板启用正确的引脚配置。

您可以通过为您的电路板的软件包创建所需的支持文件，为您的自定义板添加板卡支持。请参阅 API 文档：<http://www.keil.com/pack/doc/mw/Board/html/index.html>

创建应用程序

本章将引导您完成上一章所述使用 CMSIS 创建和修改项目所需的步骤。

注意

本节中的示例代码适用于 MCB1800 评估板（LPC1857）。调整其他入门套件或板的代码。

该教程在两个基本设计概念中创建了 Blinky 项目：

RTOS 设计采用 CMSIS-RTOS RTX。

对于没有 RTOS 内核的裸机系统的无限循环设计。

CMSIS-RTOS RTX 闪光灯项目

本部分将使用以下步骤说明项目的创建：

设置项目：创建项目文件，并选择微控制器设备以及相关的 CMSIS 组件。

配置设备时钟频率：配置系统时钟。

定制 CMSIS-RTOS RTX 内核：适应 RTOS 内核。

创建源代码文件：添加并创建应用程序文件。

构建应用程序映像：编译并链接将应用程序下载到单片机器件的片上闪存。

使用调试器：第 64 页的[使用指南](#)可以指导您完成将评估板连接到 PC 并将应用程序下载到目标的步骤。

对于 Blinky 项目，您将创建以下应用程序文件：

main.c 此文件包含 main() 函数，它初始化 RTOS 内核，外设，并启动线程执行。

LED.c 该文件包含初始化和控制 GPIO 端口和线程函数 blink_LED() 的功能。LED_Initialize() 函数初始化 GPIO 端口引脚。功能 LED_On() 和 LED_Off() 控制与 LED 连接的端口引脚。

LED.h 头文件包含 LED.c 中功能的原型，并包含在文件 main.c 中。

设置项目

从 μ Vision 菜单栏中选择 **Project - New μ Vision Project**。

 选择一个空文件夹并输入项目名称，例如 Blinky。单击保存，创建一个具有指定名称的空项目文件（Blinky.uvproj）。

接下来，将打开“选择设备目标对象(Select Device for Target)”对话框。

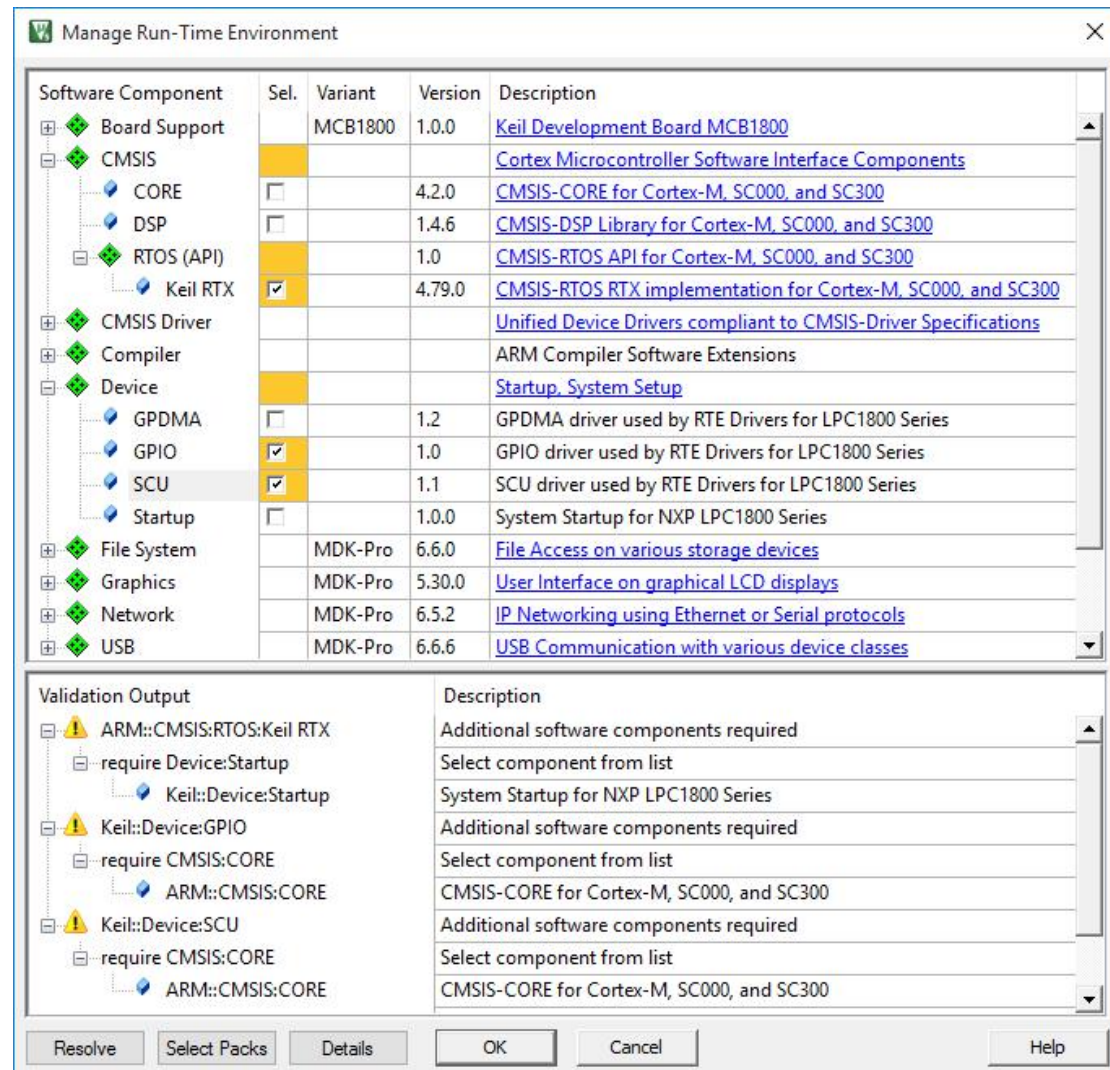
 选择 LPC1857，然后单击确定。

设备选择定义了必要的工具设置，如编译器控件，链接器的内存布局和 Flash 编程算法。

“管理运行时环境 Manage Run-Time Environment”对话框将打开，并显示已安装并可用于所选设备的软件组件。

 展开::CMSIS: RTOS (API) 并启用: Keil RTX。

 展开::Device 并启用::Device 和:SCU。



验证输出(Validation Output)地方显示与其他软件组件的依赖关系。在这种情况下，组件::Device:Startup 是必需的。

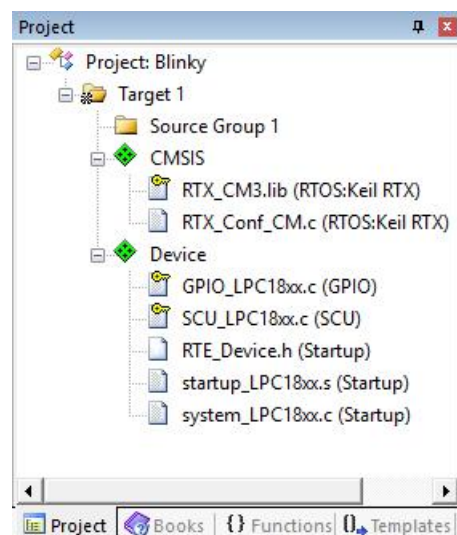
提示：单击消息将突出显示相关的软件组件。

 点击解决(Resolve)。

这解决了所有依赖关系，并启用其他必需的软件组件（这里，::CMSIS: Core 和:: Device: Startup）。

 点击 OK。

所选的软件组件与启动文件，RTX 配置文件和 CMSIS 系统文件一起包含在项目中。“项目 Project”窗口将显示所选的软件组件以及相关文件。双击文件以



在编辑器中打开它。

配置设备时钟频率

系统或核心时钟在 `system_<device>.c` 文件中定义。核心时钟也是 RTOS 内核定时器的输入时钟，因此 RTX 配置文件需要与此设置相匹配。

注意

某些设备执行系统设置作为主要功能的一部分和/或使用配置了外部实用程序的软件框架。有关详细信息，请参阅第 56 页的[设备启动变化](#)。

应用的时钟配置取决于各种因素，如时钟源（XTAL 或片内振荡器）以及存储器和外设的要求。半导体供应商提供特定于设备的文件 `system_<device>.c`，因此需要阅读相关文档。

提示：有关单片机时钟系统的详细信息，请从“图书 **Books**”窗口打开参考手册。

MCB1800 开发套件使用外部 12 MHz XTAL 运行。PLL 产生 180 MHz 的内核时钟频率。由于这是默认值，因此不需要进行任何修改。但是，您可以在文件 `system_LPC18xx.c` 中更改自定义开发板的设置。

 要编辑文件 `system_LPC18xx.c`，请在“项目”窗口中展开“设备”组，双击文件名，然后修改代码，如下所示。

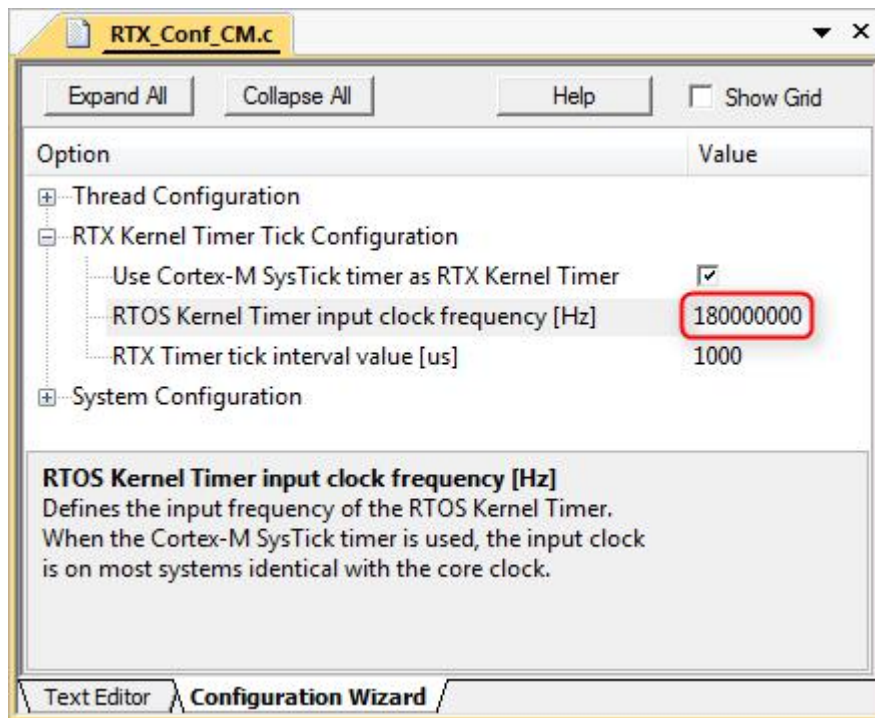
在 `system_LPC18xx.c` 中设置 PLL 参数

```
/* PLL1 output clock: 180MHz, Fcco: 180MHz, N = 1, M = 15, P = x */
#define PLL1_NSEL 0 /* Range [0 - 3]: Pre-divider ratio N */
#define PLL1_MSEL 14 /* Range [0 - 255]: Feedback-div ratio M */
#define PLL1_PSEL 0 /* Range [0 - 3]: Post-divider ratio P */
#define PLL1_BYPASS 0 /* 0: Use PLL, 1: PLL is bypassed */
#define PLL1_DIRECT 1 /* 0: Use PSEL, 1: Don't use PSEL */
#define PLL1_FBSEL 0 /* 0: FCCO is used as PLL feedback */
/* 1: FCLKOUT is used as PLL feedback */
```

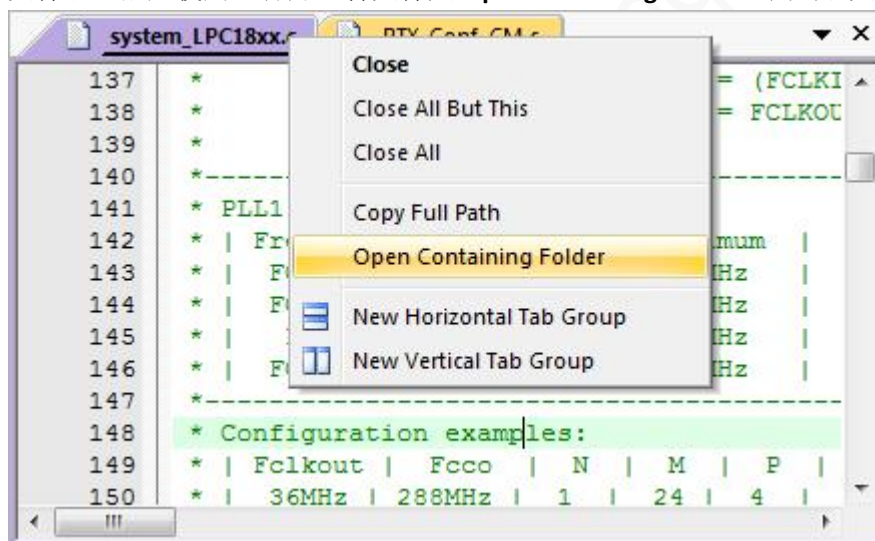
自定义 CMSIS-RTOS RTX 内核

 在“项目 **Project**”窗口中，展开组 **CMSIS**，打开文件 `RTX_Conf_CM.c`，然后单击编辑器底部的“配置向导 **Configuration Wizard**”选项卡。

展开 **RTX Kernel Timer Tick Configuration**，并将定时器时钟(Timer clock value)设置为与内核时钟相匹配。



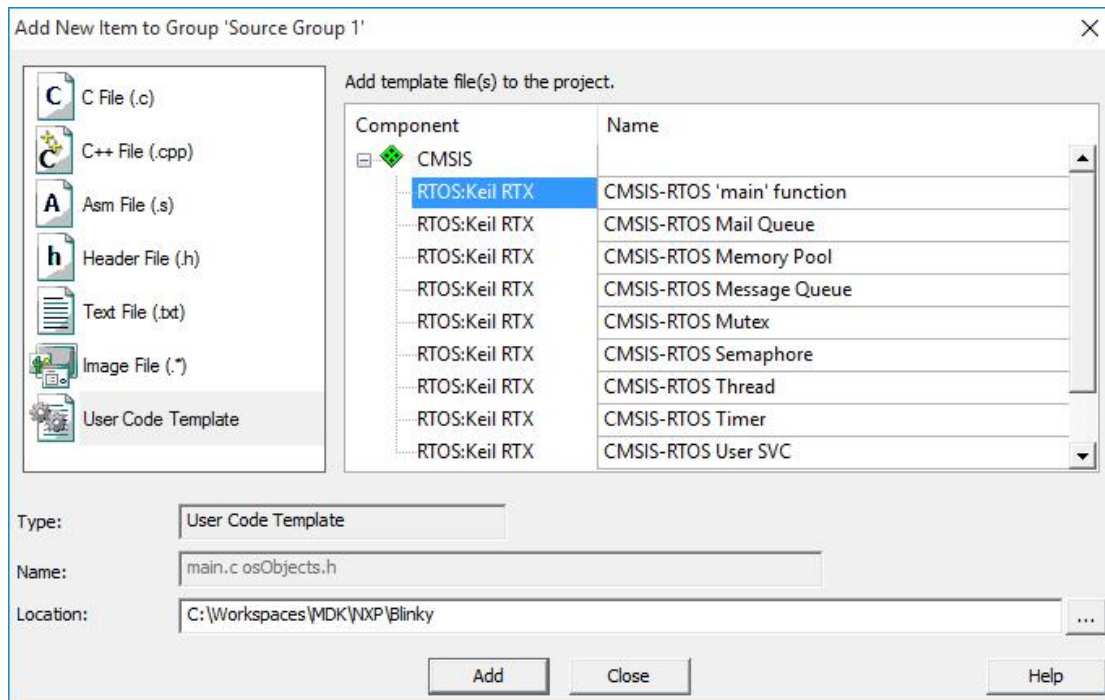
提示：您可以从示例项目中复制编译器定义设置和 `system_<device>.c`。右键单击编辑器中的文件名，然后使用“打开包含文件夹 **Open Containing Folder**”找到该文件。



创建源代码文件

使用包含类似于软件组件功能的例程的预配置的用户代码模板添加应用程序代码。

 在“项目 **Project**”窗口中，右键单击“源组 1 **Source Group 1**”，然后打开“新建项目添加到组 **Add New Item to Group**”对话框。



单击用户代码模板 **User Code Template** 列出项目中包含的软件组件的可用代码模板。选择 **CMSIS-RTOS'main'**功能，然后点击 **Add**。

这将文件 main.c 添加到项目组 **Source Group 1** 中。现在可以将特定于应用程序的代码添加到此文件中。

右键单击文件 main.c 中的空行，然后选择插入'#include files'。包括所选设备的头文件 LPC18xx.h。

然后，添加下面的代码以创建一个功能 link_LED()，使得评估板上的 LED 闪烁。使用 osThreadDef()将 blink_LED()定义为 RTOS 线程，并使用 osThreadCreate()启动它。

```

/*-----
* CMSIS-RTOS 'main' function template main.c
*-----*/
#define osObjectsPublic // Define objects in main module
#include "osObjects.h" // RTOS object definitions
#include "LPC18xx.h" // Device header
#include "LED.h" // Initialize and set GPIO Port
/*
* main: initialize and start the system
*/
int main (void)
{
    osKernelInitialize (); // Initialize CMSIS-RTOS
    // initialize peripherals here
    LED_Initialize (); // Initialize LEDs
    // create 'thread' functions that start executing,
    // example: tid_name = osThreadCreate (osThread(name), NULL);
    Init_BlinkyThread (); // Start Blinky thread
    osKernelStart (); // Start thread execution
    while (1);
}

```


 创建一个名为 LED.c 的空 C 文件，使用对话框 **Add New Item to Group** 并添加代码来初始化并访问控制 LED 的 GPIO 端口引脚。

```
/*-----*
* File LED.c
*-----*/

#include "SCU_LPC18xx.h"
#include "GPIO_LPC18xx.h"
#include "cmsis_os.h" // ARM::CMSIS:RTOS:Keil RTX
void blink_LED (void const *argument); // Prototype function
osThreadDef (blink_LED, osPriorityNormal, 1, 0); // Define blinky thread
void LED_Initialize (void)
{
    GPIO_PortClock (1); // Enable GPIO clock
    /* Configure pin: Output Mode with Pull-down resistors */
    SCU_PinConfigure (13, 10,
        (SCU_CFG_MODE_FUNC4|SCU_PIN_CFG_PULLDOWN_EN));
    GPIO_SetDir (6, 24, GPIO_DIR_OUTPUT);
    GPIO_PinWrite (6, 24, 0);
}
void LED_On (void)
{
    GPIO_PinWrite (6, 24, 1); // LED on: set port
}
void LED_Off (void)
{
    GPIO_PinWrite (6, 24, 0); // LED off: clear port
}
// Blink LED function
void blink_LED(void const *argument)
{
    for (;;)
    {
        LED_On (); // Switch LED on
        osDelay (500); // Delay 500 ms
        LED_Off (); // Switch off
        osDelay (500); // Delay 500 ms
    }
}
void Init_BlinkyThread (void)
{
    osThreadCreate (osThread(blink_LED), NULL); // Create thread
}
```

注意

您还可以使用第 44 页所述的板支持组件提供的功能。

使用对话框添加新项目组来创建一个名为 LED.h 的空头文件，并定义 LED.c 的功能原型。

```
/*-----*
* File LED.h
*-----*/

void LED_Initialize ( void ); // Initialize GPIO
```

```
void LED_On ( void ); // Switch Pin on
void LED_Off ( void ); // Switch Pin off
void blink_LED ( void const *argument ); // Blink LEDs in a thread
void Init_BlinkyThread ( void ); // Initialize thread
```

构建应用程序映像



构建应用程序，编译和链接所有相关的源文件。



构建输出 Build Output 显示有关构建过程的信息。无错误构建显示程序大小信息，零错误和零警告。

```
Build Output
*** Using Compiler 'V5.06 (build 20)', folder: 'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Target 1'
compiling main.c...
compiling LED.c...
compiling RTX_Conf_CM.c...
compiling GPIO_LPC18xx.c...
compiling SCU_LPC18xx.c...
assembling startup_LPC18xx.s...
compiling system_LPC18xx.c...
linking...
Program Size: Code=8512 RO-data=924 RW-data=80 ZI-data=4752
After Build - User command #1: C:\Keil_v5\ARM\BIN\ElfDwT.exe .\Objects\Blinky.axf BASEADDRESS(0x1A000000)
ELFDWT - Signature Creator V1.1.0.0
COPYRIGHT Keil - An ARM Company, Copyright (C) 2014
*** Updated Signature over Range[32] (0x1A000000 - 0x1A000018): @0x1A00001C = 0x53FFCF3E
*** Processing completed, no Errors.
".\Objects\Blinky.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:04
```

第 64 页的[使用调试器](#)部分引导您完成将评估板连接到工作站并将应用程序下载到目标硬件的步骤。

提示：您可以通过检查 LED 的一秒间隔来验证目标硬件的正确时钟和 RTOS 配置设置。

设备启动变化

一些设备执行系统设置的重要部分作为设备硬件抽象层（HAL）的一部分，因此设备初始化是从主函数(main function)内完成的。这样的设备经常使用配置有外部实用程序的软件框架。

:: Device 软件组件可能包含启动设备所需的其他组件。有关详细信息，请参阅在线帮助系统。在下一节中，将举例说明设备启动变化。

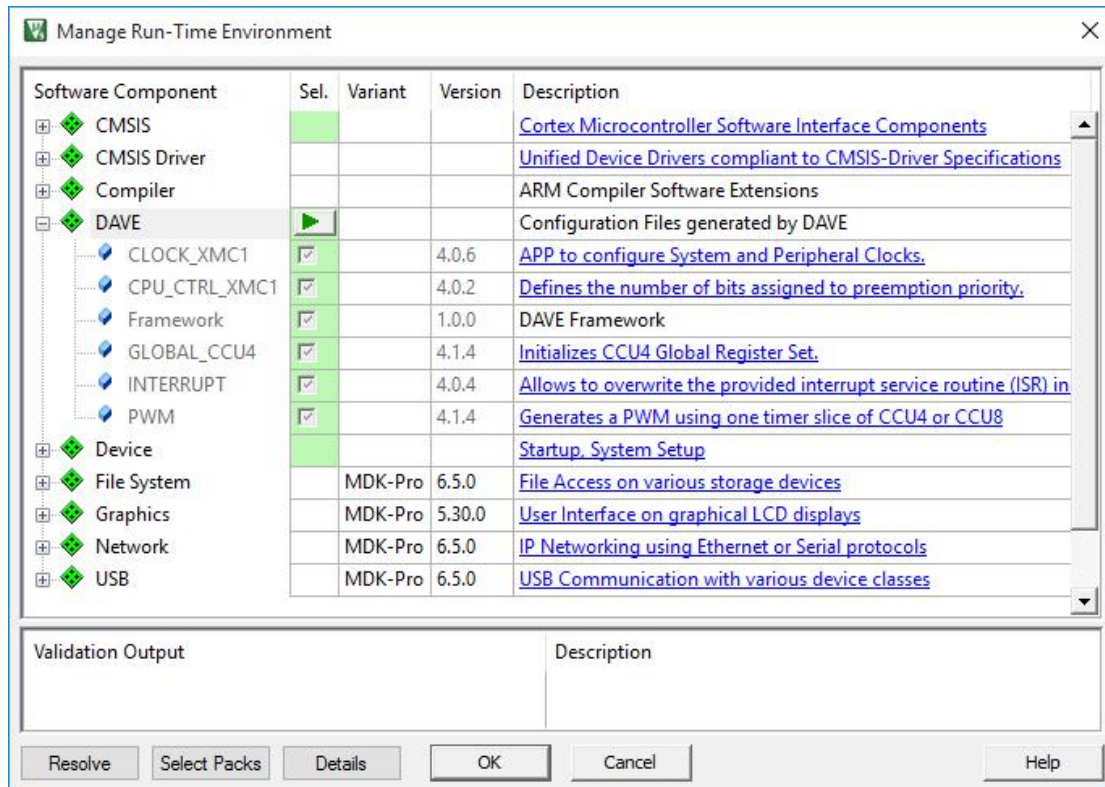
例子：英飞凌 XMC1000 使用 DAVE

使用英飞凌的 **DAVE™**，您可以根据叫做 DAVE 的应用自动生成代码。在基于 Eclipse 的 IDE 中，您可以添加、配置和连接应用程序以适应您的应用程序。在此过程中，您将使用 **CLOCK_XMC_1_0** 应用程序配置时钟设置（如果是 XMC1000 系列）。此应用程序将内核中的正确寄存器设置为达到所需的频率。在生成代码的末尾，它调用 CMSIS 函数 `SystemCoreClockUpdate()`。

有关将 DAVE 项目导入 μ Vision 的所有步骤，请参见

http://www.keil.com/appnotes/docs/apnt_258.asp 中的应用笔记 258。

μ Vision 导入项目后，“管理运行时环境 **Manage Run-Time Environment**”窗口显示组::**DAVE3**，生成的应用程序作为组件。



在 μ Vision 内部，组件:: DAVE 被锁定。使用开始按钮(start button)  打开 DAVE 以更改应用程序的配置。

clock_xmc1_conf.c 文件包含用于设置时钟寄存器的数据结构。以下是一个示例，显示 DAVE 如何根据工具中的配置设置值：

clock_xmc1_conf.c 的代码

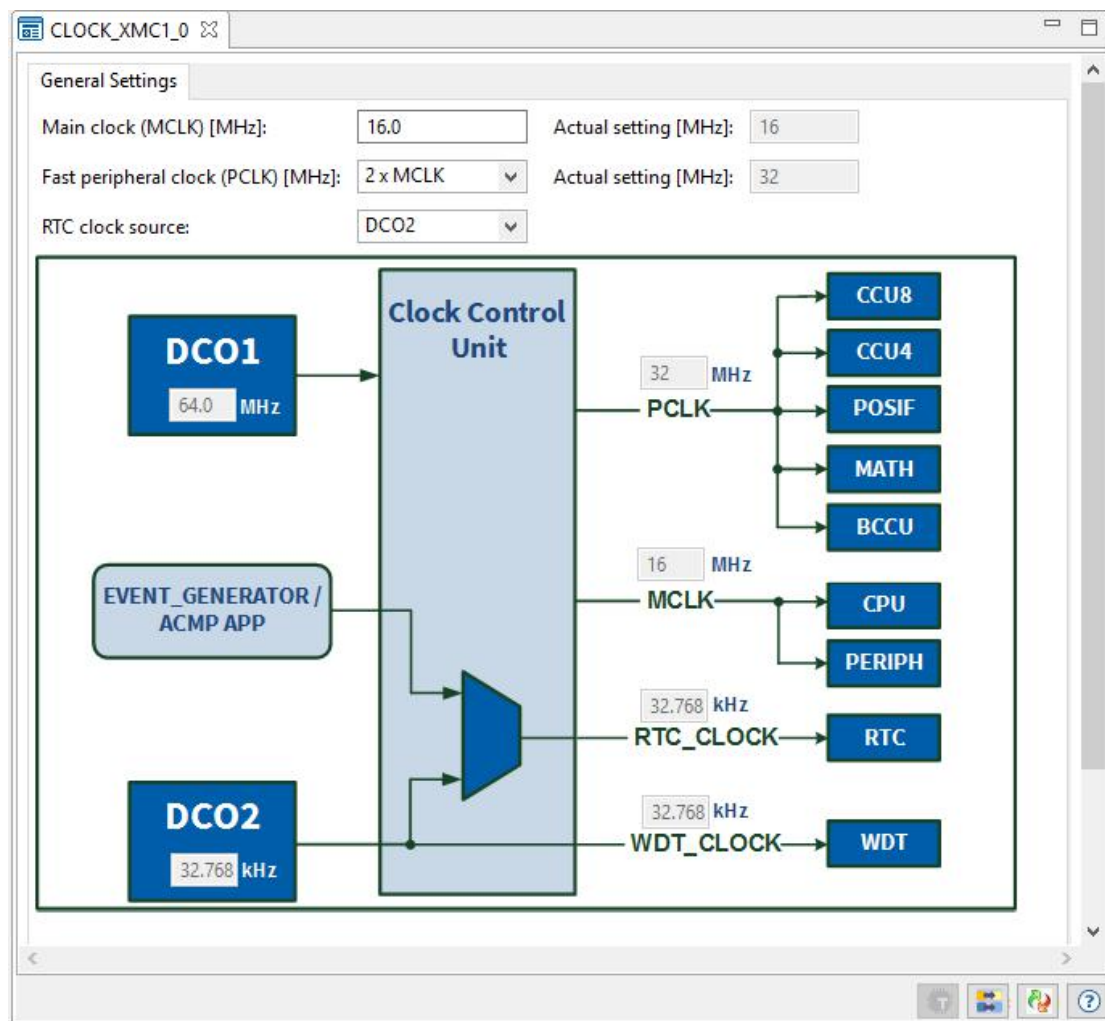
```

:
/*****
* DATA STRUCTURES
*****/
const XMC_SCU_CLOCK_CONFIG_t CLOCK_XMC1_0_CONFIG =
{
    .pclk_src = XMC_SCU_CLOCK_PCLKSRC_DOUBLE_MCLK,
    .rtc_src = XMC_SCU_CLOCK_RTCCLKSRC_DCO2,
    .fdiv = 0U, /**< Fractional divider */
    .idiv = 1U, /**< 8Bit integer divider */
};

```

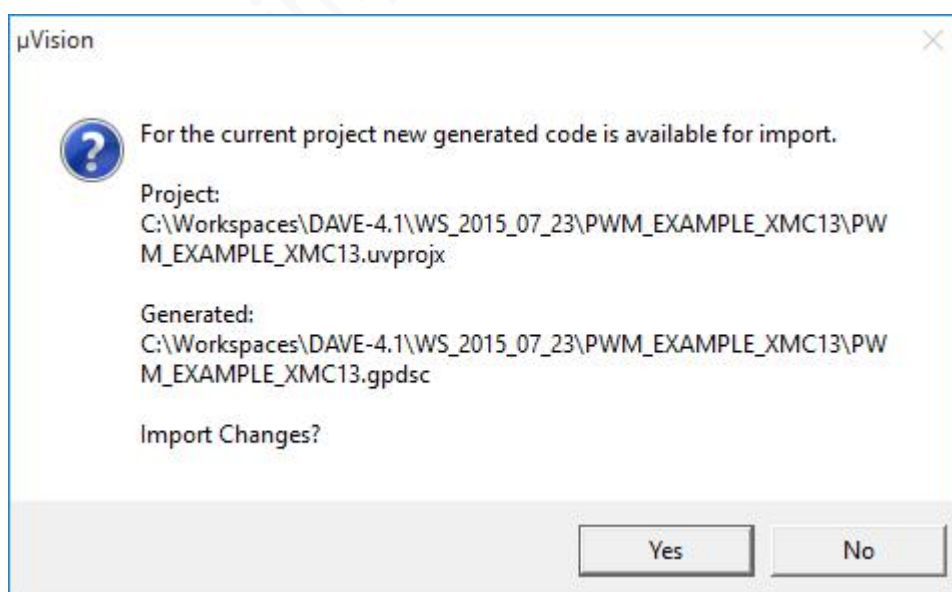
使用 DAVE 更改时钟设置

如果需要更改这些时钟值，请打开“管理运行时环境 **Manage Run-Time Environment**”窗口，然后按开始按钮  打开 DAVE。使用配置 APP 实例..(Configure APP Instance...). 更改时钟设置：



在 DAVE 中重新运行代码生成按钮 。

这将会更改生成的文件，这些文件将被 **μ Vision** 自动识别：



单击 **Yes** 重新加载已更改的文件。

例子: STM32Cube

许多 STM32 设备正在使用可以使用 RTE_Device.h 配置文件或使用 STM32CubeMX 的经典方法配置的 STM32Cube 框架。

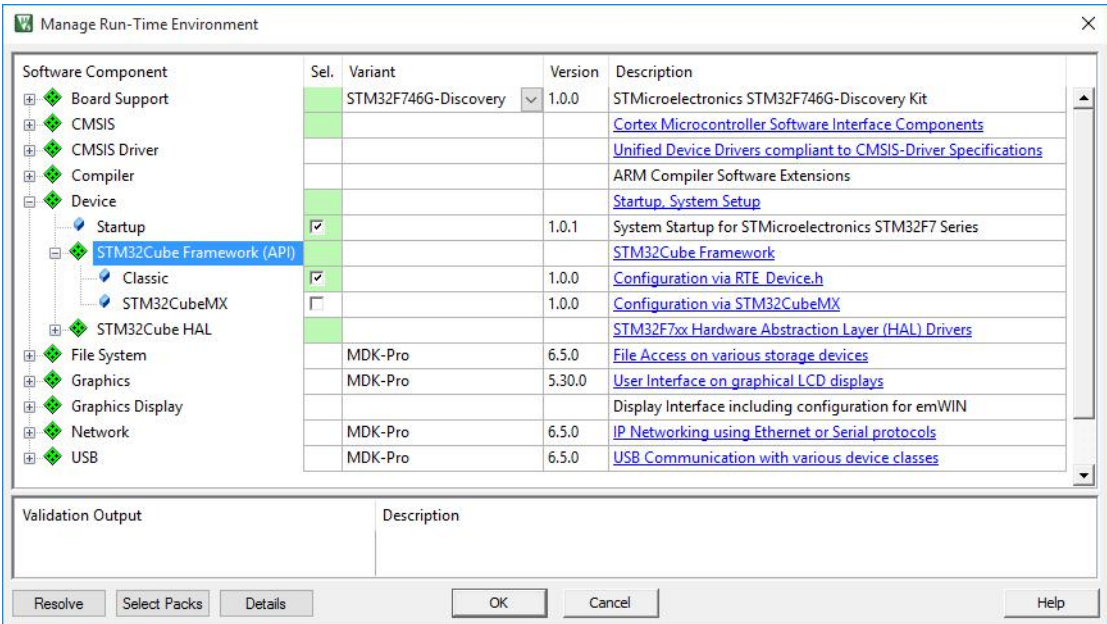
经典的 STM32Cube 框架组件提供了一个实现系统设置的特定用户代码模板。使用 STM32CubeMX，将启动所需的 main.c 文件和其他源文件复制到 **STM32CubeMX: Common Sources** 组下面的项目中。

使用经典框架设置项目

本示例使用经典方法为 STM32F746G-Discovery 套件创建一个项目。在“管理运行时环境 **Manage Run-Time Environment**”窗口中，选择以下内容：

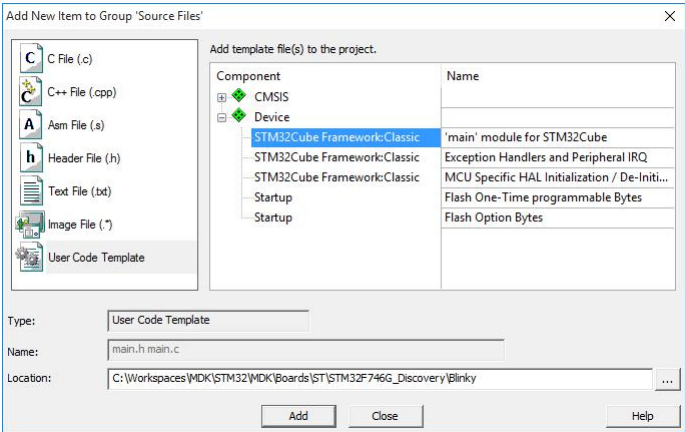
展开::设备: STM32Cube 框架 (API) (::Device:STM32Cube Framework (API))并启用: 经典(Classic)。

展开::设备(::Device)并启用: 启动(:Startup)。



单击解析(Resolve)以启用其他所需的软件组件，然后单击确定 **OK**。

在“项目 **Project**”窗口中，右键单击“源组 1 **Source Group 1**”，然后打开“将新建项目添加到组 **Add New Item to Group**”对话框。



 单击用户代码模板(**User Code Template**)列出项目中包含的软件组件的可用代码模板。选择 STM32Cube 的“主”模块(**‘main’ module for STM32Cube**), 然后单击“添加 **Add**”。main.c 文件包含 *SystemClock_Config()*函数。在这里, 您需要进行时钟配置的设置:
main.c 的代码

```

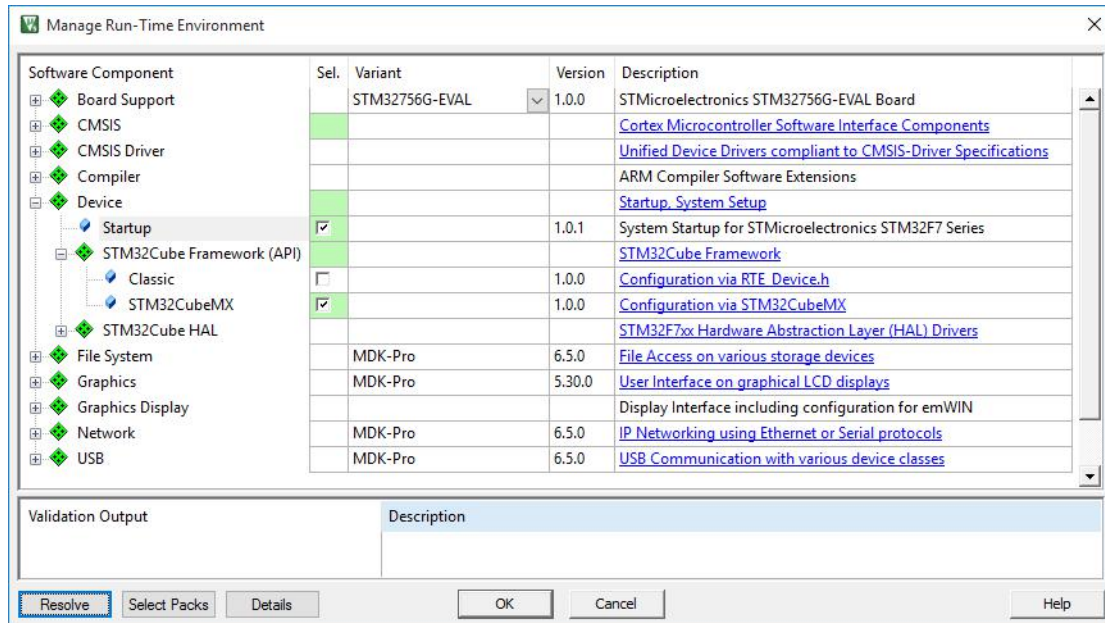
:
static void SystemClock_Config (void)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct;
    RCC_OscInitTypeDef RCC_OscInitStruct;
    /* Enable HSE Oscillator and activate PLL with HSE as source */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_OFF;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 25;
    RCC_OscInitStruct.PLL.PLLN = 432;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 9;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);
    /* Activate the OverDrive to reach the 216 MHz Frequency */
    HAL_PWREx_EnableOverDrive();
    /* Select PLL as system clock source and configure the HCLK, PCLK1
and
PCLK2 clocks dividers */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK |
RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2);
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_7);
}
:

```

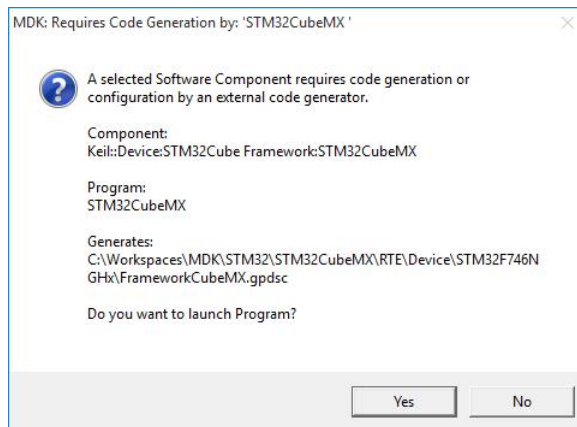
使用 STM32CubeMX 设置项目

此示例创建与之前使用的 **STM32CubeMX** 相同的项目。在“管理运行时环境 **Manage Run-Time Environment**”窗口中, 选择以下内容:

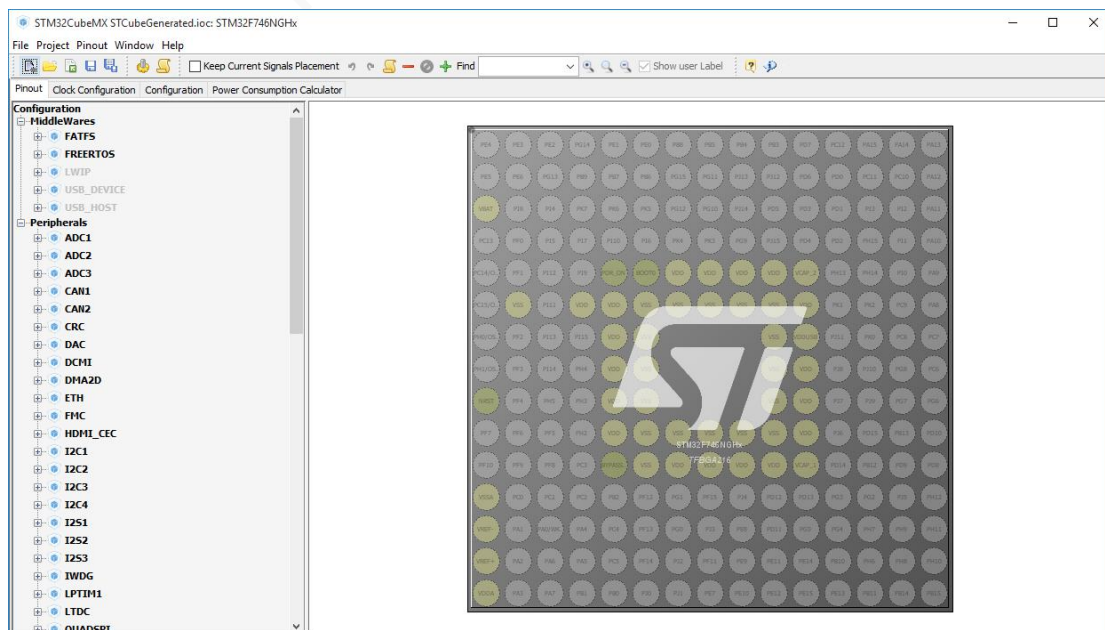
 展开::设备: STM32Cube 框架 (API) (**::Device:STM32Cube Framework (API)**)并启用: **STM32CubeMX**。展开::设备(**::Device**)并启用: 启动(**Startup**)。



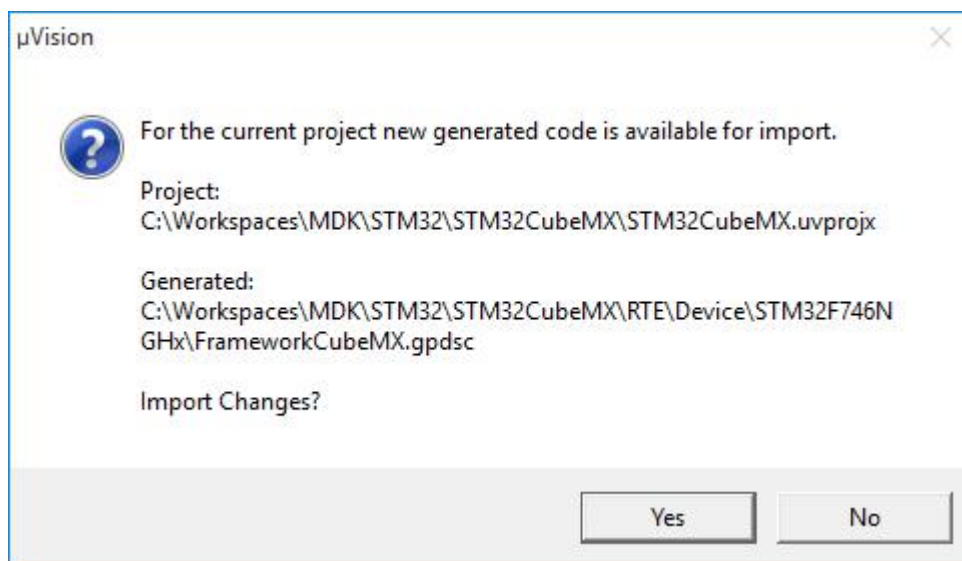
单击解析(Resolve)以启用其他所需的软件组件，然后单击确定 OK。
一个新窗口将要求您启动 STM32CubeMX。



STM32CubeMX 以选定的正确设备启动:



 根据需要配置设备。完成后，转到项目 **Project->生成代码 Generate Code** 创建 GPDSC 文件。μ Vision 会通知您：



 单击是导入项目。main.c 和其他生成的文件被添加到名为 **STM32CubeMX Common Sources** 的文件夹。

调试应用程序

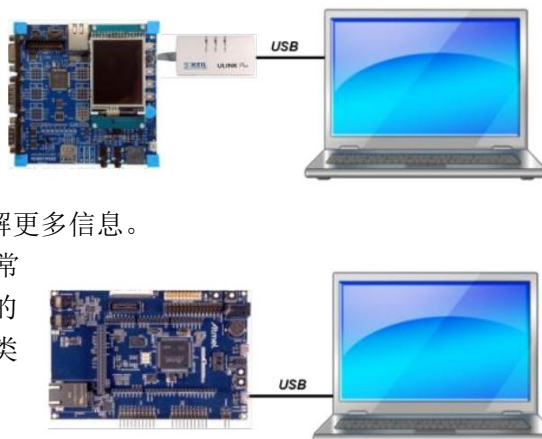
集成到基于 ARM Cortex-M 处理器的 ARM CoreSight™ 技术提供强大的调试和跟踪功能。它允许运行控制启动和停止程序、断点、内存访问和 Flash 编程。大多数设备都提供了采样、数据跟踪、包括程序计数器（PC）中断和仪器跟踪等异常功能。设备使用 ETM，ETB 或 MTB 集成指令跟踪，以便对程序执行进行分析。有关调试和跟踪功能的完整概述，请参阅 www.keil.com/coresight。

调试器连接

MDK 包含连接到各种调试/跟踪适配器的 μ Vision 调试器，并允许您对闪存进行编程。它支持传统功能，如简单和复杂的断点、监视窗口和执行控制。使用跟踪、支持事件/异常查看器、逻辑分析器、执行分析器和代码覆盖等附加功能。

ULINK2 和 ULINK-ME 调试适配器可以连接到 JTAG / SWD 调试连接器，并支持串行线输出（SWO）的跟踪。ULINKpro 调试/跟踪适配器还与 ETM 跟踪连接器进行接口，并使用流跟踪技术捕获完整的指令跟踪以进行代码覆盖和执行分析。请参考 www.keil.com/ulink 了解更多信息。

基于 CMSIS-DAP 的 USB JTAG / SWD 调试接口通常是评估板或入门工具包的一部分，并提供集成的调试功能。MDK 还支持几种专有接口，提供类似的技术。



MDK 连接到第三方调试解决方案，如 Segger J-Link 或 J-Trace。一些入门套件板提供 J-Link Lite 技术作为板载解决方案。

使用调试器

接下来，您将调试上一章中在硬件上创建的 Blinky 应用程序。您需要配置调试连接和 Flash 编程实用程序。

选择调试适配器并配置调试选项。

从工具栏中选择“目标选项 **Options for Target**”，单击“调试 **Debug**”选项卡，启用“使用 **Use**”，然后选择适用的调试驱动程序。



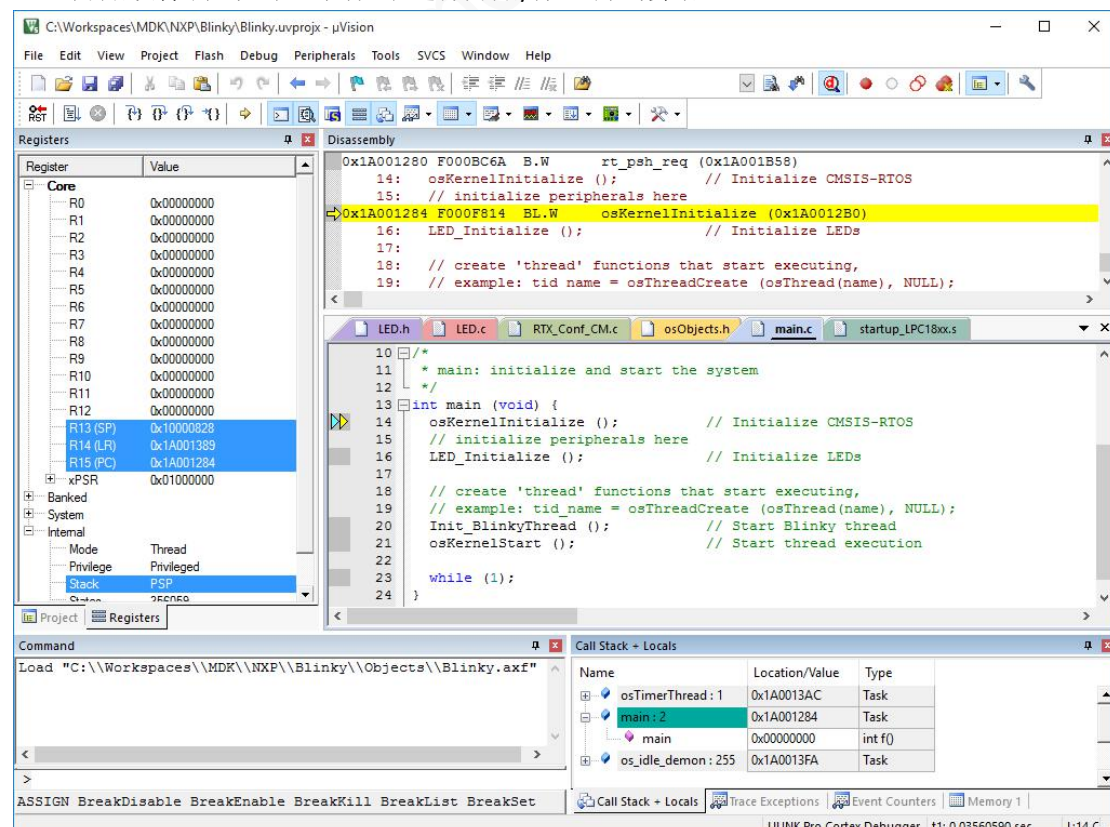
器件选择已经为片上存储器配置了 Flash 编程算法。使用设置按钮验证配置。

将应用程序编程到闪存中。

从工具栏中选择下载 **Download**。“构建输出 **Build Output**”窗口显示有关下载进度的消息。



开始硬件调试。从工具栏中选择开始/停止调试会话。



在调试会话开始时， μ Vision 加载应用程序，执行启动代码，并停止在主 C 函数。

 单击工具栏上的运行。LED 以一秒的频率闪烁。

调试工具栏

调试工具栏可以快速访问许多调试命令，例如：

 **Step** 步骤操作会穿过程序和进入函数调用。

 **Step Over** 步骤会穿过程序和跳过函数调用。

 **Step Out** 步骤会跳出当前函数。

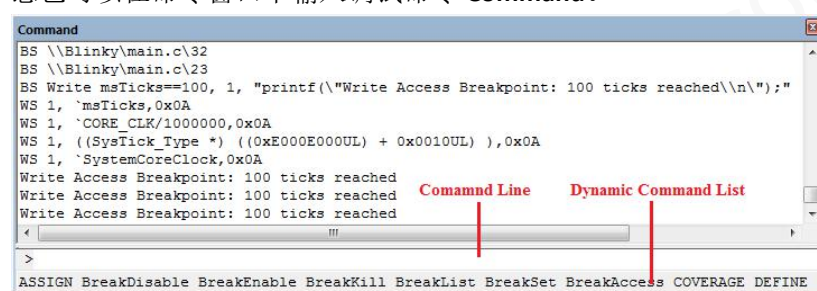
 **Stop** 会停止程序执行。

 **Reset** 会对 CPU 一个复位。

 **Show** 显示到下一个执行的语句（当前 PC 指针的位置）。

命令窗口

您也可以在命令窗口中输入调试命令 **Command**。



在命令行 **Command Line** 输入调试命令或按 F1 访问详细的帮助信息。

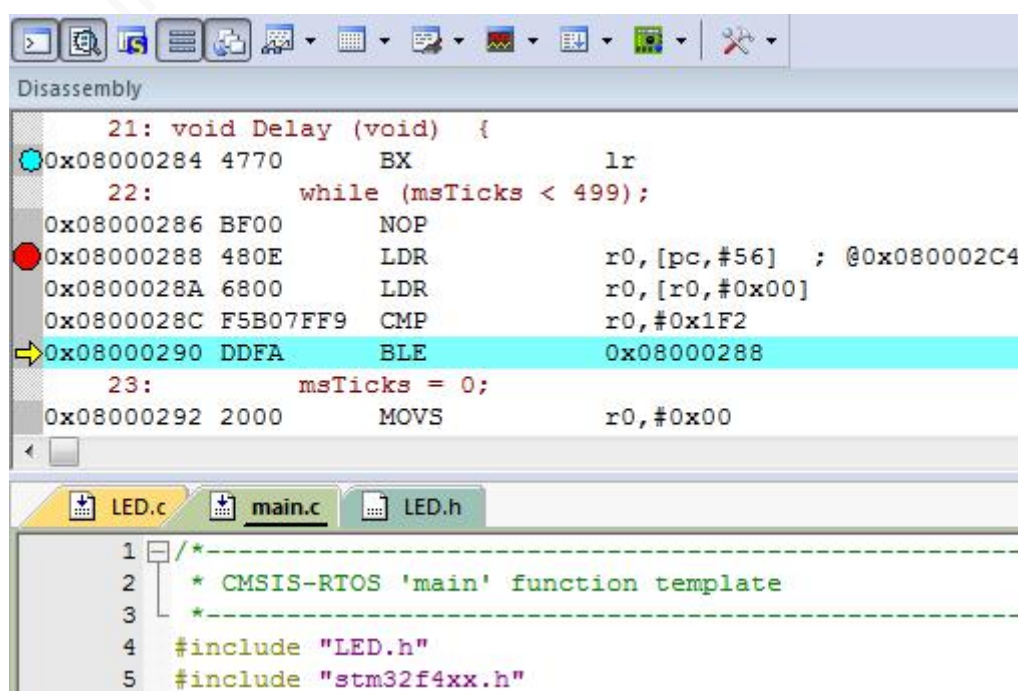
汇编窗口

汇编

Disassembly

窗口显示与源代码混合的汇编代码中的程序执行（如果可用）。当这是活动窗口时，所有调试步进命令在组件级工作。

窗口边界显示断点，书



签和下一个执行语句的标记。

断点

您可以设置断点

- § 在创建或编辑程序源代码时。单击编辑器或“汇编”窗口的灰色边距以设置断点。
- § 使用工具栏中的断点按钮。
- § 使用菜单 **Debug - Breakpoints**。
- § 在命令窗口中输入命令。
- § 使用汇编窗口或编辑器的上下文菜单。

断点窗口

您可以使用断点来定义复杂的断点窗口。

从“调试 **Debug**”菜单中打开“断点 **Breakpoints**”窗口。

使用“当前断点”字段中的复选框来启用或禁用断点。

双击现有断点以修改定义。

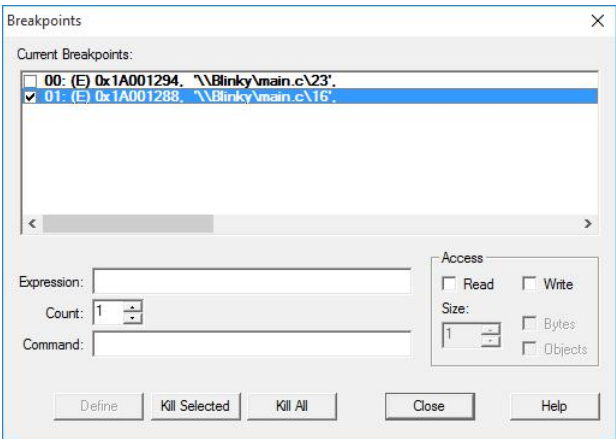
输入一个表达式 **Expression** 来添加一个新的断点。根据表达式，定义以下断点类型之一：

§ 执行断点(**Execution Breakpoint**) (E)：当表达式指定代码地址时创建，并在达到代码地址时触发。

§ 访问断点(**Access Breakpoint**) (A)：当表达式指定访问此内存地址的内存访问（读，写或两者）和触发器时创建。使用 **compare** (==) 运算符来比较指定的值。

如果为断点指定了一个命令 **Command**，μ Vision 将执行命令并恢复执行目标程序。

计数值 **Count value** 指定在断点停止程序执行之前断点表达式为真的次数。



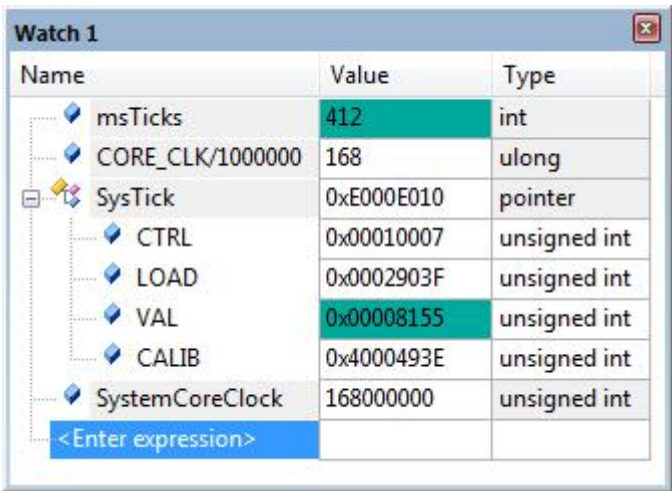
观看窗口

观察窗口允许您观察程序符号，寄存器，内存区域和表达式。

 使用 **View - Watch Windows** 从工具栏或菜单打开 **Watch** 窗口。

将变量添加到 **Watch** 窗口中：

- § 单击字段 < 输入表达式 **Enter expression**>，然后双击或按 F2。
- § 在编辑器中，当光标位于变量上时，使用上下文菜单选择添加<项目名称>到... **Add <item name> to...**
- § 将变量拖放到 **Watch** 窗口。
- § 在命令窗口中，使用 **WATCHSET** 命



Name	Value	Type
msTicks	412	int
CORE_CLK/1000000	168	ulong
SysTick	0xE000E010	pointer
CTRL	0x00010007	unsigned int
LOAD	0x0002903F	unsigned int
VAL	0x00008155	unsigned int
CALIB	0x4000493E	unsigned int
SystemCoreClock	168000000	unsigned int
<Enter expression>		

令。

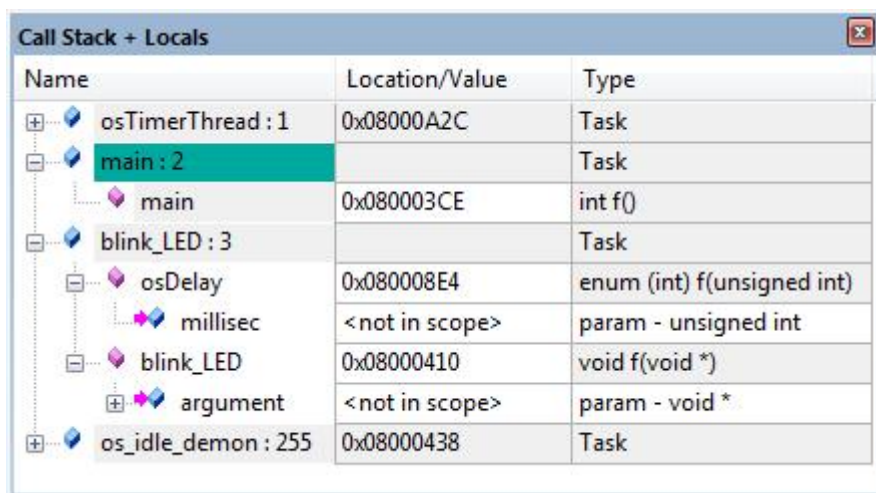
当程序执行停止时，或者在启用视图-定期窗口(View - Periodic Window Update)更新的程序执行期间，窗口内容将被更新。

调用堆栈和本地窗口

“调用堆栈+本地(Call Stack + Locals)”窗口显示当前程序位置的函数嵌套和变量。

从工具栏或菜单中使用 **View - Call Stack Window** 打开 **Call Stack + Locals Window**。

当程序执行停止时，“调用堆栈+本地(Call Stack + Locals)”窗口自动显示当前函数与局部变量一起嵌套。针对使用 CMSIS-RTOS RTX 的应用程序显示线程。



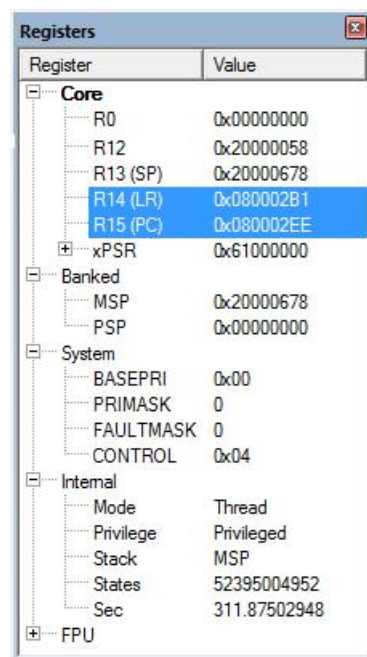
Name	Location/Value	Type
osTimerThread : 1	0x08000A2C	Task
main : 2		Task
main	0x080003CE	int f()
blink_LED : 3		Task
osDelay	0x080008E4	enum (int) f(unsigned int)
millisec	<not in scope>	param - unsigned int
blink_LED	0x08000410	void f(void *)
argument	<not in scope>	param - void *
os_idle_demon : 255	0x08000438	Task

寄存器窗口

寄存器 **Register** 窗口显示微控制器寄存器的内容。

从工具栏或菜单“视图 - 寄存器窗口(View - Registers Window)”中打开“寄存器”窗口。

您可以通过双击寄存器的值来修改寄存器的内容，或按 F2 编辑所选值。目前修改的寄存器是以蓝色突出显示。当程序执行停止时，窗口会更新值。



Register	Value
Core	
R0	0x00000000
R12	0x20000058
R13 (SP)	0x20000678
R14 (LR)	0x080002B1
R15 (PC)	0x080002EE
xPSR	0x61000000
Banked	
MSP	0x20000678
PSP	0x00000000
System	
BASEPRI	0x00
PRIMASK	0
FAULTMASK	0
CONTROL	0x04
Internal	
Mode	Thread
Privilege	Privileged
Stack	MSP
States	52395004952
Sec	311.87502948
FPU	

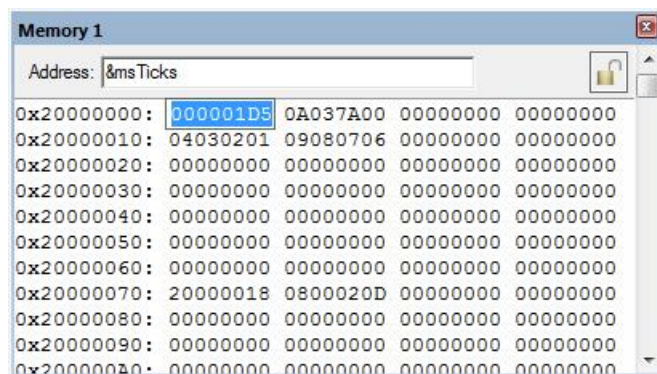
内存窗口

使用内存 **Windows** 监视内存区域。

使用视图 - 内存窗口 **View - Memory Windows** 从工具栏或菜单打开内存窗口。

§ 在地址字段中输入表达式来监视内存区域。

§ 要修改内存内容，请使用内存窗口的上下文菜单中的“修改内存存在...”



Memory 1			
Address: &msTicks			
0x20000000:	000001D5	0A037A00	00000000 00000000
0x20000010:	04030201	09080706	00000000 00000000
0x20000020:	00000000	00000000	00000000 00000000
0x20000030:	00000000	00000000	00000000 00000000
0x20000040:	00000000	00000000	00000000 00000000
0x20000050:	00000000	00000000	00000000 00000000
0x20000060:	00000000	00000000	00000000 00000000
0x20000070:	20000018	0800020D	00000000 00000000
0x20000080:	00000000	00000000	00000000 00000000
0x20000090:	00000000	00000000	00000000 00000000
0x200000A0:	00000000	00000000	00000000 00000000

命令双击该值。

§ 上下文菜单允许您选择输出格式。

§ 要定期更新内存窗口，启用视图 - 定期窗口更新。使用工具箱中的更新 Windows 手动刷新窗口。

🔒 通过单击锁定按钮停止刷新内存窗口。您可以使用锁定功能通过查看第二个内存窗口中的相同部分来比较相同地址空间的值。

外设寄存器

外设寄存器是存储器映射寄存器，处理器可以向其写入和读取以控制外设。菜单外设提供对核心外设的访问，如嵌套向量中断控制器或系统节拍计时器。您可以使用系统查看器访问设备外设寄存器。

注意

菜单外设的内容随选定的微控制器而变化。

系统查看器

系统查看器窗口显示有关设备外设寄存器的信息。

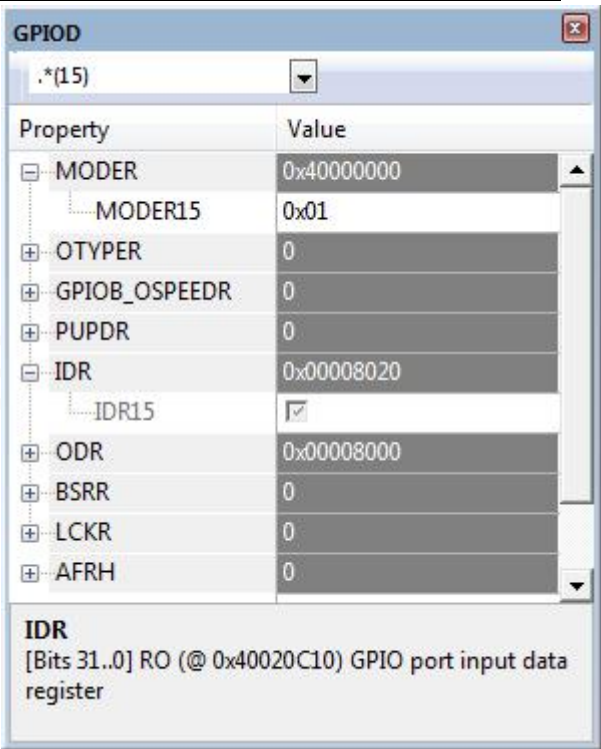
🔍 从工具栏或菜单“外设 - 系统查看器 (Peripherals - System Viewer.)”中打开一个外设寄存器。

使用系统查看器，您可以：

§ 查看外设寄存器的属性和值。启用视图 - 定期窗口 (View — Periodic Window Update) 更新时，周期性更新值。

§ 调试时更改属性值。

§ 在搜索字段中使用 **TR1 正则表达式 (TR1 Regular Expressions)** 搜索特定属性。 μ Vision 用户指南 http://www.keil.com/support/man/docs/uv4/uv4_f_search_expr.htm 的附录介绍正则表达式的语法。



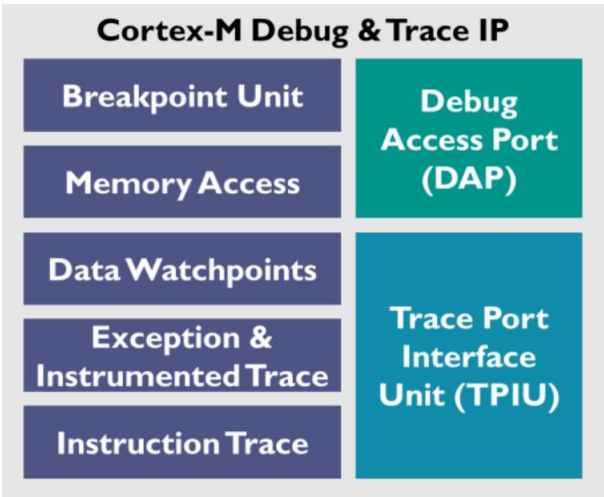
有关访问和使用外设寄存器的详细信息，请参阅在线文档。

跟踪

如前所述，运行停止调试在测试时间关键程序（如电机控制或复杂通信应用程序）方面存在一些限制。作为一个例子，断点和单步执行命令改变了系统的动态行为。或者，使用本节中介绍的跟踪功能分析运行系统。

Cortex-M 处理器集成了 CoreSight 逻辑，可以生成以下跟踪信息：

§ 数据观察点 **Data Watchpoints** 使用数



据值和程序地址记录存储器访问，并且可选择性的停止程序执行。

§ 异常跟踪 **Exception Trace** 输出有关中断和异常的详细信息。

§ 仪器跟踪 **Instrumented Trace** 通信程序事件，并启用 `printf` 风格的调试消息和 RTOS 事件查看器。

§ 指令跟踪 **Instruction Trace** 流程用于记录和分析的完整程序执行。

跟踪端口接口单元 **Trace Port Interface Unit**（TPIU）可在大多数基于 Cortex-M3，Cortex-M4 和 Cortex-M7 处理器的微控制器上使用，并通过以下方式输出以上跟踪信息：

§ 串行线迹输出 **Serial Wire Trace Output**（SWO）仅与串行线调试模式（不使用 JTAG）结合使用，不支持指令跟踪。

§ 4-引脚跟踪 **4-Pin Trace Output** 输出可用于高端微控制器，并具有指令跟踪所需的高带宽。在某些微控制器上，跟踪信息可以存储在可使用标准调试接口读取的片上跟踪缓冲区 **Trace Buffer** 中。

§ Cortex-M3，Cortex-M4 和 Cortex-M7 有一个可选的嵌入式跟踪缓冲区 **Embedded Trace Buffer**（ETB），用于存储上述所有跟踪数据。

§ Cortex-M0+ 具有可选的微追踪缓冲区 **Micro Trace Buffer**（MTB），仅支持指令跟踪。所需的跟踪接口需要由微控制器和调试适配器支持。下表显示了各种调试适配器的支持的跟踪方法。

特点	ULINKpro	ULINKpro-D	ULINK2	ST-Link v2
串行线输出(SWO)	√	√	√	√
最大 SWO 时钟频率	200MHz	200MHz	3.75MHz	2MHz
用于流式传输的 4 引脚跟踪输出	√	×	×	×
嵌入式跟踪缓冲区(ETB)	√	√	√	×
微跟踪缓冲区(MTB)	√	√	√	×

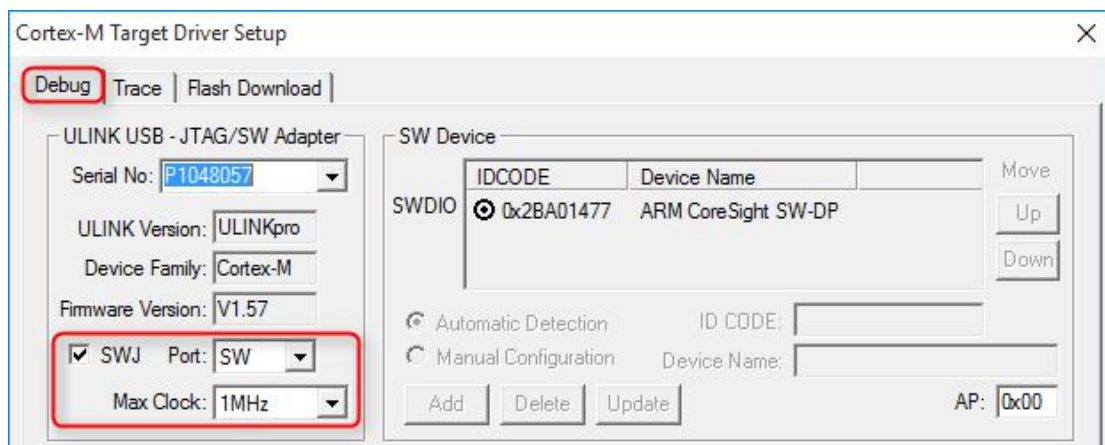
跟踪串行线输出

要使用串行线迹输出（SWO），请使用以下步骤：

 单击工具栏上的**工程选项 Options for Target**，然后选择“**调试 Debug**”选项卡。验证您是否选择并启用了正确的调试适配器。



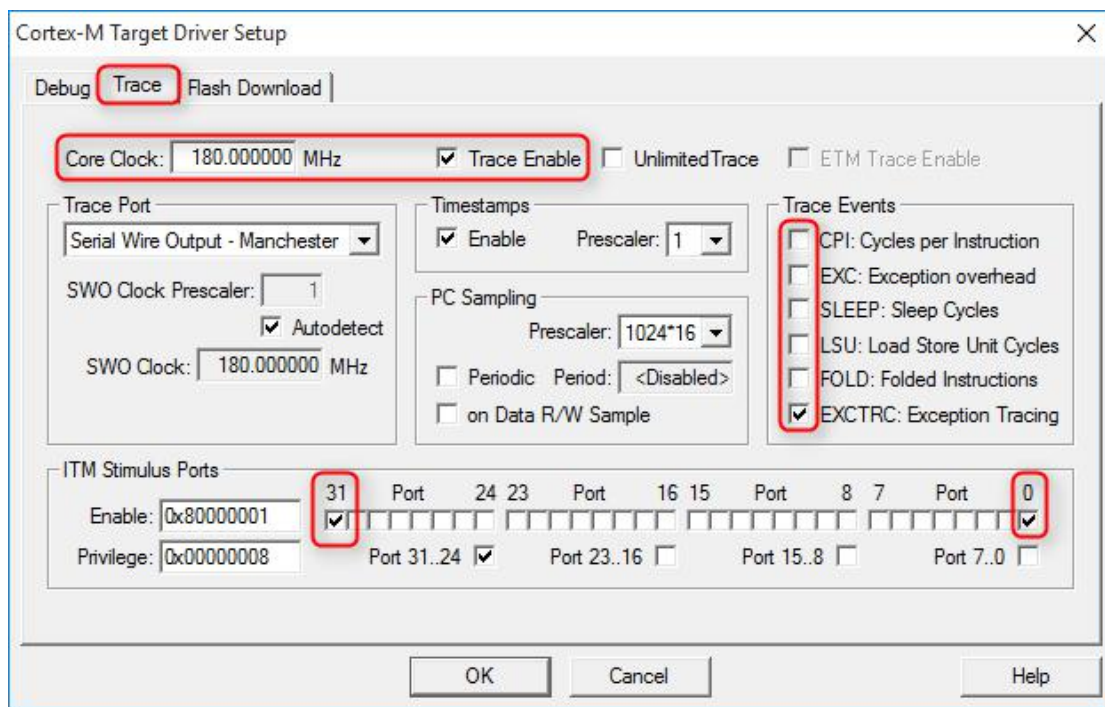
 单击**设置 Settings** 按钮。在**调试 Debug** 对话框中，选择调试 **Port: SW**，并设置与设备调试单元通信的**最大时钟频率(Max Clock)**。



单击跟踪 Trace 选项卡。确保核心时钟 Core Clock 具有正确的设置。设置跟踪启用 Trace Enable 并选择要监视的跟踪事件 Trace Events。

§ 启用 TM Stimulus Port 0 进行 printf 调试。

§ 启用 ITM Stimulus Port 31 以查看 RTOS 事件。



注意

当启用了许多跟踪功能时，串行线输出通信可能会溢出。μ Vision 状态栏显示此类连接错误。

ULINKpro 调试/跟踪适配器具有高跟踪带宽，并且这种通信溢出是罕见的。仅启用当前需要的跟踪功能以避免跟踪通信中的溢出。

跟踪例外

异常跟踪(Exception Trace)窗口显示关于异常和中断的统计数据。

单击跟踪窗口(Trace Windows)并从工具栏中选择跟踪例外，或使用菜单视图 - 跟踪 - 跟踪例外(View - Trace - Trace Exceptions)来打开窗口。

Trace Exceptions									
EXTRC: Exception Tracing Timestamps Enable									
Num	Name	Count	Total Time	Min Time In	Max Time...	Min Time Out	Max Time Out	First Time [s]	Last Time [s]
6	UsageFault	0	0 s						
11	SVCall	0	0 s						
12	DebugMonitor	0	0 s						
14	PendSV	0	0 s						
15	SysTick	1258	74.643 us	59.524 ns	59.524 ns	136.905 ns	1.000 ms	0.00103092	1.25403151
16	WWDG	0	0 s						
17	PVD	0	0 s						
18	TAMP_STAMP	0	0 s						
19	RTC_WKUP	0	0 s						

要在“跟踪例外 Trace Exceptions”窗口中检索数据：

§ 如上所述在“调试设置跟踪”对话框中设置跟踪启用 Trace Enable。

§ 启用 EXTRC：异常跟踪(EXTRC: Exception Tracing)。

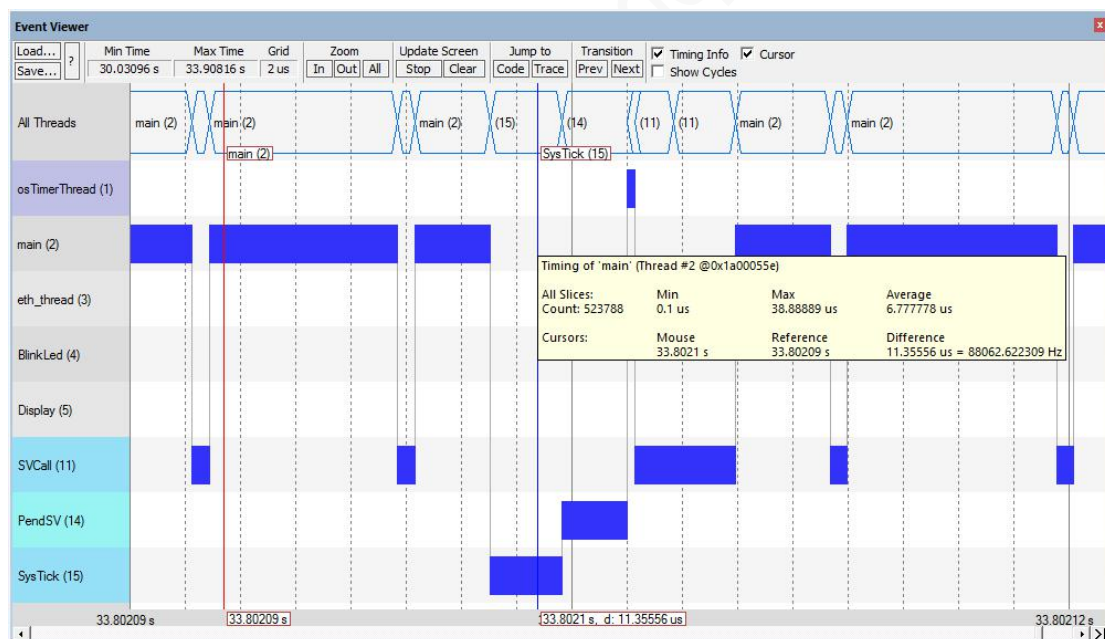
§ 设置时间戳启用(Timestamps Enable)。

注意

逻辑分析仪中配置的变量访问也显示在跟踪数据窗口中。

事件查看器

事件查看器(Event Viewer)显示 RTOS 线程以及中断和异常定时信息。使用菜单 **Debug - OS Support - Event Viewer** 打开此窗口。



要在“事件查看器”窗口中检索数据：

§ 如上所述在“调试设置跟踪”对话框中设置跟踪启用 Trace Enable。

§ 启用 ITM Stimulus Port 31 用于 CMSIS-RTOS 线程时序信息。

§ 启用 EXTRC：异常跟踪 EXTRC: Exception Tracing 给中断和异常定时信息。

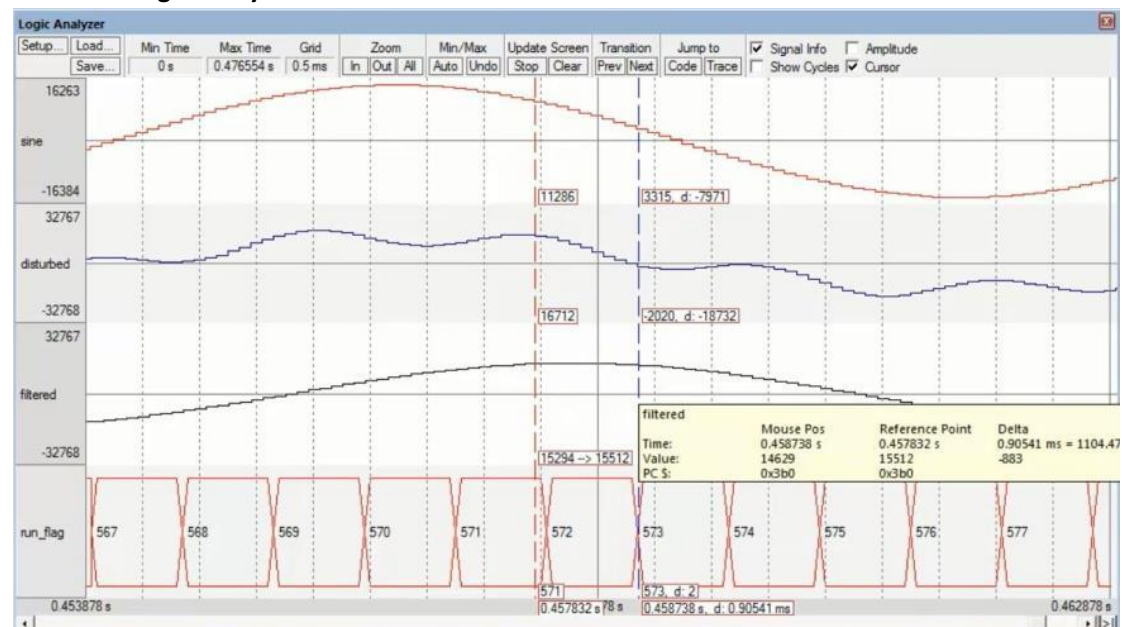
§ 设置时间戳启用。

注意

调试器还提供了无 Trace 的详细 RTOS 和线程状态信息。有关详细信息，请参阅 [CMSIS-RTOS 系统和线程查看器](#)（第 36 页）。

逻辑分析仪

逻辑分析器窗口随时间显示最多四个变量值的更改。要向 Logic Analyzer 添加变量，请在调试模式下右键单击它，然后选择 **Add <variable> to ...-Logic Analyzer**。选择 **View-Analysis Windows-Logic Analyzer** 打开逻辑分析器窗口。



要在逻辑分析仪窗口中检索数据：

§ 如上所述在“调试设置跟踪”对话框中设置跟踪启用 **Trace Enable** 。

§ 设置时间戳启用。

注意

逻辑分析仪中监控的变量访问也显示在跟踪数据窗口中。有关详细信息，请参阅 *μVision 用户指南-调试* http://www.keil.com/support/man/docs/uv4/uv4_debugging.htm。

调试（printf）查看器

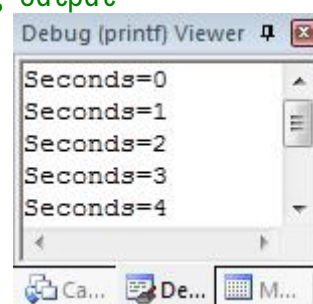
调试（printf）查看器窗口显示通过 ITM Stimulus Port 0 顺序传输的数据流。要启用 printf（）调试，请按照第 42 页所述 [使用编译器软件](#) 组件。

这个 fputc() 函数将任何 printf() 消息（如下所示）重定向到 Debug(printf)Viewer。

```
int seconds; // Second counter
:
while (1)
{
    LED_On (); // Switch on
    delay (); // Delay
    LED_Off (); // Switch off
    delay (); // Delay
    printf ("Seconds=%d\n", seconds++); // Debug output
}
```

单击 **Serial Windows** 并从工具栏中选择 Debug（printf）Viewer，或者使用菜单 View - Serial Windows - Debug（printf）Viewer 打开窗口。

要在 **Debug（printf）Viewer** 窗口中检索数据：



- § 如上所述在“调试设置跟踪”对话框中设置跟踪启用 **Trace Enable**。
- § 设置时间戳启用。
- § 启用 **ITM Stimulus Port 0**。

事件计数器

事件计数器显示累积数字，显示事件被触发的频率。

 从工具栏使用 **Trace Windows - Event Counters**

 从菜单 **View - Trace - Event Counters**

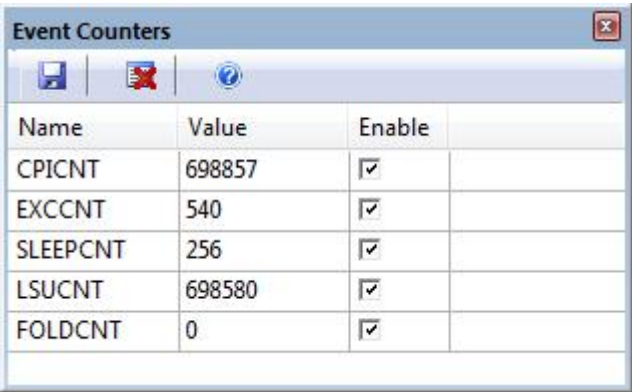
要在此窗口中检索数据：

§ 如上所述在“调试设置跟踪”对话框中设置跟踪启用 **Trace Enable**。

§ 在对话框中根据需要启用事件计数器 **Trace Enable**。

事件计数器是性能指标：

- § **CPICNT**：异常开销周期：表示 Flash 等待状态。
- § **EXCCNT**：每个指令的额外周期：表示异常频率。
- § **SLEEP CNT**：睡眠周期：表示睡眠模式下花费的时间。
- § **LSUCNT**：加载存储单元周期：指示执行多周期加载存储指令所需的附加周期。
- § **FOLD CNT**：折叠指令：表示以零周期执行的指令。



Name	Value	Enable
CPICNT	698857	☒
EXCCNT	540	☒
SLEEP CNT	256	☒
LSUCNT	698580	☒
FOLD CNT	0	☒

4 引脚跟踪输出

4 引脚使用跟踪输出可提供跟踪串行线输出部分中描述的所有功能，但具有较高的跟踪通信带宽。指令跟踪也是可能的。

ULINKpro 调试/跟踪适配器支持此并行 4 引脚跟踪输出（也称为 ETM 跟踪），详细了解程序执行情况。

注意

有关以下功能的更多信息，请参阅 μ Vision 用户指南-调试

http://www.keil.com/support/man/docs/uv4/uv4_debugging.htm。

当与 ULINKpro 一起使用时，MDK 可以流式传输以下高级分析功能的指令跟踪数据：

§ 代码覆盖 **Code Coverage** 标记已执行的代码，并提供有关代码执行的统计信息。这有助于识别零星执行错误，并且通常是软件认证的要求。

§ 性能分析器 **Performance Analyzer** 记录并显示功能和程序块的执行时间。它显示了处理器周期的使用情况，使您可以在优化算法中找到热点。

跟踪数据窗口 **Trace Data Window** 显示 Cortex-M 设备执行指令的历史记录。

跟踪片上跟踪缓冲区

在某些情况下，微控制器或目标硬件上的跟踪输出引脚不可用。作为替代，可以使用支持跟踪数据窗口 **Trace Data Window** 的片上跟踪缓冲区 **Trace Buffer**。

中间件

今天的微控制器设备提供广泛的通信外设，以满足许多嵌入式设计要求。中间件对于有效利用这些复杂的片上外设至关重要。

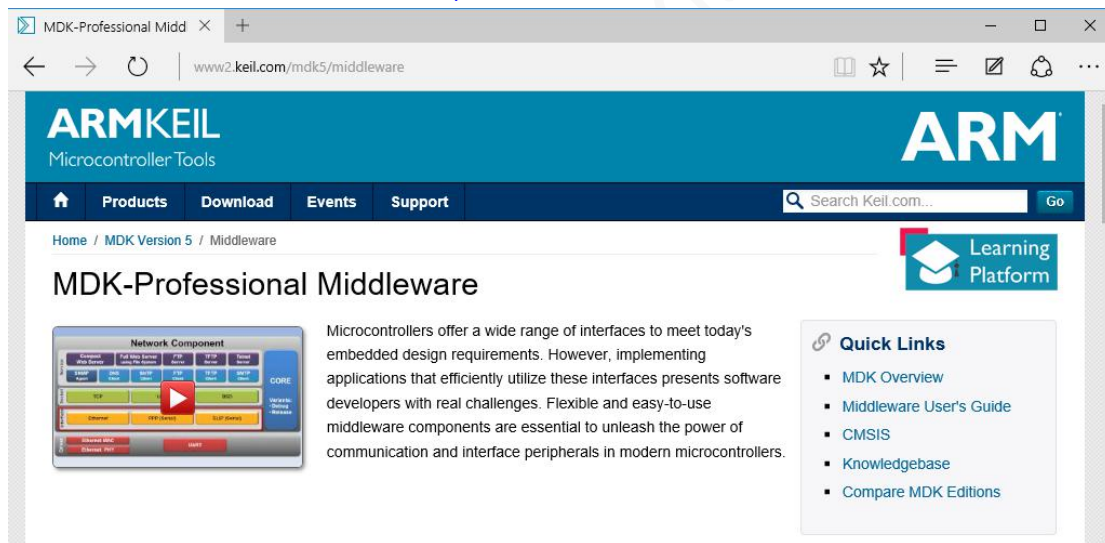
注意

本章介绍 MDK-Professional 和 MDK-Plus 的一部分中间件。MDK 也适用于其他几家供应商的中间件。

有关公共软件包的列表，请参阅 <http://www.keil.com/pack>。

MDK 中间件软件包包含免版税（买断式授权）中间件，其中包含用于 TCP / IP 网络的组件 (TCP/IP networking)、USB 主机(USB Host)和 USB 设备(USB Device)通信、用于数据存储的文件系统(file system)和图形用户界面(graphical user interface)。

有关详细信息，请参阅 www.keil.com/middleware。



此网页提供了中间件的概述，并提供以下链接：

§ MDK 中间件用户指南

§ 设备列表以及有关设备特定驱动程序的信息

§ 有关使用说明的示例项目的信息

中间件使用特定于设备的 CMSIS 驱动程序与设备外围设备进行接口连接。有关详细信息，请参阅第 39-页的 [CMSIS 驱动程序](#)。

组合几个组件对于微控制器应用是常见的。“管理运行时环境 **Manage Run-Time Environment**”对话框可以轻松选择和组合 MDK 中间件。甚至可以使用作为软件包提供的第三方组件来扩展中间件组件列表。

使用 MDK 中间件的典型例子有：

§ 具有存储功能的 Web 服务器：网络和文件系统组件

USB 记忆棒：USB 设备和文件系统组件

§ 具有显示和记录功能的工业控制单元：图形，USB 主机和文件系统组件请参见第 89 页上的 FTP 服务器示例，其中例示了几个中间件组件的配置。

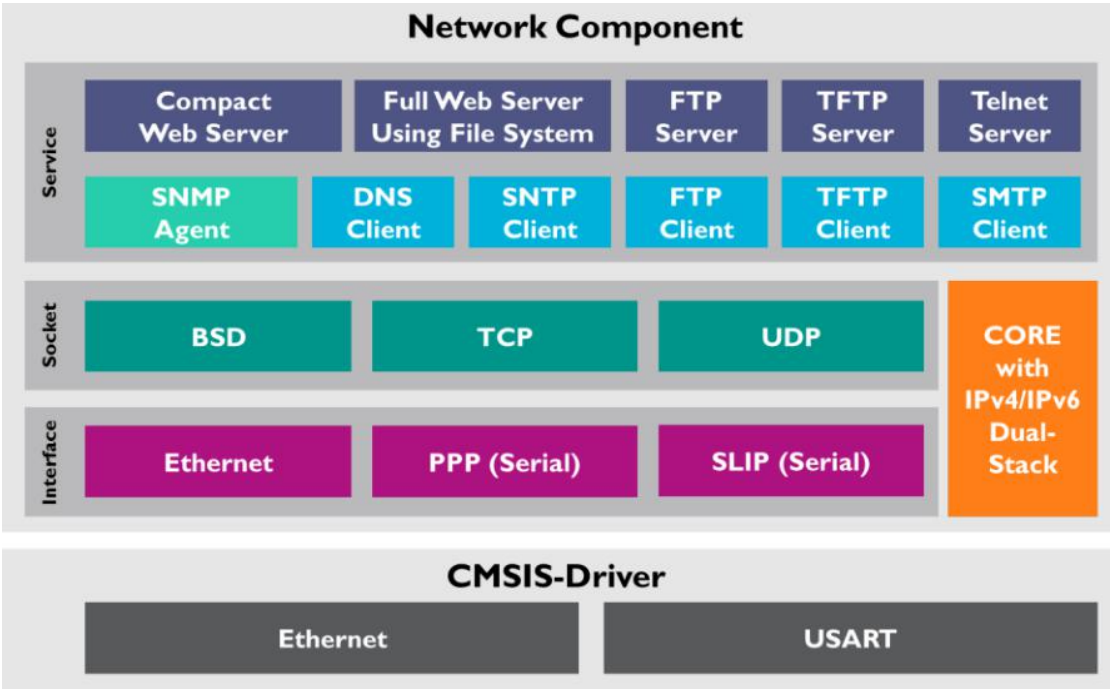
以下部分概述了 MDK 中间件的每个软件组件。

注意

每个安装都提供了 MDK-Professional 的七天评估许可证。有关详细信息，请参阅第 9 页的 [安装](#) 章节。

网络组件

网络组件使用 TCP/IP 通信协议，并支持服务，协议套接字和物理通信接口。它支持 IPv4 和 IPv6 连接。



各种**服务**提供常见联网任务的程序模板。

§ **Compact Web Server** 将 Web 页面存储在 ROM 中，而 **Full Web Server** 则使用**文件系统**组件进行页面数据存储。两台服务器都支持使用 CGI 脚本、AJAX 和 SOAP 技术的动态页面内容。

§ **FTP** 或 **TFTP** 支持文件传输。FTP 提供完整的文件操作命令，而 TFTP 可以引导加载远程设备。两者都可用于客户端和服务端。

§ **Telnet Server** 通过 IP 网络提供命令行界面。

§ **SNMP Agent** 通过简单网络管理协议将设备信息报告给网络管理员。

§ **DNS 客户端** 将域名解析为相应的 IP 地址。它使用可自由配置的名称服务器。

§ **SNTP 客户端** 同步时钟，使设备能够通过数据网络获得准确的时间信号。

§ **SMTP 客户端** 使用简单邮件传输协议(Simple Mail Transfer Protocol)发送状态邮件。

所有**服务**依赖于可以是 **TCP**（面向连接，可靠的全双工协议），**UDP**（用于数据流传输的面向事务的协议）或 **BSD**（Berkeley Sockets 接口）的通信套接字。

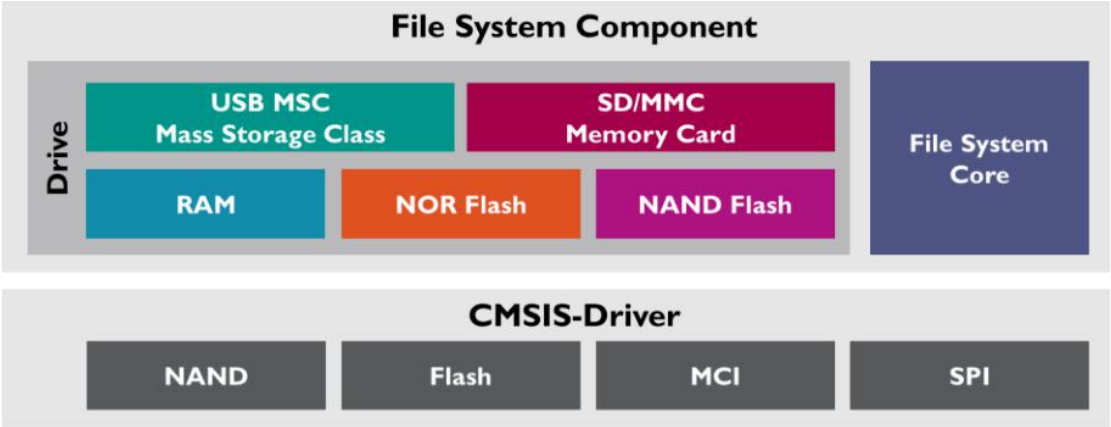
物理**接口**可以是以**以太网**（用于 LAN 连接）或串行连接（例如 **PPP**（用于两个设备之间的直接连接））或 **SLIP**（通过串行连接的 Internet 协议）。

根据接口，网络组件依赖于 **CMSIS 驱动程序**来提供特定于设备的硬件接口。以太网需要**以太网 MAC** 和 **PHY** 驱动，而串行连接（PPP / SLIP）需要 **UART** 或 **Modem** 驱动程序。

网络核心(Network Core)可用于具有广泛诊断消息的调试变量和省略这些诊断的发布变量。它支持使用 IPv4 和 IPv6 的 IP 通信。

文件系统组件

文件系统组件允许您的嵌入式应用程序在存储设备（如 RAM，NAND 或 NOR 闪存，存储卡或 USB 记忆棒）中创建，保存，读取和修改文件。



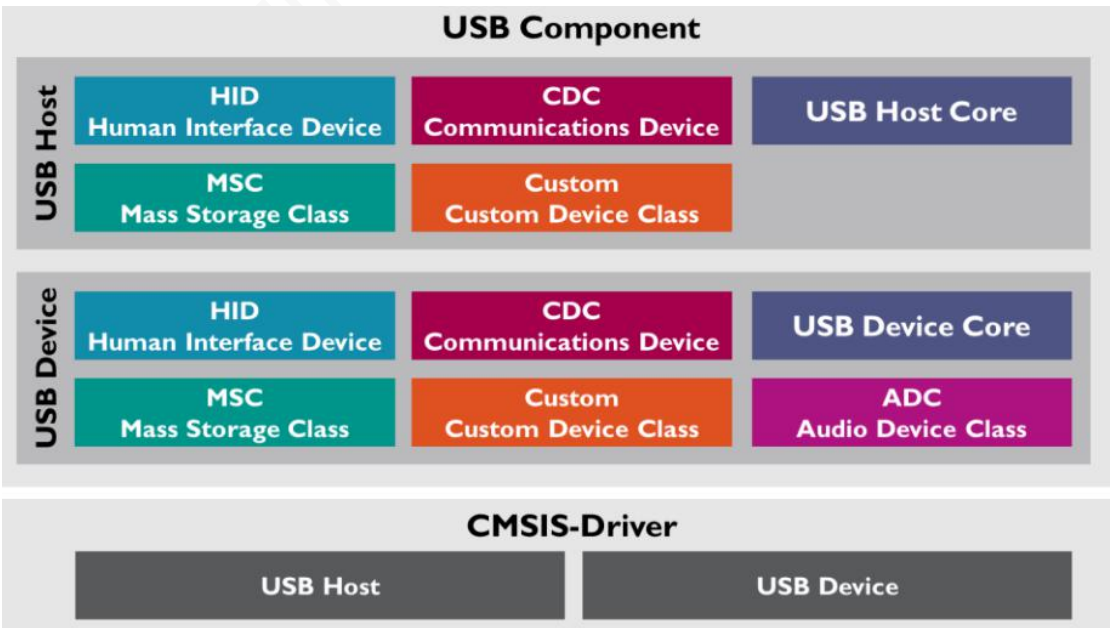
每个存储设备被访问并引用为驱动器（Drive）。文件系统组件支持相同类型的多个驱动器。例如，您的系统中可能有多个存储卡。

文件系统核心(File System Core)是线程安全的，支持同时访问多个驱动器，并使用两个文件名变体中可用的 FAT 系统：8.3 短文件名和最多 255 个字符的长文件名。

为了访问物理介质，例如 NAND 和 NOR 闪存芯片，或使用 MCI 或 SPI 的存储卡，必须存在 [CMSIS 驱动程序](#)。

USB 组件

USB 设备组件实现 USB 主机和设备(USB Host and Device)功能，并使用大多数计算机系统上可用的标准设备驱动程序类，从而避免繁琐的主机驱动程序开发。



§ 人机接口设备类(Human Interface Device Class) (HID) 实现键盘，操纵杆或鼠标。当然了，

HID 也可以用于简单的数据交换。

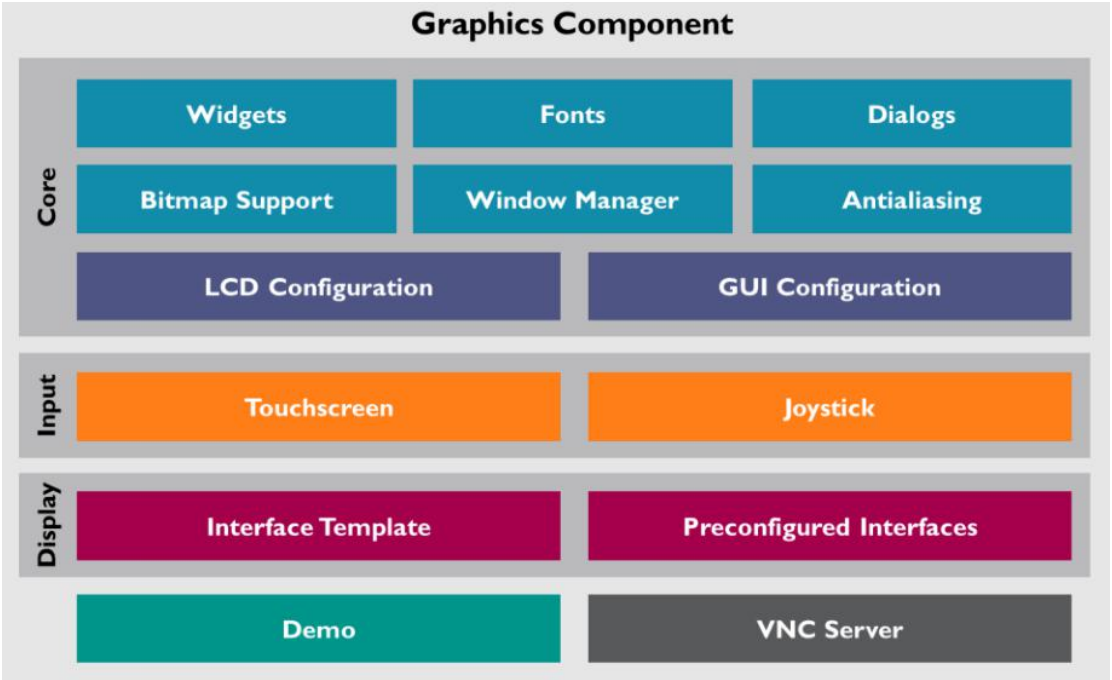
- § 使用大容量存储类(**Mass Storage Class**) (MSC) 进行文件交换 (例如 USB 记忆棒)。
- § 通信设备类(**Communication Device Class**) (CDC) 实现虚拟串行端口 (使用子类 ACM) 或网络连接使用子类 NCM)。
- § 音频设备类(**Audio Device Class**) (ADC) 执行音频流。
- § 对于新的或不支持的 USB 类使用自定义类(**Custom Class**)。

USB 组件支持实现多种设备类的复合 USB 设备(**Composite USB devices**)。

该组件需要 USB [CMSIS 驱动程序](#)。根据应用,它必须符合 USB 1.1 (全速 USB) 和/或 USB 2.0 (高速 USB) 规范。

图形组件

图形组件是一个综合库, 其中包含构建图形用户界面所需的所有内容。



核心功能包括:

- § 一个窗口管理器(**Window Manager**)来操纵任意数量的窗口或对话框。
- § 即用型字体 **Fonts** 和窗口元素, 称为窗口小部件 **Widgets** 和对话框 **Dialogs**。
- § 位图 **Bitmap Support** 支持, 包括 JPEG 和其他常见格式。
- § 抗锯齿 **Anti-Aliasing**, 平滑显示。
- § 灵活的, 可配置的显示 **Display** 和用户界面 **User Interface** 参数。
- § 可以使用触摸屏 **Touch Screen** 或操纵杆 **Joystick** 等输入设备来控制用户界面。

图形组件可以使用预配置的界面 **preconfigured interfaces** 来连接到广泛的显示控制器, 用于普及显示。调整界面模板 **interface template** 以添加对新显示的支持。

VNC 服务器 允许通过图形用户界面,使用网络组件进行远程控制 TCP/IP。

演示 **Demo** 显示所有主要功能, 并且是 GUI 的丰富的代码片段。

物联网 IoT

MDK-Professional 中的中间件提供了可与安全通信和物联网（IoT）连接的 mbed 软件组件的接口。

§ **mbed TLS** 将加密和 SSL / TLS 功能与为嵌入式系统优化的库集合添加。

§ **mbed Client** 实现了 OMA 轻量级 M2M 协议（来自 Open Mobile Alliance <http://openmobilealliance.org>），并连接到将 IoT 设备连接到 Web 应用程序的 mbed 设备服务器。

迁移到中间件版本 7

MDK 具有内置功能，可帮助您将 μ Vision 项目迁移到新的中间件版本 7。大多数组件仅需要配置文件更新（见下文）。然而，网络组件需要更多的迁移工作，因为它已经从 IPv4 到 IPv4 / IPv6 的双栈支持发生了变化。

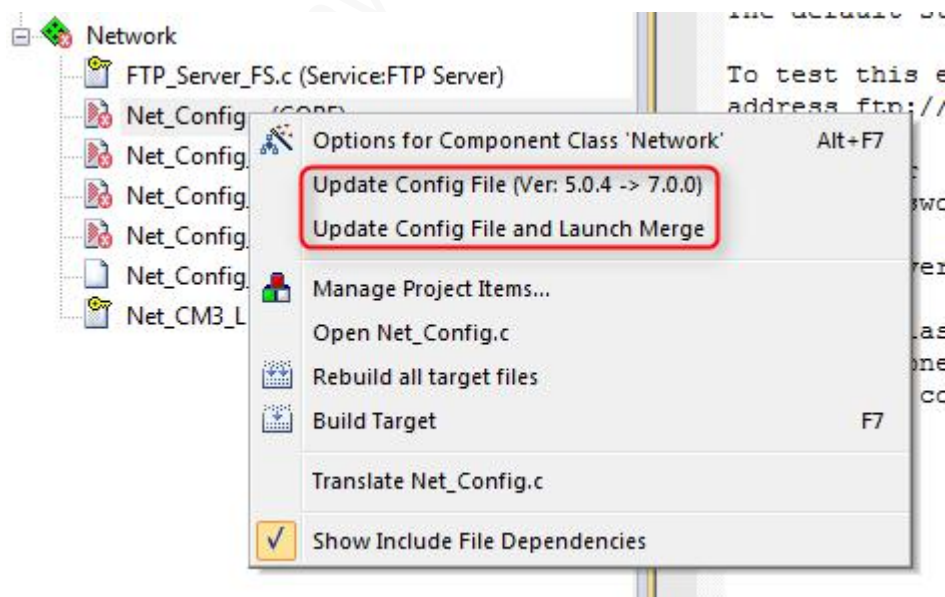
网络组件更改

核心变化

网络组件的核心以前在发布 **Release** 或调试 **Debug** 版本中可用。在中间件版本 7 中，这被更改为 **IPv4 / IPv6 Release** 或 **IPv4 / IPv6 Debug**。当您使用旧组件打开项目时，将在“构建输出 **Build Output**”窗口中看到错误。请更改为相应的新变体。

配置文件更新

μ Vision 项目 **Project** 窗口中的特殊图标突出显示需要更新的配置文件。您可以选择覆盖旧的配置文件或更新并合并内容：



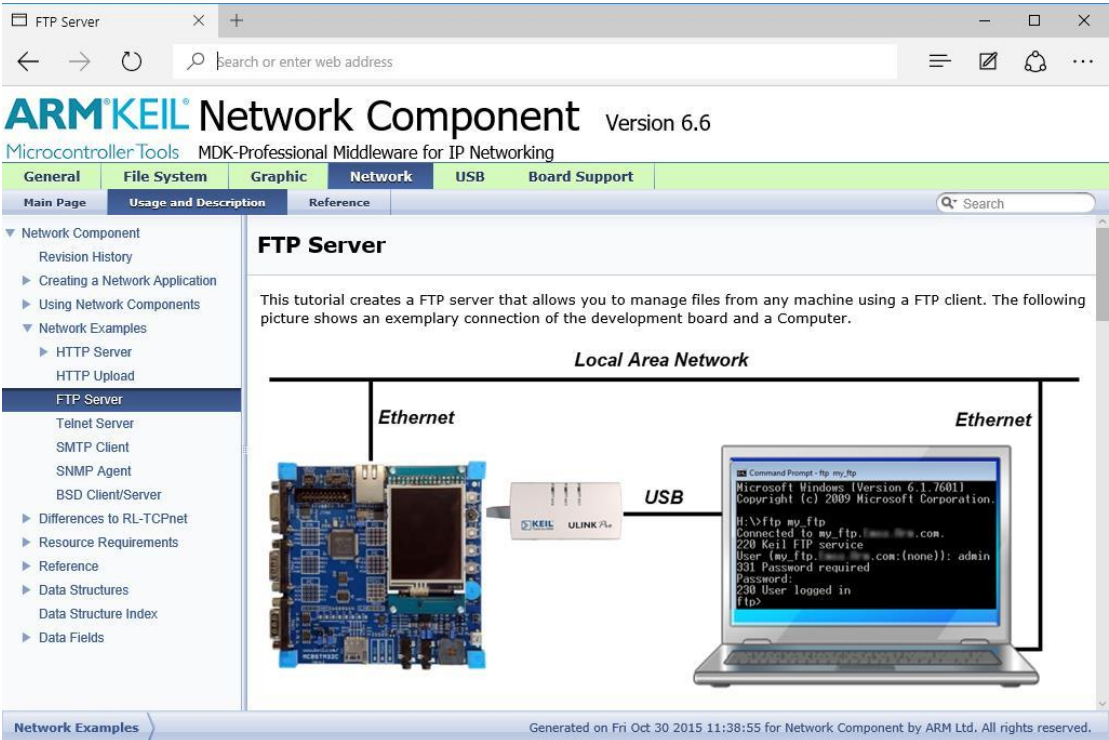
转到 **Tools->Configure Merge Tool** 指定您选择的合并工具。

API 更改

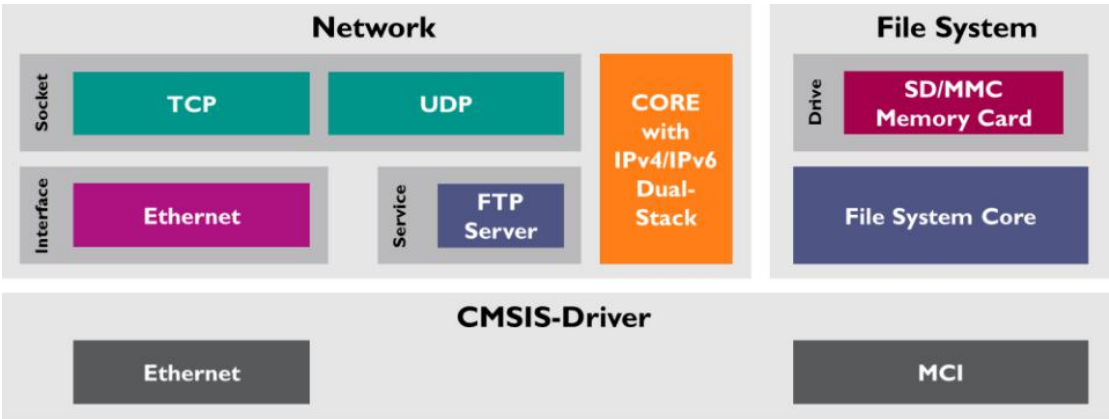
网络组件的文档提供了有关如何将项目从旧的 API 迁移到新 API 的部分。它提供了关于服务，套接字和接口的迁移以及并排的一般建议
是否从中间件 v5 / v6 甚至 RL-TCPnet 迁移 API。

FTP 服务器示例

FTP 服务器示例是一个参考应用程序，显示了几个中间件组件的组合。有关可用的各种示例项目的更多信息，请参阅[使用示例项目验证安装](#)（第 12 页）。
使用 FTP 服务器时，您可以通过 TCP / IP 网络交换和操作文件。中间件文档有关 FTP 服务器和参考应用程序的更多详细信息：



几个中间件组件是此 FTP 服务器的构建块。**文件系统**需要处理文件操作。**网络组件**的各个部分构建了网络接口。
需要 MDK 中间件的以下软件组件来创建 FTP 服务器示例：



如前所述，[CMSIS-Driver](#) 提供了单片机外设和 MDK 中间件之间的接口。

“管理运行时环境 **Manage Run-Time Environment**” 对话框显示为 FTP 服务器选择的软件组件示例：

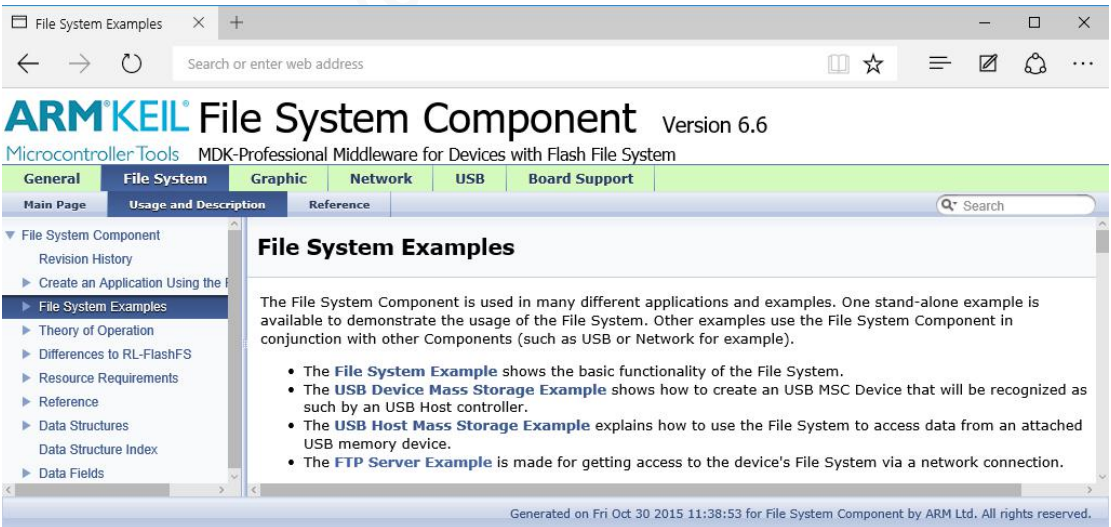
Software Component	Sel.	Variant	Version	Description
CMSIS Driver				Unified Device Drivers compliant to CMSIS
Ethernet (API)			2.01	Ethernet MAC and PHY Driver API for Cortex-M
Ethernet MAC (API)			2.01	Ethernet MAC Driver API for Cortex-M
Ethernet MAC	☒		2.02	Ethernet MAC Driver for LPC1800 Series
Ethernet PHY (API)			2.00	Ethernet PHY Driver API for Cortex-M
DP83848C	☒		6.00	Ethernet PHY DP83848C Driver
KSZ8081RNA	☐		6.00	Ethernet PHY KSZ8081RNA Driver
LAN8720	☐		6.00	Ethernet PHY LAN8720 Driver
ST802RT1	☐		6.00	Ethernet PHY ST802RT1 Driver
Flash (API)			2.00	Flash Driver API for Cortex-M
I2C (API)			2.02	I2C Driver API for Cortex-M
MCI (API)			2.02	MCI Driver API for Cortex-M
MCI	☒		2.01	MCI Driver for LPC1800 Series
NAND (API)			2.01	NAND Flash Driver API for Cortex-M
SPI (API)			2.01	SPI Driver API for Cortex-M
SSP	☒		2.03	SPI (SSP) Driver for LPC1800 Series
USART (API)			2.01	USART Driver API for Cortex-M
USB Device (API)			2.01	USB Device Driver API for Cortex-M
USB Host (API)			2.01	USB Host Driver API for Cortex-M
Compiler				Startup, System Setup
Device				
GPDMA	☒		1.01	GPDMA driver used by RTE Drivers for LPC1800 Series
GPIO	☒		1.00	GPIO driver used by RTE Drivers for LPC1800 Series
SCU	☒		1.00	SCU driver used by RTE Drivers for LPC1800 Series
Startup	☒		1.0.0	System Startup for NXP LPC1800 Series
File System		MDK-Pro	6.2.4	File Access on various storage devices

Software Component	Sel.	Variant	Version	Description
File System		MDK-Pro	6.2.4	File Access on various storage devices
CORE	☒	LFN	6.2.4	File System with Long Filename support for
Drive				Storage Devices and Media Types
Graphics		MDK-Pro	5.26.1	User Interface on graphical LCD displays
Network		MDK-Pro	6.2.0	IP Networking using Ethernet or Serial protocols
CORE	☒	Release	6.2.0	Networking Core for Cortex-M (Release)
Interface				Connection Mechanism
ETH	☒		6.2.0	Network Ethernet Interface
PPP	☐	Standard Modem	6.2.0	Network PPP over Serial Interface - Standard Modem
SLIP	☐	Standard Modem	6.2.0	Network SLIP Interface - Standard Modem
Service				Network Services
DNS Client	☐		6.2.0	DNS Client
FTP Client	☐		6.2.0	FTP Client
FTP Server	☒		6.2.0	FTP Server
SMTP Client	☐		6.2.0	SMTP Client
SNMP Agent	☐		6.2.0	SNMP Agent
SNTP Client	☐		6.2.0	SNTP Client
TFTP Client	☐		6.2.0	TFTP Client
TFTP Server	☐		6.2.0	TFTP Server
Telnet Server	☐		6.2.0	Telnet Server
Web Server Co...	☐		6.2.0	Web Server (HTTP) with Read-only Web Resources
Web Server	☐		6.2.0	Web Server (HTTP) with Web Resources on
Socket				Network protocol
BSD	☐		6.2.0	BSD Socket
TCP	☒		6.2.0	TCP Socket
UDP	☒		6.2.0	UDP Socket

使用中间件

使用 MDK 中间件组件创建自己的应用程序。有关详细信息，请参阅“MDK 中间件用户指南”，其中包含描述每个组件的部分：

- § 示例项目 **Example projects** 概述了软件组件的关键产品功能。这些示例在几个评估板上进行了测试，实施和验证。将其作为参考应用程序或您的开发的起点。
- § 资源需求 **Resource Requirements** 描述了 CMSIS-RTOS 的线程和堆栈资源以及内存占用。
- § 创建应用程序 **Create an Application** 包含嵌入式应用程序中使用组件所需的步骤。
- § 参考文件 **Reference** 包含 API 和文件文档。



学习平台 www.keil.com/learn 提供了几个教程和视频，例示了中间件的典型用例。还请参考这些应用笔记：

§ 具有文件系统和图形用户界面的 USB 主机应用程序：

www.keil.com/appnotes/docs/apnt_268.asp

§ 支持 Web 的 MEMS 传感器平台：

www.keil.com/appnotes/docs/apnt_271.asp

§ 支持 Web 的录音机:

www.keil.com/appnotes/docs/apnt_272.asp

§ 带 USB 设备接口的模拟/数字数据记录器:

www.keil.com/appnotes/docs/apnt_273.asp

使用各种中间件组件的通用步骤如下:

§ [添加软件组件 Add Software Components](#) (第 94 页): 在“管理运行时环境”对话框中, 选择应用程序所需的软件组件。

§ [配置中间件 Configure Middleware](#) (第 96 页): 调整相关配置文件中软件组件的参数。

§ [配置驱动程序 Configure Drivers](#) (第 98 页): 识别和配置将中间件组件连接到微控制器的物理 I/O 引脚的外围接口。

§ [调整系统资源 Adjust System Resources](#) (第 99 页): 中间件组件使用 RTOS, 内存和堆栈资源, 这可能意味着配置, 例如 CMSIS-RTOS RTX。

§ [实现应用程序功能 Implement Application Features](#) (第 100 页): 使用所选组件的 API 函数来实现应用程序特定的行为。代码模板可帮助您创建相关的源代码。

§ [构建和下载 Build and Download](#) (第 103 页): 编译和链接应用程序后, 请使用第 64 页的使用调试器中所述的步骤将映像下载到目标硬件。

§ [验证和调试 Verify and Debug](#) (第 103 页): 测试实用程序以及调试和跟踪功能在[创建应用程序一章](#)中有介绍。

USB 设备 HID 示例

虽然上述步骤是通用的并且适用于 MDK 中间件的所有组件, 但以下 USB 设备 HID 示例在实践中显示了这些步骤。此示例创建一个通过 USB 将微控制器连接到主机的 USB HID 设备应用程序。在 PC 上, 实用程序 HIDClient.exe 用于控制开发板上的 LED。

该 USB 设备 HID 示例使用 LPC1857 微控制器的 MCB1800 开发板。它是基于项目 Blinky 与 [CMSIS-RTOS RTX](#) 在第 45 页以及源文件 main.c, LED.c, LED.h 和配置文件。

注意

在其他入门工具包或评估板上使用此示例时, 必须调整代码和引脚配置。该示例可作为支持 CMSIS 驱动程序的许多单片机设备系列的 Pack Installer 中的预构建项目提供。

添加软件组件

要创建 USB 设备 HID 示例, 请从第 45 页所述的 CMSIS-RTOS RTX 项目闪烁开始。

◆ 使用“管理运行时环境”对话框添加特定的软件组件。

从 USB 组件 (如第 85 页所述):

§ 选择:: USB:CORE 包括 USB 通信所需的基本功能。

§ 将:: USB:Device 设置为'1'以创建一个 USB 设备实例。

§ 设置 ::USB:Device:HID 设置为'1'以创建 HID 设备类实例。如果您选择同一类的多个实例或包括其他设备类,

您将创建一个复合 USB 设备。

从 CMSIS 驱动程序（第 39 页所述）：

选择::CMSIS Driver:USB Device (API) 适合您应用的适当驱动程序。一些设备可能具有用于 USB 全速和高速的特定驱动程序，而其他微控制器可能具有组合驱动程序，这里选择 USB0。

提示：单击“说明”列中的超链接可查看每个软件组件的详细文档。

使用“管理运行时环境”对话框添加特定的软件组件。下图显示了添加这些组件后的“管理运行时环境”对话框。

Manage Run-Time Environment				
Software Component	Sel.	Variant	Version	Description
+				Cortex Microcontroller Software Interface Compo
+				Unified Device Drivers compliant to CMSIS-Driver
+			2.01	Ethernet MAC and PHY Driver API for Cortex-M
+			2.01	Ethernet MAC Driver API for Cortex-M
+			2.00	Ethernet PHY Driver API for Cortex-M
+			2.00	Flash Driver API for Cortex-M
+			2.02	I2C Driver API for Cortex-M
+			2.02	MCI Driver API for Cortex-M
+			2.01	NAND Flash Driver API for Cortex-M
+			1.00	SAI Driver API for Cortex-M
+			2.01	SPI Driver API for Cortex-M
+			2.01	USART Driver API for Cortex-M
+			2.01	USB Device Driver API for Cortex-M
+			2.01	USB Device Driver API for Cortex-M
+			2.01	USB Host Driver API for Cortex-M
+				ARM Compiler Software Extensions
+				Startup, System Setup
+		MDK-Pro	6.6.0	File Access on various storage devices
+		MDK-Pro	5.30.0	User Interface on graphical LCD displays
+		MDK-Pro	6.5.2	IP Networking using Ethernet or Serial protocols
+		MDK-Pro	6.6.6	USB Communication with various device classes
+			6.6.6	USB Core for Cortex-M
+			6.6.6	USB Device
+			6.6.6	USB Host
+				USB Device Classes
+			6.6.6	USB Device: Audio Device Class (ADC)
+			6.6.6	USB Device: Communication Device Class (CDC)
+			6.6.6	USB Device: Custom Class
+			6.6.6	USB Device: Human Interface Device (HID) Class
+			6.6.6	USB Device: Mass Storage Class (MSC)

配置中间件

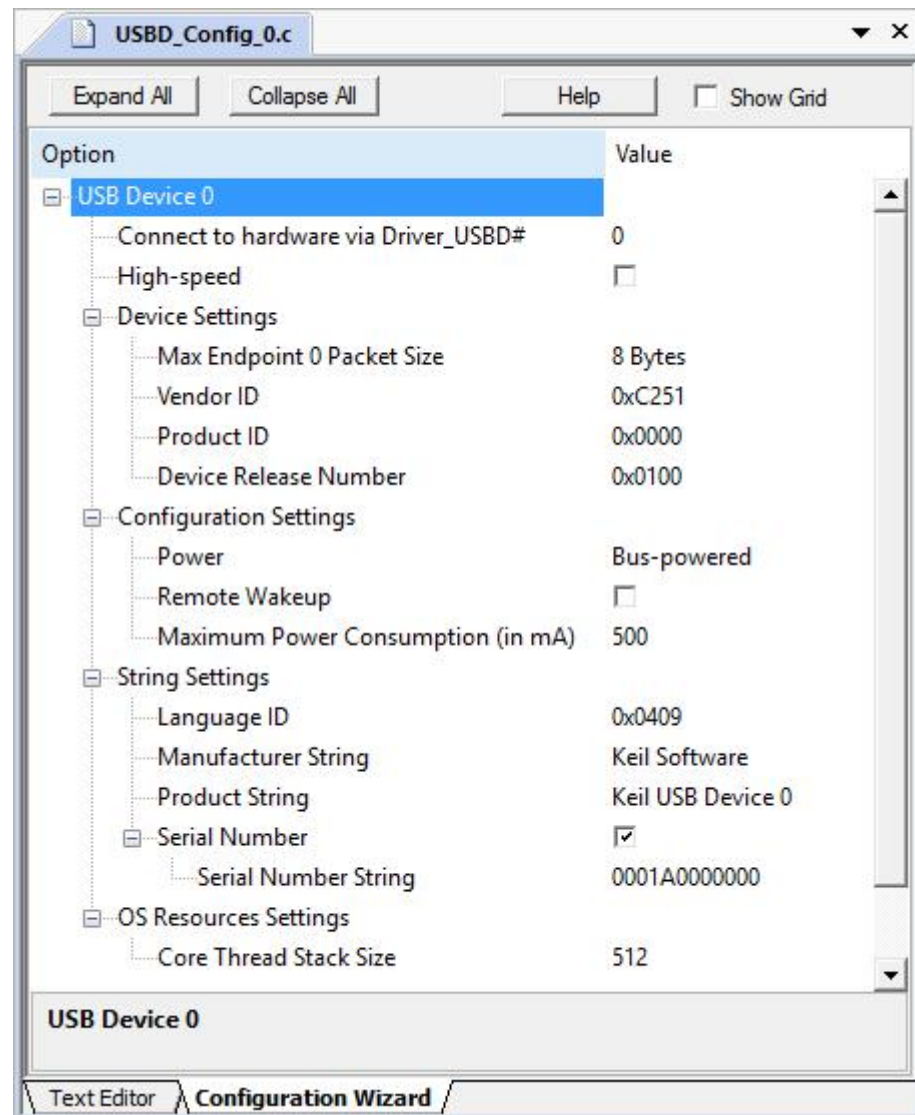
每个 MDK 中间件组件都有一组配置文件，用于调整应用程序特定的参数并确定驱动程序接口。从组件类组中的“项目 **Project**”窗口访问这些配置文件。它们通常具有名称，如 <Component>_Config_0.c 或 <Component>_Config_0.h。

这些文件中的某些设置需要在下一部分主题的驱动程序和设备配置文件（RTE_Device.h）中

进行相应的设置。

对于 USB HID 设备示例，有两个配置文件可用：

USBD_Config_0.c 和 USBD_Config_HID_0.h。



文件 USBD_Config_0.c 包含特定 USB 设备的一些重要设置：

§ 设置 **Connect to Hardware via Driver_USBD#** 指定了反映外设接口的 *控制结构*，在这种情况下，用作设备接口的 USB 控制器。对于只有一个 USB 控制器的微控制器，数字为“0”。有关详细信息，请参阅第 39 页的 CMSIS 驱动程序。

§ 如果 USB 控制器支持，请选择“高速 **High-Speed**”。使用此设置需要支持 USB 高速通信的驱动程序。

§ 将供应商 Vendor ID (VID) 设置为私有 VID。USB 实施者论坛

<http://www.usb.org/developers/vendor> 提供了有关如何申请有效的供应商 ID 的更多信息。

§ 每个设备都需要一个唯一的产品 ID。主机的操作系统与 VID 一起使用，为您的设备找到合适的驱动程序。

§ 设置制造商和产品字符串以识别 PC 操作系统中的 USB 设备。

文件 USBD_Config_HID_0.h 包含设备类特定的端点设置。对于此示例，不需要更改。

配置驱动程序

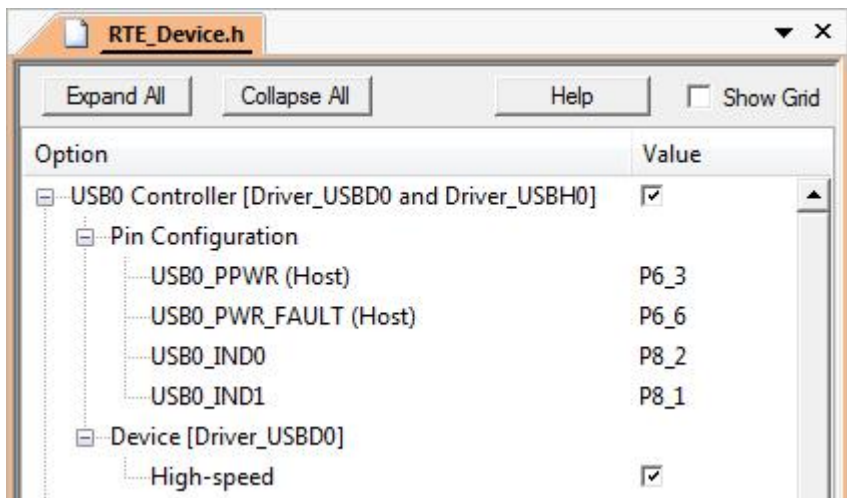
驱动程序具有定义属性的某些属性，例如 I/O 引脚分配，时钟配置或 DMA 通道的使用。对于许多设备，RTE_Device.h 配置文件包含这些驱动程序属性。它通常需要配置应用程序使用的实际外设接口。根据微控制器设备，您可以启用不同的硬件外设，指定引脚设置或更改实施的时钟设置。

USB HID 设备示例需要以下设置：

§ 启用 USB0 控制器 **USB0 Controller** 并展开本节。

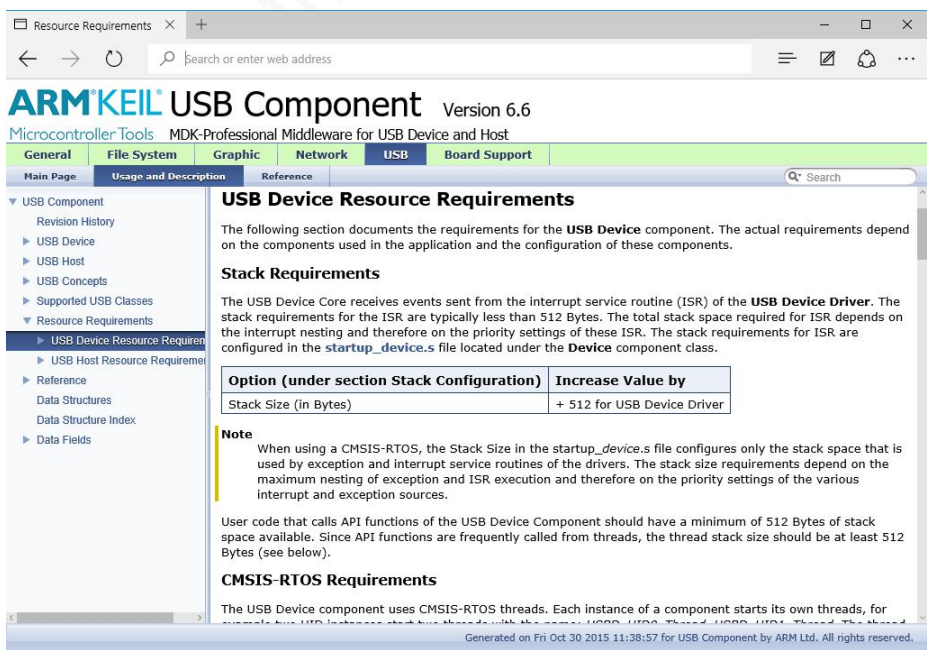
§ 如下所示更改引脚配置 **Pin Configuration**。

§ 启用设备：高速。



调整系统资源

每个中间件组件都具有一定的内存和 RTOS 资源要求。“MDK 中间件用户指南”中的“资源要求 **Resource Requirements**”一节记录了每个组件的要求。



大多数中间件组件使用 CMSIS-RTOS。重要的是，RTOS 配置为支持要求。

对于 CMSIS-RTOS RTX，RTX_Conf_CM.c 文件配置线程和堆栈设置。有关更多信息，请参阅第 30 页的 CMSIS-RTOS RTX 配置。

对于 USB HID 设备示例，适用以下设置：

§ ::USB:Device 组件需要一个线程（称为 USB_Dn_CoreThread）和 512 个字节的用户提供的堆栈。

§ ::USB:Device:HID 组件还需要一个线程（称为 USB_D_HIDn_Thread）和 512 个字节的用户提供的堆栈。

通过 RTX_Conf_CM.c 文件中的设置反映这些要求：

§ 并发运行线程数(**Number of concurrent running threads**): 6（默认值）足以同时运行 USB 设备组件的两个线程。如果用户应用程序执行其他线程，请调整此设置。

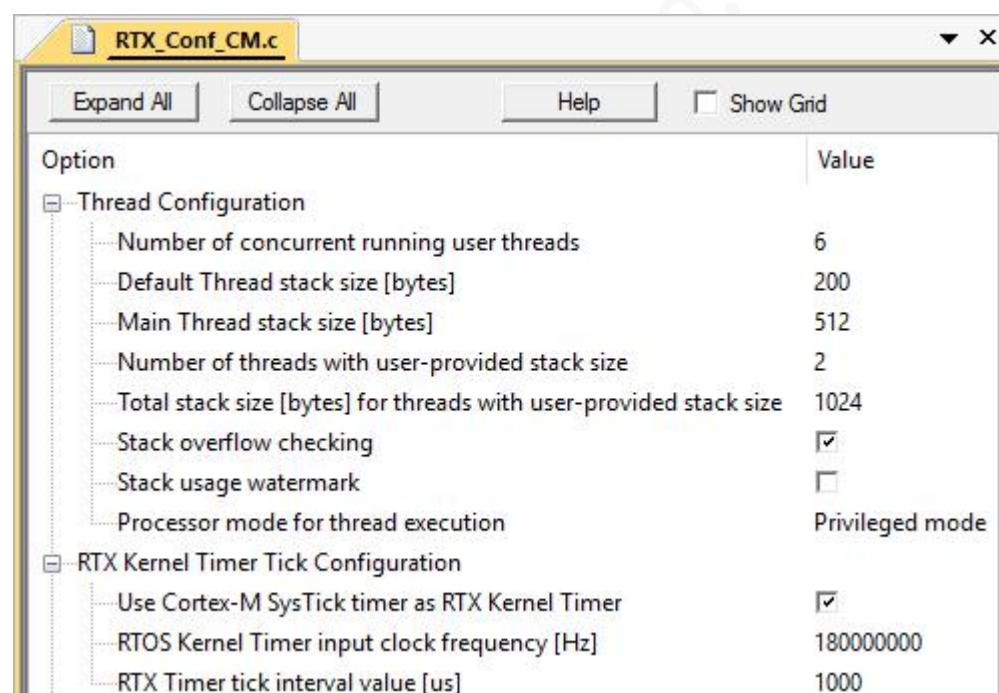
§ 默认线程堆栈大小(**Default Thread stack size**)[字节]: 这个设置并不重要，因为 USB 组件在用户提供的堆栈上运行。

§ 主线程堆栈大小 **Main Thread stack size** [字节]: 512.初始化 USB 设备组件的 API 调用需要堆栈。

§ 具有用户提供的堆栈大小的线程数: 2.使用用户提供的堆栈指定两个线程（用于:: USB: Device 和:: USB: Device: HID）。

§ 具有用户提供的堆栈大小的线程的总堆栈大小[字节]: 1024.指定两个线程的总堆栈大小。

§ 定时器时钟值 **Timer Clock value**[Hz]需要与系统时钟（18000000）匹配



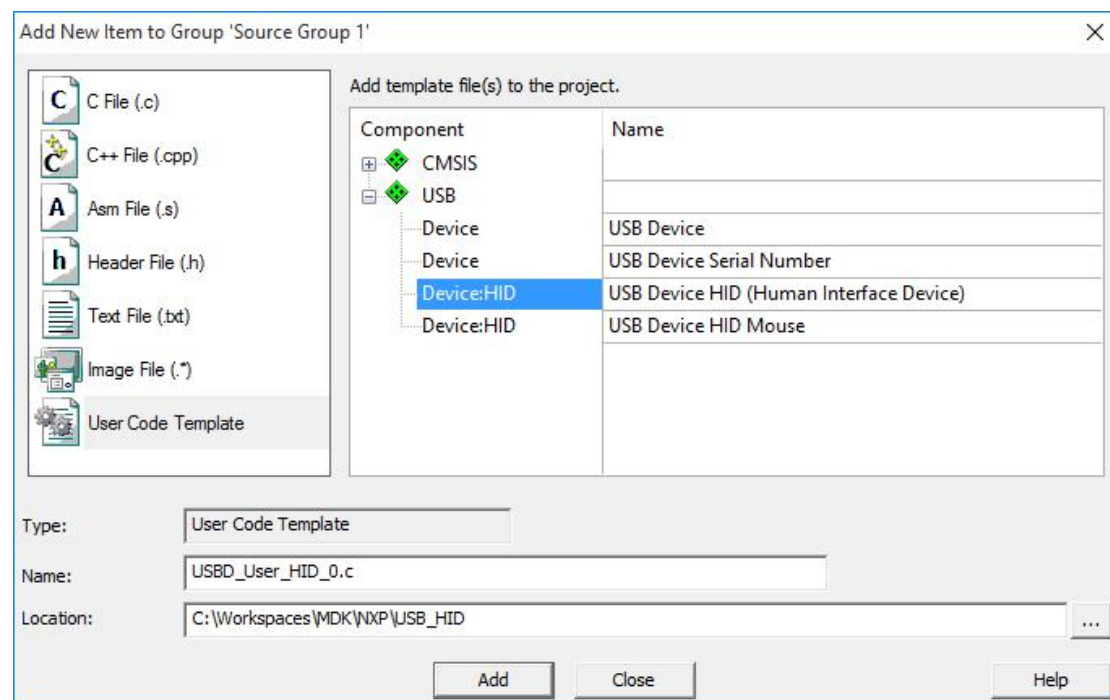
实施应用功能

现在，创建实现应用程序特定功能的代码。这包括对最初为 CMSIS-RTOS RTX Blinky 项目创建的文件 main.c，LED.c 和 LED.h 的修改，第 45 页。

中间件提供用户代码模板 **User Code Templates** 作为应用程序的起点。

👉 在“项目 **Project**”窗口中，右键单击“源组 1 **Source Group 1**”，然后打开“将新建项目添加到组 **Add New Item to Group.**”对话框。从::USB:Device:HID - USB Device HID (Human

Interface Device)中选择用户代码模板，然后单击添加 **Add**。



要将 PC USB 应用程序连接到微控制器设备，请修改处理 USB 主机数据的 USBD_HID0_SetReport()函数。对于此示例，数据是使用实用程序 HIDClient.exe 创建的。

在编辑器中打开文件 USBD_User_HID_0.c，修改代码，如下所示。这将控制评估板上的 LED。

```
#include "LED.h" // access functions to LEDs
:
bool USBD_HID0_SetReport (uint8_t rtype, uint8_t req, uint8_t rid,
                          const uint8_t *buf, int32_t len)
{
    uint8_t i;
    switch (rtype)
    {
        case HID_REPORT_OUTPUT:
            for (i = 0; i < 4; i++)
            {
                if (*buf & (1 << i)) LED_On (i);
                else LED_Off (i);
            }
            break;
        case HID_REPORT_FEATURE:
            break;
    }
    return true;
}
```

扩展文件 LED.c 中的功能，以控制板上的几个 LED，并删除闪烁 LED 的线程，因为不再需要。

在编辑器中打开文件 LED.c，修改代码如下图所示。

```
/*-----*/
* File LED.c
*-----*/
```

```

#include "SCU_LPC18xx.h"
#include "GPIO_LPC18xx.h"
#include "cmsis_os.h" // ARM::CMSIS:RTOS:Keil RTX
const GPIO_ID LED_GPIO[] = // LED GPIO definitions
{
    { 6, 24 },
    { 6, 25 },
    { 6, 26 },
    { 6, 27 }
};
void LED_Initialize (void)
{
    GPIO_PortClock (1); // Enable GPIO clock
    /* Configure pin: Output Mode with Pull-down resistors */
    SCU_PinConfigure (13, 10,
(SCU_CFG_MODE_FUNC4|SCU_PIN_CFG_PULLDOWN_EN));
    GPIO_SetDir (6, 24, GPIO_DIR_OUTPUT);
    GPIO_PinWrite (6, 24, 0);
    SCU_PinConfigure (13, 11,
(SCU_CFG_MODE_FUNC4|SCU_PIN_CFG_PULLDOWN_EN));
    GPIO_SetDir (6, 25, GPIO_DIR_OUTPUT);
    GPIO_PinWrite (6, 25, 0);
    SCU_PinConfigure (13, 12,
(SCU_CFG_MODE_FUNC4|SCU_PIN_CFG_PULLDOWN_EN));
    GPIO_SetDir (6, 26, GPIO_DIR_OUTPUT);
    GPIO_PinWrite (6, 26, 0);
    SCU_PinConfigure (13, 13,
(SCU_CFG_MODE_FUNC4|SCU_PIN_CFG_PULLDOWN_EN));
    GPIO_SetDir (6, 27, GPIO_DIR_OUTPUT);
    GPIO_PinWrite (6, 27, 0);
}
void LED_On (uint32_t num)
{
    GPIO_PinWrite (LED_GPIO[num].port, LED_GPIO[num].num, 1);
}
void LED_Off (uint32_t num)
{
    GPIO_PinWrite (LED_GPIO[num].port, LED_GPIO [num].num, 0);
}

```

 在编辑器中打开文件 LED.h，并对其进行修改，以配合 LED.c 的更改。LED_On() 和 LED_Off() 的功能现在有一个参数。

```

/*-----*/
* File LED.h
*-----*/

void LED_Initialize ( void );
void LED_On ( uint32_t num );
void LED_Off ( uint32_t num );

```

更改文件 main.c，如下所示。不要启动闪烁 LED 的线程，添加代码来初始化并启动 USB 设备组件。有关详细信息，请参阅“中间件用户指南”。

```

/*-----*/
* File main.c
*-----*/
#define osObjectsPublic // define objects in main module
#include "osObjects.h" // RTOS object definitions
#include "LPC18xx.h" // Device header
#include "LED.h" // Initialize and set GPIO Port
#include "rl_usb.h" // Keil.MDK-Pro::USB:CORE
/*
* main: initialize and start the system
*/
int main (void)
{
    osKernelInitialize (); // Initialize CMSIS-RTOS
// initialize peripherals here
    LED_Initialize (); // Initialize LEDs
    USBD_Initialize (0); // USB Device 0 Initialization
    USBD_Connect (0); // USB Device 0 Connect
    osKernelStart (); // Start thread execution
    while (1);
}

```

构建和下载

构建项目并将其下载到目标，如章节创建应用程序（第 45 页）和使用调试器（第 64 页）中所述。

验证和调试

使用其他 USB 电缆将开发板连接到 PC。这提供了到微控制器的 USB 设备外围设备的连接。

连接电路板后，会显示一条通知，表明安装了 USB HID 设备的设备驱动程序。

作为 MDK 一部分的实用程序 HIDClient.exe 可以测试 PC 和开发板之间的连接。此实用程序位于 MDK 安装文件夹\Keil\ARM\Utilities\HID_Client\Release。

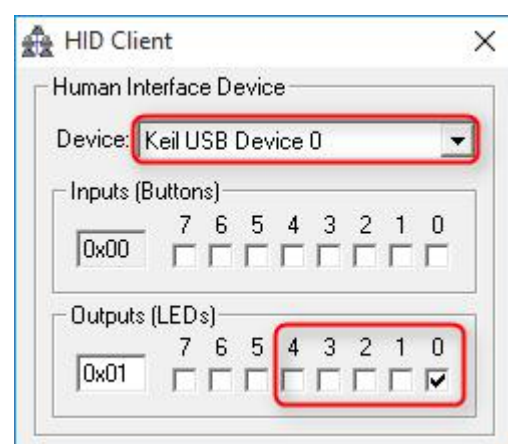
要测试 USB HID 设备的功能，请运行 HIDClient.exe 实用程序，并按照下列步骤操作：

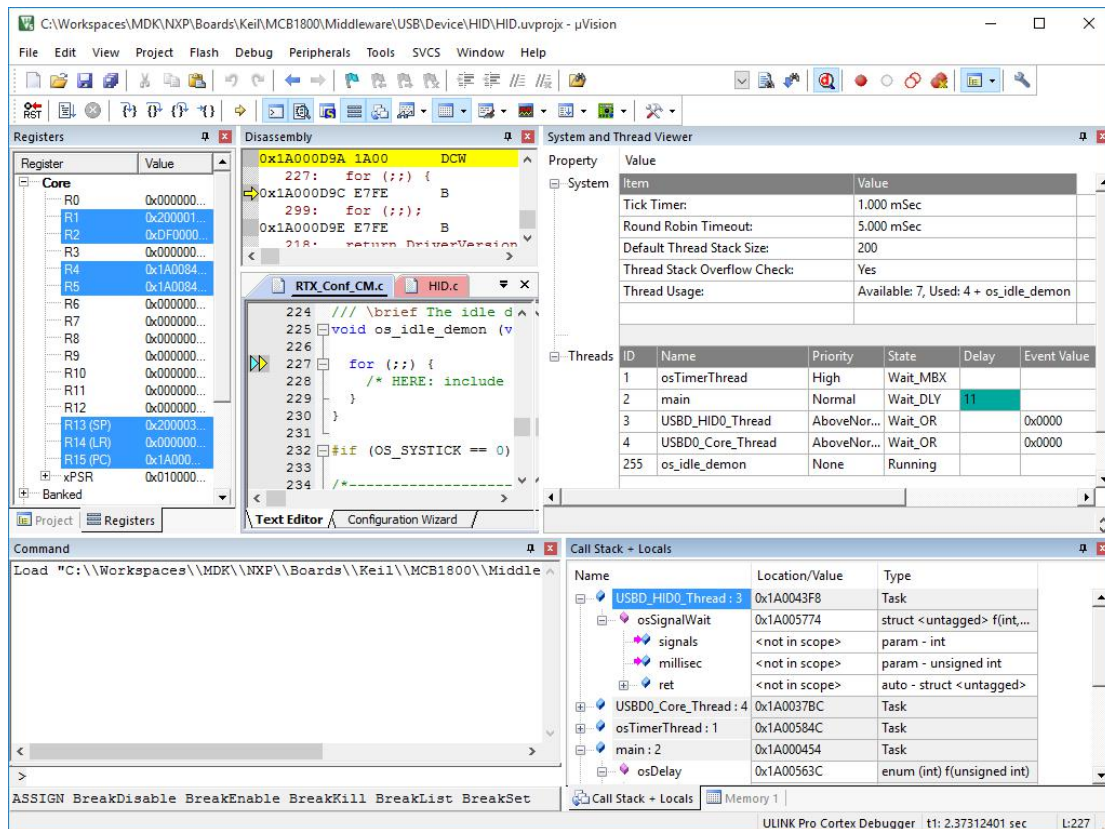
§ 选择设备建立通信通道。在我们的例子中，它是“Keil USB Device 0”。

§ 通过更改输出（LED）(Outputs (LEDs)) 复选框来测试应用程序。相应的 LED 将在开发板上相应地切换。

如果连接到开发板时遇到问题，可以使用调试器查找根本原因。

 从工具栏中选择开始/停止调试会话 **Start/Stop Debug Session**。





使用调试窗口缩小问题。断点帮助您停止某些代码行，以便您可以检查变量内容。

注意

调试通信协议可能很困难。当启动调试器或使用断点时，通信协议超时可能会超出使得难以调试应用程序。因此，请小心使用断点。

如果 USB 通信失败，请断开 USB，重置目标硬件，运行应用程序，并将其重新连接到 PC。

完



英文水平有限，有错误请查看原版。

或反馈到 imyumeng@qq.com